

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ



ISO 9001:2015

LIỄU KIẾN AN

THIẾT KẾ VÀ PHÁT TRIỂN HỆ THỐNG
QUẢN LÝ CÔNG VIỆC CÁ NHÂN
DỰA TRÊN NỀN TẢNG WEB

ĐỒ ÁN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN

VĨNH LONG, NĂM 2026

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ

THIẾT KẾ VÀ PHÁT TRIỂN HỆ THỐNG
QUẢN LÝ CÔNG VIỆC CÁ NHÂN
DỰA TRÊN NỀN TẢNG WEB

ĐỒ ÁN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN

Sinh viên: **LIỄU KIẾN AN**

Lớp: **DA22TTD**

MSSV: **110122028**

GVHD: **GVC. ThS. NGUYỄN KHẮC QUỐC**

VĨNH LONG, NĂM 2026

BẢN NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP

(Dành cho người hướng dẫn)

Ngành: Công nghệ thông tin.

Họ và tên người hướng dẫn: Nguyễn Khắc Quốc

Đơn vị công tác: Đại học Trà Vinh

Họ và tên sinh viên: Liễu Kiện An

MSSV: 110122028

Tên đề tài: Thiết kế và phát triển hệ thống quản lý công việc cá nhân dựa trên nền tảng web.

1. Tinh thần, thái độ học tập, nghiên cứu của sinh viên:

.....

.....

.....

2. Khả năng nghiên cứu khoa học:

.....

.....

.....

3. Hình thức và nội dung của đồ án:

.....

.....

.....

4. Kết luận:

.....

.....

Vĩnh Long, ngày tháng năm 2026
Người hướng dẫn

Nguyễn Khắc Quốc

LỜI CAM ĐOAN

Tôi xin cam đoan rằng đồ án tốt nghiệp với đề tài “Thiết kế và phát triển hệ thống quản lý công việc cá nhân dựa trên nền tảng web” là kết quả nghiên cứu, tìm hiểu và thực hiện của bản thân. Toàn bộ quá trình phân tích, thiết kế, xây dựng hệ thống, triển khai các chức năng và hoàn thiện báo cáo được thực hiện trên cơ sở kiến thức đã được học tại trường kết hợp với quá trình tự nghiên cứu, tham khảo các tài liệu có liên quan.

Trong quá trình thực hiện đề tài, tôi có sử dụng một số tài liệu, công trình nghiên cứu, sách, bài báo khoa học và các nguồn tham khảo trên Internet nhằm phục vụ cho việc nghiên cứu cơ sở lý thuyết, lựa chọn giải pháp công nghệ và xây dựng hệ thống. Tất cả các tài liệu, hình ảnh, số liệu và nội dung tham khảo đều được trích dẫn đầy đủ, rõ ràng theo đúng quy định. Những kết quả nghiên cứu, đánh giá và sản phẩm phần mềm được trình bày trong báo cáo là trung thực, phản ánh đúng quá trình thực hiện đề tài và chưa từng được sử dụng để bảo vệ bất kỳ học vị hay công trình nghiên cứu nào trước đây.

Vĩnh Long , ngày tháng năm 2026

Sinh viên thực hiện

Liều Kiện An

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Nguyễn Khắc Quốc, giảng viên hướng dẫn, đã tận tình chỉ bảo, định hướng và luôn đồng hành cùng em trong suốt quá trình thực hiện đề tài. Những kiến thức chuyên môn cùng những góp ý quý báu của thầy là nền tảng quan trọng giúp em hoàn thành khóa luận.

Em xin chân thành cảm ơn Ban Giám hiệu Đại học Trà Vinh cùng quý thầy cô Trường Kỹ thuật và Công nghệ đã truyền đạt những kiến thức bổ ích và tạo điều kiện thuận lợi để em học tập, nghiên cứu và hoàn thành chương trình đào tạo.

Em cũng xin gửi lời cảm ơn đến gia đình, bạn bè và các bạn sinh viên đã luôn động viên, hỗ trợ và đóng góp ý kiến trong quá trình thử nghiệm hệ thống.

Mặc dù đã nỗ lực hoàn thành khóa luận, do kiến thức và kinh nghiệm còn hạn chế nên đề tài khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để khóa luận được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Sinh viên thực hiện

Liều Kiện An

MỤC LỤC

LỜI CAM ĐOAN.....	ii
LỜI CẢM ƠN.....	iii
MỤC LỤC.....	iv
DANH MỤC HÌNH ẢNH.....	x
DANH MỤC BẢNG BIỂU.....	xii
DANH MỤC TỪ VIẾT TẮT.....	xiii
TÓM TẮT ĐỒ ÁN.....	xiv
MỞ ĐẦU.....	xvi
CHƯƠNG 1. TỔNG QUAN NGHIÊN CỨU	1
1.1. Đặt vấn đề	1
1.2. Mục đích nghiên cứu	2
1.3. Nội dung nghiên cứu.....	3
1.4. Đối tượng và phạm vi nghiên cứu	5
1.4.1 Đối tượng nghiên cứu	5
1.4.2 Phạm vi nghiên cứu	6
1.5. Đối tượng và phạm vi nghiên cứu	6
1.5.1 Phạm vi nghiên cứu	6
1.5.2 Phương pháp nghiên cứu thực nghiệm	7
1.6. Đóng góp của đề tài	8
CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT	10
2.1. Tổng quan kiến trúc Client-Server 3 tầng.....	10
2.2. Cơ sở lý thuyết về luồng dữ liệu và quản lý trạng thái.....	11
2.3. Khái niệm về API	12

2.4. Cấu trúc request và response trong API.....	14
2.5. Kiến trúc RESTful API.....	16
2.5.1 Tổng quan về kiến trúc RESTful API.....	16
2.5.2 Ưu điểm của RESTful API	18
2.5.3 Nhược điểm của RESTful API.....	19
2.6. Tổng quan về Node.js	20
2.6.1 Tìm hiểu Node.js	20
2.6.2 Các đặc điểm kỹ thuật của Node.js	20
2.6.3 Ưu điểm của Node.js	22
2.6.4 Nhược điểm của Node.js.....	22
2.6.5 Vai trò của Node.js trong hệ thống	23
2.7. Tổng quan về Express.js	23
2.7.1 Tìm hiểu Express.js	23
2.7.2 Các đặc điểm chính của Express.js	24
2.7.3 Cấu trúc project với Express.js	25
2.8. Hệ quản trị cơ sở dữ liệu MySQL	26
2.8.1 Tổng quan MySQL.....	26
2.8.2 Đặc điểm kỹ thuật của hệ quản trị cơ sở dữ liệu MySQL	26
2.8.3 Thiết kế cơ sở dữ liệu	27
2.8.4 Ưu điểm của MySQL.....	28
2.8.5 Nhược điểm của MySQL	29
2.9. Tổng quan về HTML, CSS và JavaScript	29
2.9.1 Tìm hiểu về HTML (HyperText Markup Language)	29
2.9.2 Tìm hiểu về CSS (Cascading Style Sheets).....	29

2.9.3 Tìm hiểu về JavaScript	29
2.9.4 Vai trò của HTML, CSS và JavaScript trong hệ thống	30
2.10. Giao diện Vue.js và Single Page Application (SPA)	30
2.11. Quản lý trạng thái toàn cục với Pinia	31
2.12. Socket.IO và Giao tiếp thời gian thực	31
2.13. Trí tuệ nhân tạo (AI) tạo sinh và mô hình Google Gemini.....	32
2.14. Cơ chế bảo mật JWT và mã hóa bcryptjs	32
CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU	34
3.1. Mô tả bài toán.....	34
3.1.1 Bối cảnh nghiên cứu	34
3.1.2 Thực trạng quản lý công việc	34
3.1.3 Mục tiêu	35
3.1.3.1 Đối với người dùng:.....	35
3.1.3.2 Đối với quản trị viên (Admin):.....	37
3.2. Phân tích hệ thống	38
3.2.1 Đặc tả yêu cầu hệ thống.....	38
3.2.1.1 Yêu cầu chức năng.....	38
3.2.1.2 Đối với quản trị viên.....	39
3.2.2 Kiến trúc hệ thống	40
3.2.3 Các tầng thành phần chính trong kiến trúc	41
3.2.4 Mô tả luồng xử lý dữ liệu.....	42
3.3. Phân tích thiết kế hệ thống.....	44
3.3.1 Sơ đồ UseCase.....	45
3.3.2 Tác nhân admin	47

3.3.3 Tác nhân người dùng	49
3.3.4 Tác nhân khách.....	51
3.4. Sơ đồ website	53
3.5. Cơ sở dữ liệu	54
3.5.1 Mô hình ERD	55
3.5.2 Bảng user (Người dùng)	56
3.5.3 Bảng công việc chính.....	57
3.5.4 Bảng subtasks (Nhiệm vụ con)	58
3.5.5 Bảng task_comments (Bình luận)	59
3.5.6 Bảng task_attachment (Tập đính kèm công việc)	60
3.5.7 Bảng task_dependencies (Ràng buộc phụ thuộc)	61
3.5.8 Bảng task_activities (Nhật ký hệ thống).....	62
CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU	64
4.1. Môi trường phát triển.....	64
4.1.1 Công cụ phát triển phần mềm	64
4.1.2 Cấu hình môi trường hệ thống	65
4.1.3 Cài đặt các nền tảng thực thi (Runtimes).....	66
4.1.4 Cấu hình biến môi trường và bảo mật	66
4.2. Triển khai phân hệ Frontend (Vue 3 Application).....	67
4.2.1 Tổ chức cấu trúc dự án	67
4.2.2 Triển khai các tính năng kỹ thuật trọng tâm	68
4.2.3 Thiết kế đáp ứng và tối ưu hóa giao diện (Responsive Design)	69
4.3. Triển khai phân hệ Backend Node.js & Express (Core & AI Service)	69
4.3.1 Cấu trúc tổ chức dự án.....	70

4.3.2 Triển khai phân hệ AI Service (Tích hợp trong Backend Express)	70
4.3.3 Tổ chức API và gọi Google Generative AI (Gemini SDK)	71
4.3.4 Các chức năng thông minh và Cơ chế dự phòng (Fallbacks)	71
4.4. Triển khai lưu trữ và quản lý tài nguyên đính kèm	75
4.4.1 Xử lý tải tệp tin bằng Multer và lưu trữ trên Server.....	75
4.4.2 Quản lý liên kết đính kèm của Task	75
4.5. Tích hợp và giao tiếp hệ thống (System Integration)	76
4.6. Kiểm thử hệ thống	77
4.6.1 Kiểm thử đơn vị.....	77
4.6.2 Kiểm thử tích hợp.....	77
4.6.3 Kiểm thử chức năng.....	78
4.6.4 Kiểm thử hiệu năng	80
4.6.5 Kiểm thử bảo mật	81
4.7. Kết quả triển khai hệ thống.....	81
4.7.1 Giao diện trang chủ.....	81
4.7.2 Giao diện trang đăng ký	82
4.7.3 Giao diện trang đăng nhập	83
4.7.4 Giao diện trang dashboard người dùng	83
4.7.5 Giao diện trang quản lý điểm số.....	84
4.7.6 Giao diện trang nhiệm vụ.....	85
4.7.7 Giao diện trang dự án	86
4.7.8 Giao diện trang quản lịch học	87
4.7.9 Giao diện trang mục tiêu.....	87
4.7.10 Giao diện trang ghi chú.....	88

4.7.11 Giao diện trang báo cáo	89
4.7.12 Giao diện trang hồ sơ.....	90
4.7.13 Giao diện trang dashboard admin.....	90
4.7.14 Giao diện quản lý người dùng.....	92
4.7.15 Giao diện trang quản lý dự án	93
4.7.16 Giao diện trang quản lý danh mục.....	93
4.7.17 Giao diện trang quản lý thông báo	94
4.7.18 Giao diện trang quản lý thông báo	95
4.7.19 Giao diện trang quản lý thông báo	96
4.7.20 Giao diện trang quản lý thông báo	97
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	98
5.1. Kết luận.....	98
5.2. Hướng phát triển.....	99
5.2.1 Phát triển ứng dụng di động đa nền tảng	100
5.2.2 Tích hợp công nghệ NFC và mã QR cho quản lý thực tế	100
5.2.3 Cảnh báo thông minh dựa trên dữ liệu lịch sử cá nhân	100
5.2.4 Mở rộng khả năng quản lý đa tổ chức và nhóm quy mô lớn	100
5.2.5 Phát triển API mở và tích hợp hệ sinh thái bên thứ ba.....	101
5.2.6 Nâng cấp kiến trúc hệ thống và triển khai điện toán đám mây.....	101
5.2.7 Bổ sung công cụ phân tích trực quan nâng cao.....	101

DANH MỤC HÌNH ẢNH

Hình 2.1: Tổng quan kiến trúc Client-Server 3 tầng.....	10
Hình 3.1: Kiến trúc hệ thống.....	38
Hình 3.2: Sơ đồ UseCase.....	43
Hình 3.3: Tác nhân admin.....	45
Hình 3.4: Tác nhân người dùng.....	47
Hình 3.5: Tác nhân khách.....	49
Hình 3.6: Sơ đồ website.....	51
Hình 3.7: Mô hình ERD.....	55
Hình 4.1: Giao diện trang chủ.....	77
Hình 4.2: Giao diện trang đăng ký.....	78
Hình 4.3: Giao diện trang đăng nhập	78
Hình 4.4: Giao diện trang dashboard.....	79
Hình 4.5: Giao diện trang quản lý điểm số	81
Hình 4.6: Giao diện trang nhiệm vụ.....	81
Hình 4.7: Giao diện trang dự án cá nhân	81
Hình 4.8: Giao diện trang lịch tự học.....	82
Hình 4.9: Giao diện trang mục tiêu.....	83
Hình 4.10: Giao diện trang quản lý thông báo.....	84
Hình 4.11: Giao diện trang nhật ký hoạt động.....	84
Hình 4.12: Giao diện trang cài đặt.....	85
Hình 4.13: Giao diện trang dashboard admin.....	86
Hình 4.14: Giao diện trang quản lý người dùng.....	87

Hình 4.15: Giao diện trang quản lý dự án.....	88
Hình 4.16: Giao diện trang quản lý danh mục.....	89
Hình 4.17: Giao diện trang quản lý thông báo.....	90
Hình 4.18: Giao diện trang nhật ký hoạt động.....	91
Hình 4.19: Giao diện trang cài đặt hệ thống.....	92
Hình 4.20: Giao diện trang xuất dữ liệu.....	92

DANH MỤC BẢNG BIỂU

Bảng 2.1: Các phương thức HTTP để thực hiện thao tác dữ liệu.....	17
Bảng 2.2: Thư viện Node.js được sử dụng trong hệ thống quản lý công việc.....	20
Bảng 2.3: Một số middleware được sử dụng trong hệ thống	23
Bảng 2.4: Các bảng chính để lưu trữ và quản lý thông tin trong Taskflow	25
Bảng 3.1: Các tác nhân của hệ thống	42
Bảng 3.2: Trường dữ liệu tập hợp người dùng (User)	53
Bảng 3.3: Chi tiết các trường dữ liệu bảng công việc	54
Bảng 3.4: Chi tiết các trường dữ liệu bảng nhiệm vụ con	55
Bảng 3.5: Chi tiết các trường dữ liệu bảng bình luận.....	56
Bảng 3.6: Chi tiết các trường dữ liệu bảng teepk đính kèm	56
Bảng 3.7: Chi tiết các trường dữ liệu bảng ràng buộc phụ thuộc	57
Bảng 3.8: Chi tiết các trường dữ liệu bảng nhật ký hệ thống	58
Bảng 4.1: Các công cụ dùng cho lập trình	60
Bảng 4.2: Cấu hình môi trường hệ thống	61
Bảng 4.3: Kiểm thử chức năng	73

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
AI	Trí tuệ nhân tạo (Artificial Intelligence)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
BSON	Binary JSON
CSR	Render phía client (Client_Side Rendering)
GPS	Hệ thống định vị toàn cầu (Global Positioning System)
HTTP/HTTPS	Giao thức truyền tải siêu văn bản / Bảo mật
ISR	Tái tạo tĩnh tăng dần (Incremental Static Regeneration)
JSON	Ký hiệu đối tượng javascript (JavaScript Object Notation)
JWT	JSON Web Token
LLM	Mô hình ngôn ngữ lớn (Large Language Models)
NPM	Trình quản lý gói node (Node Package Manager)
OOP	Lập trình hướng đối tượng (Object_Oriented Programming)
RAG	Tạo văn bản tăng cường truy xuất (Retrieval Augmented Generation)
RBAC	Kiểm soát truy cập dựa trên vai trò (Role_Based Access Control)
REST	Chuyển giao trạng thái đại diện (Representational State Transfer)
SSR	Render phía server (Server_Side Rendering)
UI	Giao diện người dùng (User Interface)
UX	Trải nghiệm người dùng (User Experience)

TÓM TẮT ĐỒ ÁN

Vấn đề nghiên cứu

Trong kỷ nguyên số hóa, việc quản lý thời gian và công việc hiệu quả đã trở thành yếu tố then chốt đối với sinh viên và người đi làm. Tuy nhiên, tại Việt Nam, nhiều người vẫn duy trì thói quen ghi chép thủ công trên sổ tay hoặc sử dụng các ứng dụng rời rạc, dẫn đến tình trạng thông tin bị phân mảnh, khó theo dõi tiến độ và dễ bỏ sót các nhiệm vụ quan trọng. Sinh viên thường gặp khó khăn trong việc cân đối giữa lịch học, theo dõi deadline bài tập và quản lý điểm số (GPA).

Thực trạng hiện nay cho thấy các công cụ quản lý thông thường chỉ đóng vai trò ghi chép tĩnh, thiếu tính tương tác thông minh và chưa có cơ chế tạo động lực duy trì thói quen. Vì vậy, việc nghiên cứu và phát triển một hệ thống quản lý công việc cá nhân tập trung, tích hợp trí tuệ nhân tạo (AI) và cơ chế trò chơi hóa (Gamification) là yêu cầu cấp thiết nhằm nâng cao năng suất cá nhân và xây dựng thói quen làm việc khoa học cho người dùng, đặc biệt là trong môi trường học tập tại Đại học Trà Vinh.

Hướng tiếp cận

Đề tài được xây dựng theo mô hình kiến trúc Client – Server ba tầng gồm tầng hiển thị (Presentation Layer), tầng xử lý nghiệp vụ (Application Layer) và tầng dữ liệu (Data Layer). Phần hệ Frontend được triển khai như một ứng dụng đơn trang (SPA) sử dụng framework Vue.js 3 kết hợp với Pinia để quản lý trạng thái, mang lại giao diện hiện đại theo phong cách Glassmorphism.

Phía Backend được phát triển bằng Node.js và Express.js nhằm xây dựng hệ thống RESTful API hiệu năng cao. Hệ thống sử dụng MySQL làm hệ quản trị cơ sở dữ liệu quan hệ để đảm bảo tính nhất quán và an toàn dữ liệu. Đặc biệt, đề tài tiếp cận hướng giải quyết thông minh bằng cách tích hợp Google Gemini API làm trợ lý ảo và Socket.io để hỗ trợ giao tiếp thời gian thực, cùng các thư viện hỗ trợ như Chart.js cho trực quan hóa dữ liệu và html2pdf.js để xuất báo cáo.

Cách giải quyết vấn đề

Đề tài đã xây dựng hệ thống TaskFlow với các trục tính năng cốt lõi nhằm giải quyết triệt để bài toán quản lý cá nhân. Hệ thống cung cấp các phương pháp quản trị

thời gian hiện đại như bảng kéo thả Kanban, ma trận ưu tiên Eisenhower, kế hoạch theo khung giờ (Timebox) và phòng tập trung Pomodoro tích hợp âm thanh lofi. Đối với sinh viên, hệ thống tích hợp phân hệ quản lý thời khóa biểu tự động sinh nhiệm vụ, quản lý điểm số GPA và thẻ ghi nhớ (Flashcards) để ôn tập.

Điểm nổi bật của giải pháp là việc ứng dụng Trí tuệ nhân tạo (AI) để tự động phân rã công việc phức tạp thành checklist chi tiết và tư vấn lộ trình học tập cá nhân hóa. Cơ chế Gamification được thiết kế dưới dạng hồ sơ nhập vai RPG, nơi người dùng tích lũy điểm kinh nghiệm (XP) và thăng cấp khi hoàn thành nhiệm vụ, giúp tạo động lực bền vững. Ngoài ra, hệ thống còn hỗ trợ quản lý dự án nhóm, thiết lập sơ đồ phụ thuộc công việc (Dependency Graph) và sử dụng Cron Job để gửi email nhắc nhở tự động khi sắp đến hạn.

Kết quả đạt được

Kết quả nghiên cứu đã hiện thực hóa thành công hệ thống TVU TaskFlow, đáp ứng toàn diện nhu cầu quản lý công việc và học tập. Hệ thống đã vượt qua các bài kiểm thử đa tầng từ kiểm thử đơn vị, kiểm thử tích hợp đến kiểm thử hiệu năng bằng Apache JMeter với tỷ lệ lỗi 0%, đảm bảo khả năng chịu tải đồng thời của nhiều người dùng.

Hệ thống cho phép người dùng theo dõi hiệu suất thông qua Dashboard trực quan, xuất báo cáo năng suất định dạng PDF và quản lý linh hoạt các tài liệu đính kèm. Về bảo mật, TaskFlow áp dụng cơ chế xác thực JWT, mã hóa mật khẩu bằng Bcrypt và các lớp Middleware bảo vệ giúp ngăn chặn các lỗ hổng như SQL Injection. Kết quả của đề tài không chỉ cung cấp một công cụ quản lý năng suất mạnh mẽ mà còn tạo tiền đề cho việc mở rộng sang các công nghệ mới như NFC, mã QR và chuyển đổi kiến trúc sang Microservices để phục vụ quy mô người dùng lớn hơn trong tương lai.

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và xu hướng số hóa toàn diện, việc quản lý công việc cá nhân hiệu quả đã trở thành yếu tố then chốt đối với mọi đối tượng trong xã hội hiện đại. Khối lượng công việc ngày càng gia tăng cùng với nhu cầu cân bằng giữa học tập, làm việc và cuộc sống cá nhân đặt ra yêu cầu cấp thiết về các công cụ hỗ trợ tổ chức, theo dõi và hoàn thành công việc một cách khoa học và có hệ thống. Tuy nhiên, nhiều người hiện nay vẫn duy trì thói quen quản lý công việc bằng cách ghi chép thủ công trên sổ tay, sử dụng các bảng tính excel rời rạc hoặc các ứng dụng đơn lẻ không liên kết với nhau, dẫn đến tình trạng thông tin bị phân mảnh, khó theo dõi tiến độ và dễ bị bỏ sót các nhiệm vụ quan trọng.

Việc thiếu một nền tảng tập trung để quản lý công việc cá nhân gây ra nhiều bất lợi trong thực tiễn. Học sinh, sinh viên thường gặp khó khăn trong việc theo dõi hạn chót bài tập, ôn tập kiến thức, quản lý điểm số và cân đối lịch học tập. Người đi làm phải đối mặt với thách thức trong việc sắp xếp ưu tiên giữa nhiều dự án, cuộc họp và các nhiệm vụ được giao. Các phương pháp quản lý thủ công không chỉ tiêu tốn nhiều thời gian mà còn không cung cấp được cái nhìn tổng quan về tiến độ, từ đó làm giảm năng suất và hiệu quả làm việc. Ngoài ra, việc theo dõi thói quen làm việc, phân tích hiệu suất cá nhân và tìm kiếm động lực duy trì sự tập trung cũng không thể thực hiện một cách thuận tiện.

Xuất phát từ thực tế đó, đề tài “Thiết kế và phát triển hệ thống quản lý công việc cá nhân trên nền tảng web” được lựa chọn nhằm nghiên cứu và phát triển một hệ thống quản lý công việc đa năng, hướng tới đối tượng cốt lõi là sinh viên và người đi làm. Hệ thống không chỉ cung cấp các phương pháp quản lý thời gian khoa học mà còn tích hợp các công cụ học tập thông minh, ứng dụng trí tuệ nhân tạo (AI) làm trợ lý đồng hành và ứng dụng cơ chế trò chơi hóa (Gamification) độc đáo, nhằm hỗ trợ người dùng nâng cao năng suất cá nhân, giảm thiểu rủi ro trễ hạn và xây dựng thói quen làm việc tích cực trong kỷ nguyên số.

2. Mục tiêu nghiên cứu

Đề tài được thực hiện với mục tiêu nghiên cứu và xây dựng một hệ thống quản lý công việc cá nhân toàn diện trên nền tảng web, đáp ứng nhu cầu học tập, làm việc và tự phát triển của cá nhân cũng như phối hợp nhóm. Hệ thống được thiết kế với các chức năng và giải pháp cốt lõi bao gồm:

Quản lý công việc khoa học: Ứng dụng các phương pháp quản trị thời gian hiện đại bao gồm bảng kéo thả kanban, lịch biểu tương tác (Calendar), ma trận ưu tiên eisenhower, kế hoạch chi tiết theo khung giờ (Timebox), và phòng tập trung pomodoro tích hợp âm thanh lofi thư giãn.

Công cụ hỗ trợ học tập chuyên sâu: Tích hợp phân hệ quản lý thời khóa biểu (tự động sinh nhiệm vụ), quản lý điểm số học tập (GPA), hệ thống ghi chú tích hợp sơ đồ tư duy (Mindmap) trực quan, và thẻ ghi nhớ (Flashcards) hỗ trợ ôn tập kiến thức hiệu quả.

Cơ chế trò chơi hóa (Gamification): Thiết kế hệ thống mô phỏng nhập vai RPG (RPG Profile) với cơ chế tích lũy điểm kinh nghiệm (XP), thăng cấp (Level) và mở khóa huy hiệu khi hoàn thành nhiệm vụ, tạo động lực duy trì thói quen tích cực cho người dùng.

Tích hợp trí tuệ nhân tạo (AI): Ứng dụng mô hình ngôn ngữ lớn để phát triển trợ lý AI Coach hỗ trợ tư vấn học tập/làm việc, tự động phân rã nhiệm vụ phức tạp thành checklist chi tiết, tạo nhanh công việc bằng ngôn ngữ tự nhiên, tự động sinh bộ câu hỏi ôn tập (Quiz) từ ghi chú, và phân tích sâu xu hướng trì hoãn công việc.

Hợp tác nhóm và liên lạc thời gian thực: Phát triển mô-đun quản lý dự án nhóm với thanh tiến độ trực quan, cột mốc (Milestones), sơ đồ phụ thuộc công việc (Dependency Graph) và phòng trò chuyện thời gian thực. Về mặt kỹ thuật, hệ thống được phát triển theo mô hình Client-Server với kiến trúc 3 tầng:

Frontend: Xây dựng bằng framework vue.js kết hợp pinia để quản lý trạng thái, chart.js để trực quan hóa số liệu trực quan, html2pdf.js hỗ trợ xuất báo cáo định dạng PDF, và ứng dụng capacitor làm nền tảng đóng gói thử nghiệm ứng dụng trên thiết bị di động (Android APK).

Backend: Sử dụng node.js và express.js để xây dựng hệ thống RESTful API hiệu năng cao, tích hợp socket.io truyền tải dữ liệu thời gian thực, nodemailer xử lý gửi thư thông báo nhắc nhở tự động, và kết nối tích hợp API của mô hình AI thế hệ mới (Google Gemini API).

Database: Sử dụng hệ quản trị cơ sở dữ liệu quan hệ mysql để quản lý dữ liệu người dùng, công việc, điểm số và các hoạt động của dự án một cách an toàn, bảo mật và nhất quán.

CHƯƠNG 1. TỔNG QUAN NGHIÊN CỨU

1.1. Đặt vấn đề

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin và xu hướng số hóa toàn diện, việc quản lý công việc và thời gian cá nhân hiệu quả đã trở thành nhu cầu thiết yếu đối với mọi đối tượng trong xã hội hiện đại. Khối lượng công việc ngày càng gia tăng cùng với nhu cầu cân bằng giữa học tập, làm việc và cuộc sống cá nhân đặt ra yêu cầu cấp thiết về các công cụ hỗ trợ tổ chức, theo dõi và hoàn thành công việc một cách khoa học và có hệ thống. Tuy nhiên, nhiều người hiện nay vẫn duy trì thói quen quản lý công việc bằng cách ghi chép thủ công trên sổ tay, sử dụng các bảng tính excel rời rạc hoặc các ứng dụng đơn lẻ không liên kết với nhau, dẫn đến tình trạng thông tin bị phân mảnh, khó theo dõi tiến độ và dễ bị bỏ sót các nhiệm vụ quan trọng.

Việc thiếu một nền tảng tích hợp để quản lý công việc cá nhân gây ra nhiều bất lợi trong thực tiễn. Sinh viên thường gặp khó khăn trong việc theo dõi deadline bài tập, bài lớn, khóa luận và lịch thi. Người đi làm phải đối mặt với thách thức trong việc sắp xếp ưu tiên giữa nhiều dự án, cuộc họp và các nhiệm vụ được giao. Các phương pháp quản lý thủ công không chỉ tiêu tốn nhiều thời gian mà còn không cung cấp được cái nhìn tổng quan về tiến độ, từ đó làm giảm năng suất và hiệu quả làm việc của bất kỳ ai. Ngoài ra, việc theo dõi lịch sử hoàn thành công việc, phân tích thói quen làm việc, lập kế hoạch dài hạn và điều chỉnh mục tiêu cá nhân cũng không thể thực hiện một cách thuận tiện.

Xuất phát từ thực tế đó, đề tài “Thiết kế và phát triển hệ thống quản lý công việc cá nhân trên nền tảng web” được lựa chọn nhằm nghiên cứu và phát triển một nền tảng số phổ quát, hướng đến đối tượng người dùng từ sinh viên, giảng viên đến người đi làm và người tự do. Hệ thống được xây dựng dựa trên nền tảng web, cho phép người dùng truy cập mọi lúc, mọi nơi trên nhiều thiết bị khác nhau. Hệ thống hướng đến việc cung cấp một công cụ trực quan, dễ sử dụng và tích hợp đầy đủ tính năng cần thiết, giúp người dùng nâng cao năng suất cá nhân, giảm thiểu rủi ro bỏ sót công việc và xây dựng thói quen làm việc hiệu quả trong môi trường số.

1.2. Mục đích nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu và xây dựng một hệ thống quản lý công việc cá nhân trên nền tảng web, phù hợp với nhu cầu đa dạng của mọi đối tượng người dùng, đặc biệt là sinh viên và người đi làm. Hệ thống được thiết kế để đáp ứng nhu cầu quản lý công việc của bất kỳ cá nhân nào với giao diện thân thiện và trải nghiệm người dùng tối ưu.

Trước hết, đề tài hướng đến việc hệ thống hóa các kiến thức lý thuyết về kiến trúc Client-Server, RESTful API, vue.js, pinia, socket.io, chart.js, node.js, express.js, mysql và các thư viện liên quan như JWT, bcrypt, nodemailer (Backend) và html2pdf.js (Frontend). Những kiến thức này sẽ làm nền tảng vững chắc cho quá trình xây dựng hệ thống. Tiếp theo, thông qua khảo sát thực tế người dùng, tác giả phân tích nhu cầu quản lý công việc hiện tại để xác định yêu cầu chức năng và thiết kế hệ thống phù hợp.

Về mặt sản phẩm, hệ thống cần đáp ứng được các chức năng chính như quản lý công việc với trạng thái rõ ràng, theo dõi tiến độ, phân loại theo danh mục và độ ưu tiên, đính kèm file và theo dõi lịch sử hoạt động. Đề tài ứng dụng các phương pháp quản lý hiện đại như kanban board để theo dõi trực quan trạng thái công việc, ma trận eisenhower để phân loại ưu tiên, lịch biểu và timebox planner để quản lý thời gian, pomodoro timer để nâng cao hiệu quả làm việc tập trung, và phương pháp OKR để lập kế hoạch dài hạn cùng theo dõi thói quen với habit tracker. Hệ thống tích hợp các biểu đồ phân tích hiệu suất và công nghệ xuất báo cáo định dạng PDF giúp người dùng đánh giá thói quen làm việc, từ đó điều chỉnh kế hoạch phù hợp.

Đề tài cũng tập trung vào việc xây dựng cơ chế quản lý dự án nhóm với tính năng chat thời gian thực, hệ thống gamification với XP, level và RPG Profile để tăng động lực người dùng, cùng trợ lý ảo AI (tích hợp Google Gemini API) để gợi ý và tự động phân rã công việc. Hệ thống hỗ trợ nhắc nhở tự động qua email và thông báo thời gian thực trên hệ thống, đảm bảo người dùng không bỏ lỡ các mốc quan trọng.

Về bảo mật, đề tài ứng dụng JWT và bcrypt để xác thực người dùng, mã hóa

mật khẩu và phân quyền truy cập theo vai trò. Cuối cùng, các dịch vụ nền tảng như gửi email thông báo, xuất báo cáo PDF (phía client) và trích xuất dữ liệu CSV (phía server) cũng được xây dựng để hỗ trợ người dùng trong công tác vận hành.

1.3. Nội dung nghiên cứu

Để đạt được các mục tiêu đã đề ra, tác giả tiến hành nghiên cứu, phân tích và xây dựng hệ thống theo các nội dung chính sau.

Thực hiện khảo sát nhu cầu sử dụng của sinh viên và người đi làm nhằm tìm hiểu thói quen quản lý công việc, học tập cũng như những khó khăn thường gặp trong quá trình lập kế hoạch và theo dõi tiến độ. Đồng thời, một số ứng dụng quản lý công việc phổ biến trên thị trường cũng được nghiên cứu để đánh giá ưu điểm, hạn chế và xác định các chức năng còn thiếu. Kết quả khảo sát là cơ sở để xác định các yêu cầu chức năng, phi chức năng và định hướng thiết kế hệ thống phù hợp với nhu cầu của người dùng Việt Nam.

Song song với quá trình khảo sát, tác giả nghiên cứu các kiến thức nền tảng phục vụ cho việc phát triển hệ thống. Về kiến trúc phần mềm, hệ thống được xây dựng theo mô hình Client–Server ba tầng kết hợp với RESTful API. Backend sử dụng node.js và express.js, trong đó tập trung nghiên cứu cơ chế event loop, nonblocking I/O, middleware, routing, xác thực bằng JWT, mã hóa mật khẩu với bcrypt và giao tiếp thời gian thực thông qua Socket.IO. Ở phía frontend, hệ thống được phát triển bằng vue.js (Vue 3 với Composition API), sử dụng pinia để quản lý trạng thái toàn cục, chart.js để trực quan hóa dữ liệu thống kê và socket.io client để đồng bộ dữ liệu theo thời gian thực. Cơ sở dữ liệu được thiết kế trên mysql theo mô hình quan hệ, áp dụng các nguyên tắc chuẩn hóa dữ liệu và tối ưu truy vấn nhằm đảm bảo tính toàn vẹn và hiệu năng của hệ thống.

Cơ sở dữ liệu của hệ thống bao gồm 29 bảng, được tổ chức thành bảy nhóm chức năng chính. Nhóm quản lý người dùng gồm các bảng users, friendships, activity_logs, categories và notifications. Nhóm quản lý công việc bao gồm tasks, subtasks, task_comments, task_activities, task_attachments và task_dependencies.

Nhóm quản lý dự án bao gồm `projects`, `project_members`, `project_chats` và `project_milestones`. Nhóm hỗ trợ học tập gồm `grades`, `class_schedules`, `flashcards`, `flashcard_decks` và `notes`. Nhóm quản lý kế hoạch học tập theo phương pháp OKR gồm `semester_plans`, `objectives`, `key_results`, `objective_tasks` và `subjects`. Nhóm quản lý thói quen và thời gian gồm `habits`, `habit_logs` và `time_logs`. Ngoài ra, hệ thống còn có bảng gamification phục vụ chức năng trò chơi hóa nhằm tăng động lực sử dụng. Tất cả các bảng được liên kết thông qua hệ thống khóa ngoại để đảm bảo tính nhất quán dữ liệu và hỗ trợ truy vấn hiệu quả.

Ở phía frontend, hệ thống được xây dựng theo mô hình single page application (SPA) bằng `vue.js`, kết hợp `vue router` và `pinia` để quản lý điều hướng và trạng thái ứng dụng. Giao diện được thiết kế theo hướng responsive với CSS Grid và flexbox, giúp hiển thị tốt trên cả máy tính và thiết bị di động. Hệ thống cung cấp đầy đủ các màn hình chức năng như dashboard thống kê, quản lý công việc, chi tiết công việc, mẫu công việc, bảng kanban kéo thả, quản lý dự án, lịch làm việc, thời khóa biểu, `timebox planner`, kế hoạch học kỳ theo OKR, ma trận `eisenhower`, `habit tracker`, phòng tập trung `pomodoro`, `flashcards`, quản lý điểm số, ghi chú, báo cáo thống kê, hồ sơ RPG, thùng rác, sơ đồ phụ thuộc công việc và chatbot hỗ trợ. Ngoài ra, cơ chế phân quyền được tích hợp để hiển thị các chức năng phù hợp với từng vai trò người dùng (User và admin).

Phía backend được phát triển bằng `node.js` và `express.js` theo mô hình MVC với hơn 50 API endpoint. Hệ thống bao gồm nhiều module chức năng như xác thực và quản lý người dùng (đăng ký, đăng nhập, quên mật khẩu, đổi mật khẩu, cập nhật hồ sơ và kết bạn); Quản lý công việc với đầy đủ các chức năng CRUD, phân loại, tìm kiếm, checklist, bình luận, tệp đính kèm, lịch sử hoạt động và liên kết phụ thuộc; Quản lý dự án gồm thành viên, mốc tiến độ và trò chuyện thời gian thực; Quản lý lịch biểu, thời khóa biểu và theo dõi thời gian làm việc; quản lý kế hoạch học tập theo OKR; Theo dõi thói quen với thống kê chuỗi ngày hoàn thành; Quản lý `flashcards`, điểm số và ghi chú học tập; Thống kê dữ liệu và xuất báo cáo PDF; Quản

trị hệ thống dành cho admin; Chatbot AI hỗ trợ gợi ý và phân rã công việc; Cùng với chức năng quản lý thùng rác và mẫu công việc.

Bên cạnh các chức năng chính, hệ thống còn tích hợp nhiều dịch vụ hỗ trợ nhằm nâng cao trải nghiệm người dùng và khả năng vận hành. Dịch vụ gửi email được xây dựng bằng nodemailer thông qua giao thức SMTP để gửi thông báo nhắc nhở và hỗ trợ khôi phục mật khẩu. Chức năng xuất báo cáo PDF được triển khai ở phía frontend bằng thư viện html2pdf.js, cho phép người dùng lưu các báo cáo thống kê trực tiếp từ trình duyệt. Hệ thống cũng hỗ trợ xuất dữ liệu CSV dành cho quản trị viên thông qua các API quản trị. socket.io được sử dụng để triển khai tính năng trò chuyện thời gian thực giữa các thành viên dự án và chatbot AI. Ngoài ra, cron job được cấu hình chạy định kỳ trên máy chủ nhằm kiểm tra các nhiệm vụ sắp đến hạn và tự động gửi email nhắc nhở đến người dùng.

Sau khi hoàn thành quá trình phát triển, hệ thống được kiểm thử qua nhiều giai đoạn nhằm đảm bảo tính ổn định và chính xác. Kiểm thử đơn vị được thực hiện đối với các chức năng cốt lõi như xác thực người dùng, phân quyền và xử lý nghiệp vụ. Kiểm thử tích hợp được sử dụng để đánh giá luồng xử lý giữa các API và các thành phần của hệ thống. Kiểm thử chức năng được tiến hành trên toàn bộ các tính năng chính như đăng ký, đăng nhập, quản lý công việc, quản lý dự án, học tập, theo dõi thói quen và xuất báo cáo. Trong suốt quá trình phát triển backend, postman được sử dụng để kiểm tra từng API endpoint, giúp đảm bảo dữ liệu trả về chính xác và các chức năng hoạt động đúng theo yêu cầu thiết kế.

1.4. Đối tượng và phạm vi nghiên cứu

1.4.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm các hoạt động quản lý công việc cá nhân và tập thể của người dùng như lập kế hoạch công việc, theo dõi tiến độ, phân loại ưu tiên, quản lý thời gian, theo dõi thói quen, lập kế hoạch dài hạn theo phương pháp OKR, quản lý dự án nhóm, trao đổi thời gian thực và phân tích hiệu suất làm việc. Bên cạnh đó là công tác hỗ trợ học tập với quản lý điểm số, lịch học, flashcards

và kế hoạch học tập. Đồng thời là công tác quản trị hệ thống trên nền tảng web, từ đó xác định những yêu cầu cải tiến và tính năng cần thiết.

Về mặt công nghệ, đề tài nghiên cứu các công nghệ sử dụng trong quá trình xây dựng hệ thống bao gồm: Vue.js, pinia, chart.js, socket.io, node.js, express.js, mysql, JWT, bcrypt, nodemailer, multer, cors, helmet, morgan, node-cron, html2pdf.js và API tích hợp trí tuệ nhân tạo (Google Gemini API). Về mặt người dùng, các tác nhân tham gia hệ thống gồm admin là quản trị viên hệ thống và user là người dùng cuối.

1.4.2 Phạm vi nghiên cứu

Phạm vi nghiên cứu của đề tài được giới hạn trong việc xây dựng các chức năng chính của hệ thống quản lý công việc cá nhân bao gồm quản lý công việc cá nhân (CRUD, trạng thái, ưu tiên, danh mục, checklist, bình luận, file đính kèm, lịch sử, phụ thuộc), quản lý dự án nhóm (thành viên, milestone, chat), quản lý thời gian (lịch, thời gian biểu, timebox, theo dõi thời gian), ma trận eisenhower, habit tracker, học tập với flashcards, điểm số, ghi chú và lịch học, kế hoạch OKR, thống kê và xuất báo cáo PDF/CSV, trò chuyện thời gian thực, hệ thống gamification, AI trợ lý ảo và cơ chế quản trị hệ thống.

Hệ thống tập trung vào việc quản lý công việc và thời gian cá nhân, chưa mở rộng sang các lĩnh vực quản lý khác như nhân sự hay bán hàng. Hệ thống được phát triển trên nền tảng web chạy đa thiết bị thông qua trình duyệt web. Ở giai đoạn này, hệ thống chưa phát triển ứng dụng di động native riêng biệt mà tập trung tối ưu hóa giao diện web responsive và chạy thử nghiệm đóng gói hybrid bằng capacitor. Việc kiểm thử được thực hiện trong môi trường phát triển cục bộ với bộ dữ liệu mẫu.

1.5. Đối tượng và phạm vi nghiên cứu

1.5.1 Phạm vi nghiên cứu

Để có cơ sở vững chắc cho quá trình xây dựng hệ thống, tác giả đã tiến hành thu thập, nghiên cứu và tổng hợp các tài liệu liên quan đến kiến trúc Client-Server, mô hình ba tầng, nguyên lý hoạt động và cơ chế giao tiếp giữa các tầng. Tác giả cũng

tìm hiểu về RESTful API với các nguyên tắc thiết kế, phương thức HTTP, cấu trúc request và response, cùng các mã trạng thái thường dùng. Về node.js và express.js, tác giả nghiên cứu mô hình bất đồng bộ, non-blocking i/o, cơ chế event loop và cách tổ chức mã nguồn theo mô hình MVC.

Về vue.js, tác giả nghiên cứu composition API, hệ thống reactive, vue router và pinia. Về mysql, tác giả tìm hiểu thiết kế cơ sở dữ liệu quan hệ, tối ưu truy vấn, bảo mật dữ liệu và tích hợp với node.js qua thư viện mysql2. Ngoài ra, tác giả cũng nghiên cứu cơ chế xác thực JWT, mã hóa mật khẩu bằng bcrypt, giao tiếp thời gian thực với socket.io, trực quan hóa với chart.js, cùng các thư viện hỗ trợ như nodemailer, helmet, cors, morgan, multer, dotenv, node-cron (backend) và html2pdf.js (frontend). Tìm hiểu các phương pháp quản lý công việc hiện đại như kanban, pomodoro, ma trận eisenhower và OKR, cùng các hệ thống quản lý công việc hiện có trên thị trường để áp dụng những điểm phù hợp vào đề tài.

1.5.2 Phương pháp nghiên cứu thực nghiệm

Quá trình thực nghiệm được tiến hành qua nhiều bước. Đầu tiên là khảo sát thực tế thông qua khảo sát người dùng để tìm hiểu thói quen quản lý công việc hiện tại, những khó khăn và mong muốn cải tiến. Từ đó xác định yêu cầu chức năng và thiết kế hệ thống phù hợp. Hệ thống được xây dựng theo quy trình phát triển phần mềm tuần tự (Waterfall Model), bao gồm các giai đoạn phân tích yêu cầu, thiết kế kiến trúc, xây dựng cơ sở dữ liệu, phát triển backend, phát triển frontend, tích hợp và kiểm thử. Mỗi giai đoạn đều được kiểm tra và điều chỉnh trước khi chuyển sang giai đoạn tiếp theo.

Việc kiểm thử được thực hiện bằng nhiều công cụ khác nhau. Postman được sử dụng để kiểm tra từng API, đảm bảo các endpoint hoạt động đúng chức năng và trả về dữ liệu chính xác. Kiểm thử hộp trắng và đen được tiến hành bao phủ các chức năng đăng nhập, đăng ký, quên mật khẩu, CRUD công việc, quản lý dự án, quản lý thời gian, quản lý thói quen, quản lý flashcards, điểm số, ghi chú, kế hoạch học tập, thống kê và báo cáo. Kiểm thử giao diện kiểm tra tính responsive và trải nghiệm người dùng. Kiểm thử tích hợp kiểm tra các luồng xử lý hoàn chỉnh giữa frontend và

backend. Sau mỗi vòng kiểm thử, kết quả được ghi nhận và các lỗi phát sinh được sửa chữa kịp thời.

1.6. Đóng góp của đề tài

Sau quá trình nghiên cứu và thực hiện, đề tài đã đạt được những kết quả thiết thực sau:

Trước hết, hệ thống quản lý công việc cá nhân TaskFlow được xây dựng thành công, cung cấp một nền tảng tích hợp và phổ quát cho người dùng, thay thế các phương pháp quản lý thủ công và các ứng dụng rời rạc trước đây. Hệ thống hỗ trợ đầy đủ các nghiệp vụ từ quản lý công việc cá nhân, quản lý dự án nhóm, theo dõi thời gian và thói quen, lập kế hoạch học tập/lập kế hoạch dài hạn đến phân tích hiệu suất và báo cáo thống kê.

Những phương pháp quản lý hiện đại được tích hợp đồng bộ vào hệ thống gồm kanban board, ma trận eisenhower, pomodoro timer, phương pháp OKR và habit tracker, giúp người dùng tiếp cận nhiều cách thức tổ chức công việc khác nhau trong cùng một nền tảng. Hệ thống gamification với XP, level và RPG profile được xây dựng để tăng động lực và tạo trải nghiệm làm việc thú vị. Cơ chế chat thời gian thực qua socket.io cho phép người dùng trao đổi và phối hợp làm việc nhóm hiệu quả. AI trợ lý ảo được tích hợp giúp tự động hóa quy trình phân rã nhiệm vụ và tương tác hỏi đáp thông minh.

Về mặt kiến trúc, hệ thống được thiết kế theo mô hình Client-Server ba tầng với các công nghệ phổ biến như vue.js, node.js, express, mysql, giúp dễ dàng bảo trì, mở rộng và nâng cấp trong tương lai. Cơ chế phân quyền chặt chẽ giữa quản trị viên và người dùng đảm bảo an toàn dữ liệu và kiểm soát truy cập phù hợp. Ngoài các chức năng dành cho người dùng cuối, hệ thống còn cung cấp các công cụ quản trị như quản lý người dùng, quản lý danh mục, quản lý thông báo, theo dõi nhật ký hoạt động và cấu hình hệ thống, đáp ứng tốt nhu cầu vận hành thực tế.

Hệ thống hỗ trợ xuất báo cáo dưới dạng PDF trực tiếp từ trình duyệt và trích xuất dữ liệu quản trị sang định dạng CSV. Đề tài góp phần thúc đẩy quá trình chuyển

đổi số trong công tác quản lý thời gian và nâng cao năng suất cá nhân, mở ra hướng tiếp cận mới trong việc ứng dụng công nghệ thông tin vào giải quyết các bài toán quản lý công việc thực tiễn trong môi trường học tập và làm việc hiện đại.

CHƯƠNG 2. NGHIÊN CỨU LÝ THUYẾT

2.1. Tổng quan kiến trúc Client-Server 3 tầng

Kiến trúc Client-Server là một mô hình phân tán, trong đó phía client chịu trách nhiệm tiếp nhận tương tác từ người dùng và gửi các yêu cầu (Request) đến server. Sau khi tiếp nhận yêu cầu, server sẽ thực hiện các thao tác xử lý cần thiết như tính toán, xác thực, truy xuất hoặc cập nhật dữ liệu, sau đó trả kết quả (Response) về cho client. Mô hình này giúp tách biệt rõ ràng giữa phần giao diện người dùng và phần xử lý nghiệp vụ, từ đó nâng cao khả năng mở rộng, bảo trì và phát triển hệ thống so với các ứng dụng truyền thống.

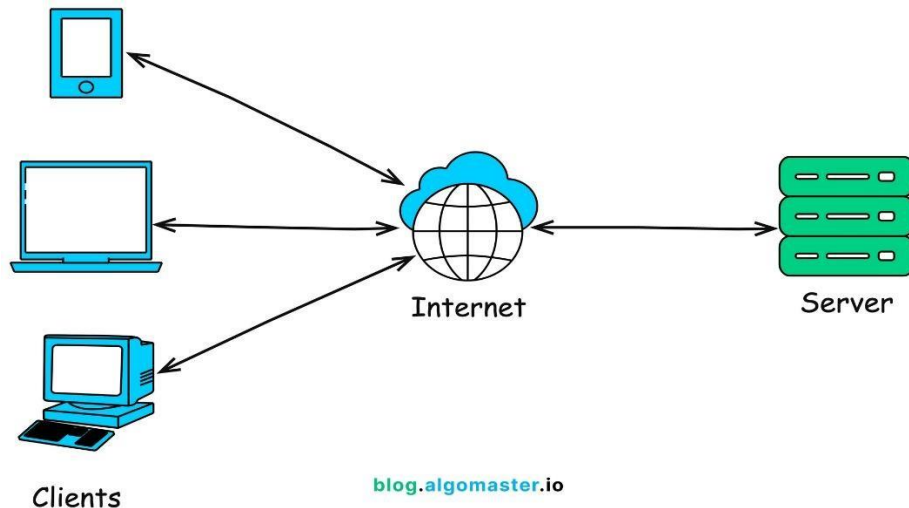
Ứng dụng web quản lý công việc và quy trình làm việc (TaskFlow) được xây dựng dựa trên kiến trúc Client-Server theo mô hình ba tầng (Three-tier Architecture). Kiến trúc này phân chia hệ thống thành ba lớp độc lập, mỗi lớp đảm nhiệm một vai trò riêng và giao tiếp với nhau thông qua các giao thức chuẩn.

Lớp giao diện (Presentation Tier-Client/Frontend): Đây là lớp tương tác trực tiếp với người dùng, được xây dựng bằng vue.js theo mô hình single page application (SPA). Lớp này chịu trách nhiệm hiển thị giao diện, tiếp nhận các thao tác của người dùng, gửi yêu cầu đến máy chủ thông qua HTTP hoặc websocket và hiển thị kết quả phản hồi nhận được từ backend.

Lớp xử lý nghiệp vụ (Application Tier – Application Server/Backend): Lớp này được phát triển bằng node.js và express.js, đóng vai trò là trung tâm xử lý của hệ thống. Backend tiếp nhận các yêu cầu từ frontend, thực hiện các nghiệp vụ như xác thực người dùng, phân quyền, xử lý logic, kiểm tra tính hợp lệ của dữ liệu, đồng thời giao tiếp với cơ sở dữ liệu để truy xuất hoặc cập nhật thông tin trước khi trả kết quả về cho client.

Lớp dữ liệu (Data Tier – Database Server): Đây là lớp chịu trách nhiệm lưu trữ và quản lý toàn bộ dữ liệu của hệ thống bằng mysql. Các thao tác thêm, sửa, xóa và truy vấn dữ liệu đều được thực hiện thông qua backend nhằm đảm bảo tính toàn vẹn, bảo mật và nhất quán của dữ liệu.

Việc áp dụng kiến trúc Client-Server ba tầng giúp hệ thống TaskFlow có cấu trúc rõ ràng, giảm sự phụ thuộc giữa các thành phần, đồng thời tạo điều kiện thuận lợi cho việc bảo trì, mở rộng chức năng và triển khai trên nhiều nền tảng khác nhau trong tương lai.



Hình 2.1: Tổng quan kiến trúc Client-Server 3 tầng

2.2. Cơ sở lý thuyết về luồng dữ liệu và quản lý trạng thái

Để hiện thực hóa một môi trường ứng dụng web tương tác mượt mà, hệ thống cần xử lý đồng thời nhiều luồng logic nghiệp vụ và đồng bộ dữ liệu liên tục trong suốt phiên làm việc của người dùng.

Giao tiếp API và Xử lý luồng: Mọi tương tác trong ứng dụng không chỉ đơn thuần là thay đổi hình ảnh trực quan mà dựa trên các luồng gửi/nhận dữ liệu. Client (Frontend) là nơi khởi tạo các yêu cầu từ thao tác người dùng, trong khi server (Backend) là nơi tiếp nhận, xác thực dữ liệu. Khi hệ thống xử lý thành công một yêu cầu API giữa hai đầu mút này, một sự kiện cập nhật cơ sở dữ liệu (Database Transaction) sẽ diễn ra, dẫn đến việc thay đổi trạng thái của thực thể (công việc, dự án).

Vòng đời component và quản lý trạng thái (State Management): Một thao tác tương tác cơ bản (ví dụ: kéo thả hoặc cập nhật trạng thái công việc) bao gồm nhiều pha: tiếp nhận thao tác, hiển thị trạng thái chờ (loading) và render lại giao diện. Chuỗi hành động này được lập trình để đồng bộ hóa giữa việc phản hồi trực quan trên màn

hình và việc xử lý dữ liệu bất đồng bộ (Asynchronous) dưới nền. Đồng thời, các tính năng đều đi kèm với cơ chế lưu trữ tập trung (thông qua pinia) nhằm tối ưu hóa hiệu năng và giảm tải số lần gọi API không cần thiết.

Phân quyền và Vai trò (RBAC): Hành vi và quyền hạn của người dùng được xây dựng dựa trên mô hình logic phân quyền theo vai trò (Role-based access control). Sơ đồ phân quyền này cho phép hệ thống nhận diện được “ngữ cảnh” của người dùng hiện tại, từ đó đưa ra quyết định cho phép cập nhật tiến độ, chỉnh sửa hoặc giới hạn vùng hiển thị dữ liệu dựa trên vai trò của họ trong hệ thống (Admin/User) hoặc vị thế của họ trong dự án nhóm.

Hệ thống tự động hóa và tương tác sự kiện: Bên cạnh các thao tác chủ động của người dùng, hệ thống còn tích hợp các luồng xử lý tự động ngầm. Các luồng này được xây dựng dựa trên hai thành phần logic chính:

Logic nghiệp vụ (Business Logic) quy định cách hệ thống vận hành, chẳng hạn như tự động cập nhật tiến độ phần trăm dự án khi một công việc con hoàn thành, hoặc tính toán các mốc thời gian trễ hạn nhằm tạo ra một hệ thống báo cáo sống động và chính xác.

Logic sự kiện (Event Logic) chịu trách nhiệm quản lý luồng tương tác giữa hệ thống và người dùng. Thành phần này hỗ trợ triển khai các thông báo cảnh báo, hiển thị hộp thoại xác nhận thao tác và xử lý các tệp tin xuất ra (như file PDF). Các sự kiện sẽ được kích hoạt tự động dựa trên những điều kiện cụ thể đã được lập trình trước, chẳng hạn như một dự án vừa chuyển sang trạng thái “Hoàn thành” hoặc tiến trình công việc đạt 100%.

2.3. Khái niệm về API

API (Application Programming Interface) là giao diện lập trình ứng dụng, được sử dụng để cho phép các phần mềm hoặc hệ thống khác nhau có thể giao tiếp và trao đổi dữ liệu với nhau. Có thể hiểu API như một cầu nối trung gian giúp các ứng dụng gửi yêu cầu và nhận dữ liệu mà không cần can thiệp trực tiếp vào cơ sở dữ liệu hay cấu trúc xử lý bên trong hệ thống.

Trong quá trình phát triển các ứng dụng web hiện nay, API giữ vai trò rất quan trọng vì giúp tách biệt giữa phần giao diện người dùng và phần xử lý dữ liệu phía máy chủ. Nhờ đó, hệ thống có khả năng hoạt động linh hoạt hơn, dễ bảo trì và thuận tiện trong việc mở rộng sau này.

Đối với hệ thống quản lý công việc (TaskFlow), API được sử dụng để kết nối giữa giao diện website (Frontend) và máy chủ xử lý dữ liệu (Backend). Khi người dùng thực hiện các thao tác trên hệ thống, dữ liệu sẽ được gửi thông qua API để xử lý và trả kết quả về giao diện hiển thị. Một số chức năng chính được thực hiện thông qua API trong đề tài gồm:

Tra cứu danh sách công việc (tasks) và các công việc con (subtasks).

Thêm mới, cập nhật trạng thái, tiến độ, hoặc xóa công việc (chuyển vào thùng rác).

Quản lý tệp đính kèm (attachments) và bình luận, trao đổi (comments) trong từng công việc.

Giao tiếp với trợ lý ảo AI để hỗ trợ phân tích và gợi ý công việc.

Trích xuất dữ liệu thống kê (stats) báo cáo tổng quan.

Đăng nhập, xác thực và phân quyền hệ thống (người dùng/quản trị viên).

Trong đề tài này, hệ thống sử dụng REST API (Representational State Transfer) để xây dựng cơ chế giao tiếp dữ liệu giữa frontend và backend. REST API hoạt động dựa trên giao thức HTTP và thường sử dụng các phương thức như GET, POST, PUT và DELETE tương ứng với các thao tác truy xuất, thêm mới, cập nhật và xóa dữ liệu.

Dữ liệu trao đổi giữa các thành phần trong hệ thống được lưu dưới định dạng JSON vì có cấu trúc đơn giản, dễ xử lý và tương thích với nhiều nền tảng khác nhau. Hiện nay, REST API là mô hình được sử dụng phổ biến trong phát triển website và hệ thống quản lý nhờ khả năng triển khai đơn giản, tốc độ xử lý nhanh và dễ dàng tích hợp với các công nghệ hiện đại.

2.4. Cấu trúc request và response trong API

Trong hệ thống quản lý công việc (TaskFlow), quá trình trao đổi dữ liệu giữa giao diện website và máy chủ được thực hiện thông qua cơ chế request – response của RESTful API. Khi người dùng thao tác trên hệ thống, chẳng hạn như xem danh sách công việc, thêm mới task hoặc cập nhật trạng thái, giao diện frontend sẽ gửi yêu cầu đến backend thông qua các API tương ứng. Sau khi tiếp nhận và xử lý dữ liệu, máy chủ sẽ trả kết quả về để hiển thị cho người dùng.

Mỗi request gửi lên hệ thống thường gồm các thành phần chính như endpoint, phương thức HTTP, header và dữ liệu body.

Endpoint là đường dẫn dùng để xác định chức năng hoặc tài nguyên cần thao tác. Ví dụ:

/api/tasks: lấy danh sách công việc hoặc tạo mới công việc.

/api/tasks/:id: lấy thông tin chi tiết, cập nhật hoặc xóa một công việc cụ thể.

/api/auth/login: xử lý chức năng đăng nhập hệ thống.

Phương thức HTTP cho biết loại thao tác mà người dùng muốn thực hiện với dữ liệu:

GET: dùng để lấy dữ liệu (ví dụ: danh sách task, danh mục).

POST: dùng để thêm mới dữ liệu (ví dụ: tạo task mới, thêm file đính kèm).

PUT/PATCH: dùng để cập nhật dữ liệu (ví dụ: thay đổi trạng thái, độ ưu tiên của task).

DELETE: dùng để xóa dữ liệu (ví dụ: xóa task, xóa bình luận).

Header chứa các thông tin đi kèm request, chẳng hạn như kiểu dữ liệu được gửi lên hoặc thông tin xác thực người dùng. Trong hệ thống này, dữ liệu được trao đổi dưới định dạng JSON và sử dụng JWT (JSON Web Token) trong header Authorization (Bearer Token) để kiểm tra quyền truy cập API.

Body là phần dữ liệu gửi từ frontend lên backend, thường được sử dụng khi thêm mới hoặc cập nhật thông tin. Ví dụ khi tạo một công việc mới vào hệ thống:

```
{
  "title": "Hoàn thiện báo cáo tháng 4",
  "description": "Tổng hợp số liệu và viết báo cáo tháng 4 cho phòng kế toán",
  "status": "todo",
  "priority": "high",
  "category": "Công việc",
  "due_date": "2026-05-31T23:59:00.000Z"
}
```

Sau khi xử lý request, server sẽ trả về response để thông báo kết quả cho phía giao diện. Trong response thường có mã trạng thái HTTP và dữ liệu phản hồi. Một số mã trạng thái thường gặp trong hệ thống TaskFlow gồm:

200 OK: truy xuất dữ liệu, cập nhật hoặc xóa thành công.

201 Created: thêm mới dữ liệu (tạo task, bình luận) thành công.

400 Bad Request: dữ liệu gửi lên không hợp lệ

401 Unauthorized: người dùng chưa đăng nhập hoặc token không hợp lệ.

403 Forbidden: không có quyền thao tác

404 Not Found: không tìm thấy dữ liệu yêu cầu

500 Internal Server Error: xảy ra lỗi phía cơ sở dữ liệu hoặc máy chủ.

Dữ liệu phản hồi thường được trả về dưới dạng JSON với cấu trúc chuẩn gồm trạng thái xử lý (success), thông báo (message) và dữ liệu liên quan (data). Ví dụ phản hồi khi tạo công việc thành công:

```
{
  "success": true,
  "message": "Tạo công việc thành công",
  "data": {
    "id": 42,
    "user_id": 1,
    "title": "Hoàn thiện báo cáo tháng 4",
    "description": "Tổng hợp số liệu và viết báo cáo tháng 4 cho phòng kế toán",
    "status": "todo",
    "priority": "high",
    "category": "Công việc",
    "due_date": "2026-05-31T23:59:00.000Z",
    "created_at": "2026-05-30T10:00:00.000Z"
  }
}
```

Việc sử dụng RESTful API giúp cho hệ thống TaskFlow hoạt động ổn định và đồng nhất hơn trong quá trình trao đổi dữ liệu. Đồng thời, cách tổ chức này cũng giúp frontend và backend có thể phát triển độc lập, dễ bảo trì và thuận lợi khi mở rộng thêm các chức năng mới (như thống kê, tích hợp AI) trong tương lai.

2.5. Kiến trúc RESTful API

2.5.1 Tổng quan về kiến trúc RESTful API

REST (Representational State Transfer) là một kiểu kiến trúc được sử dụng phổ biến trong quá trình xây dựng các hệ thống web và API hiện nay. Khái niệm này được giới thiệu bởi Roy Fielding vào năm 2000 [\[1\]](#) với mục đích tạo ra một chuẩn

giao tiếp đơn giản, dễ mở rộng và phù hợp với môi trường Internet. Những hệ thống được xây dựng dựa trên các nguyên tắc của REST thường được gọi là RESTful API.

Trong hệ thống quản lý công việc (TaskFlow), RESTful API được sử dụng để kết nối giữa giao diện người dùng và máy chủ xử lý dữ liệu. Thông qua API, frontend có thể gửi yêu cầu đến backend để thực hiện các chức năng như lấy danh sách công việc, thêm mới task, cập nhật trạng thái, quản lý tệp đính kèm hoặc phân quyền người dùng. Kết quả xử lý sau đó sẽ được backend trả ngược lại cho giao diện dưới dạng dữ liệu JSON.

Trong RESTful API, mỗi đối tượng trong hệ thống đều được xem như một tài nguyên (resource). Mỗi tài nguyên sẽ có một địa chỉ riêng gọi là URI để hệ thống có thể truy cập và xử lý. Ví dụ, tài nguyên công việc có thể được quản lý thông qua đường dẫn `/api/tasks`, còn việc xác thực người dùng được truy cập thông qua `/api/auth`.

RESTful API trong hệ thống được xây dựng dựa trên một số đặc điểm chính sau:

Stateless (Không lưu trạng thái): Mỗi request gửi từ client lên server đều phải chứa đầy đủ thông tin cần thiết để xử lý. Server sẽ không lưu trạng thái đăng nhập hay dữ liệu tạm của người dùng giữa các lần gọi API. Trong hệ thống này, việc xác thực được thực hiện thông qua JWT (JSON Web Token) gửi kèm trong header Authorization.

Client - Server: Frontend và backend được tách biệt rõ ràng. Giao diện người dùng (Frontend) được xây dựng độc lập, trong khi backend sử dụng node.js và express.js để xử lý logic và truy xuất cơ sở dữ liệu mysql. Việc phân tách này giúp hệ thống dễ nâng cấp, bảo trì và phát triển độc lập giữa các thành phần.

Cacheable: Một số dữ liệu ít thay đổi như danh sách danh mục (categories) công việc có thể được lưu tạm trên trình duyệt hoặc phía client nhằm giảm số lần gửi request lên server, từ đó giúp hệ thống phản hồi nhanh hơn.

Trong quá trình xây dựng RESTful API, hệ thống sử dụng các phương thức HTTP để thực hiện các thao tác với dữ liệu:

Bảng 2.1: Các phương thức HTTP để thực hiện thao tác dữ liệu

Phương thức	Chức năng	Ví dụ thực tế
GET	Lấy dữ liệu từ hệ thống	GET /api/tasks – Lấy danh sách công việc
POST	Thêm mới dữ liệu	POST /api/tasks – Thêm một công việc mới
PUT/PATCH	Cập nhật dữ liệu	PUT /api/tasks/42 – Cập nhật thông tin công việc có ID là 42
DELETE	Xóa dữ liệu	DELETE /api/tasks/42 – Xóa công việc có ID là 42

Dữ liệu trao đổi giữa frontend và backend được định dạng dưới dạng JSON (JavaScript Object Notation). Đây là định dạng dữ liệu nhẹ, dễ đọc và rất phù hợp khi làm việc với javascript ở cả phía client và server. Nhờ đó, quá trình xử lý và truyền dữ liệu trong hệ thống diễn ra nhanh chóng và thuận tiện hơn.

2.5.2 Ưu điểm của RESTful API

RESTful API là mô hình giao tiếp dữ liệu được sử dụng rộng rãi trong phát triển ứng dụng web hiện đại nhờ tính đơn giản, linh hoạt và khả năng mở rộng cao. Trong hệ thống quản lý công việc (TaskFlow), việc áp dụng RESTful API mang lại nhiều lợi ích trong quá trình xây dựng, vận hành và phát triển hệ thống:

Tính linh hoạt và khả năng tích hợp cao: RESTful API sử dụng giao thức HTTP cùng định dạng dữ liệu JSON nên có thể dễ dàng kết nối với nhiều nền tảng và công nghệ khác nhau. Điều này giúp hệ thống thuận tiện trong việc mở rộng tích hợp ứng dụng di động lai (Hybrid App) hoặc liên kết với các ứng dụng, phần mềm bên ngoài trong tương lai.

Phân tách rõ ràng giữa frontend và backend: Giao diện người dùng và phần xử lý dữ liệu hoạt động độc lập với nhau thông qua API. Nhờ đó, việc thay đổi giao diện frontend hoặc nâng cấp chức năng xử lý backend sẽ không ảnh hưởng trực tiếp đến thành phần còn lại, giúp hệ thống dễ bảo trì và phát triển hơn.

Khả năng mở rộng tốt: RESTful API hoạt động theo nguyên tắc Stateless nên server không cần lưu trạng thái của người dùng giữa các lần gửi request. Điều này

giúp giảm tải cho hệ thống và nâng cao khả năng xử lý đồng thời nhiều yêu cầu, đặc biệt khi số lượng công việc (tasks) và người dùng tăng lên.

Cấu trúc rõ ràng và dễ quản lý: Các endpoint được tổ chức theo dạng tài nguyên như /api/tasks, /api/auth, /api/comments giúp mã nguồn dễ hiểu, thuận tiện cho việc phát triển, kiểm tra (test) và bảo trì hệ thống.

Tối ưu hiệu suất truyền tải dữ liệu: Việc sử dụng JSON giúp dữ liệu có dung lượng nhỏ, tốc độ xử lý nhanh và phù hợp với môi trường web. Nhờ đó, hệ thống có thể phản hồi nhanh hơn và giảm băng thông trong quá trình trao đổi dữ liệu danh sách công việc hay thống kê.

2.5.3 Nhược điểm của RESTful API

Bên cạnh những ưu điểm nổi bật, RESTful API vẫn tồn tại một số hạn chế cần được xem xét trong quá trình xây dựng hệ thống:

Có thể xảy ra dư thừa dữ liệu: Trong một số trường hợp, API trả về nhiều thông tin hơn nhu cầu thực tế của phía client (ví dụ: chỉ cần lấy tiêu đề task nhưng API trả về toàn bộ mô tả, bình luận,...). Điều này có thể làm tăng dung lượng dữ liệu truyền tải và ảnh hưởng đến hiệu suất của hệ thống nếu không thiết kế tối ưu.

Chưa tối ưu cho các chức năng thời gian thực: RESTful API hoạt động theo mô hình request - response (client yêu cầu - server phản hồi) nên không phù hợp hoàn toàn với các tính năng yêu cầu cập nhật liên tục theo thời gian thực như: chat nhóm hay nhận thông báo tức thời. Để xử lý các chức năng này cần phải tích hợp thêm websocket.

Yêu cầu bảo mật chặt chẽ: Nếu API không được cấu hình bảo mật đúng cách, dữ liệu công việc có thể bị truy cập trái phép. Vì vậy cần triển khai các cơ chế như phân quyền chặt chẽ và sử dụng mã JWT Token để đảm bảo an toàn cho hệ thống.

Khó quản lý khi hệ thống phát triển lớn: Khi dự án TaskFlow ngày càng mở rộng và số lượng API tăng lên nhiều (cho subtasks, attachments, thống kê, phân quyền,...), việc quản lý phiên bản (versioning) của endpoint và viết tài liệu API có thể trở nên phức tạp nếu không có quy chuẩn thiết kế thống nhất ngay từ đầu.

2.6. Tổng quan về Node.js

2.6.1 Tìm hiểu Node.js

Node.js là môi trường thực thi javascript mã nguồn mở [\[10\]](#) cho phép chạy javascript ở phía máy chủ (Server-side). Node.js được xây dựng dựa trên v8 engine của google chrome, giúp biên dịch trực tiếp mã javascript thành mã máy nên có tốc độ xử lý nhanh và hiệu quả.

Trước đây, javascript chủ yếu được sử dụng để xử lý các chức năng phía giao diện người dùng trên trình duyệt. Tuy nhiên, với sự xuất hiện của node.js, javascript có thể được sử dụng để xây dựng cả frontend lẫn backend, giúp quá trình phát triển ứng dụng web trở nên thống nhất và thuận tiện hơn.

Một trong những đặc điểm nổi bật của node.js là hoạt động theo mô hình bất đồng bộ (Asynchronous) và non-blocking i/o. Điều này cho phép hệ thống xử lý nhiều yêu cầu cùng lúc mà không phải chờ hoàn thành từng tác vụ trước khi tiếp tục xử lý các yêu cầu khác. Nhờ đó, node.js đặc biệt phù hợp với các hệ thống web cần khả năng phản hồi nhanh và xử lý nhiều kết nối đồng thời.

Trong đề tài xây dựng hệ thống quản lý công việc (TaskFlow), node.js được sử dụng làm nền tảng phát triển backend của hệ thống. Node.js đảm nhận vai trò tiếp nhận request từ frontend, xử lý logic nghiệp vụ, kết nối cơ sở dữ liệu mysql và trả dữ liệu về cho người dùng thông qua RESTful API.

2.6.2 Các đặc điểm kỹ thuật của Node.js

Sử dụng V8 Engine: Node.js được phát triển dựa trên v8 engine của google nên có khả năng xử lý javascript với tốc độ cao và hiệu suất ổn định.

Hoạt động bất đồng bộ (Asynchronous): Các thao tác như truy xuất cơ sở dữ liệu, đọc ghi file hoặc gọi API đều được xử lý bất đồng bộ. Điều này giúp hệ thống không bị chặn trong quá trình xử lý dữ liệu.

Non-blocking I/O: Node.js có thể tiếp tục xử lý các request khác trong khi chờ dữ liệu trả về từ database hoặc các dịch vụ bên ngoài (như khi kết nối với API của Google AI), giúp tối ưu tài nguyên máy chủ.

Single thread kết hợp event loop: Mặc dù node.js hoạt động chủ yếu trên một luồng chính nhưng nhờ cơ chế event loop nên vẫn có khả năng xử lý đồng thời nhiều kết nối hiệu quả.

Hệ sinh thái thư viện phong phú: Node.js sử dụng NPM (Node Package Manager) để quản lý thư viện. Đây là kho thư viện mã nguồn mở rất lớn, hỗ trợ lập trình viên xây dựng ứng dụng nhanh hơn.

Trong hệ thống TaskFlow, các thư viện chính đã được cài đặt thông qua NPM để xây dựng backend gồm:

Bảng 2.2: Thư viện node.js được sử dụng trong hệ thống quản lý công việc

STT	Thư viện	Chức năng
1	express	Framework chính để xây dựng máy chủ HTTP và các RESTful API định tuyến. [11]
2	mysql2	Trình điều khiển (driver) kết nối, thực hiện truy vấn cơ sở dữ liệu mysql bất đồng bộ. [12]
3	jsonwebtoken	Tạo và giải mã mã xác thực JWT giúp kiểm tra phiên làm việc người dùng. [16]
4	bcryptjs	Thực hiện hàm băm mã hóa mật khẩu người dùng tăng cường bảo mật. [17]
5	multer	Middleware hỗ trợ xử lý dữ liệu form dạng multipart/form-data để tải file đính kèm. [20]
6	socket.io	Thư viện quản lý kết nối websocket thời gian thực giữa máy chủ và trình duyệt. [15]
7	nodemailer	Hỗ trợ kết nối máy chủ gửi thư điện tử (SMTP) để thực hiện gửi thông báo nhắc nhở. [19]
8	node-cron	Lập lịch chạy tự động các tác vụ kiểm tra hạn chót định kỳ trên máy chủ. [23]

STT	Thư viện	Chức năng
9	passport	Middleware xác thực hỗ trợ thiết lập đăng nhập nhanh qua google oauth 2.0. [24]
10	@google/generativeai	SDK kết nối với mô hình ngôn ngữ lớn google gemini AI để tích hợp Trợ lý ảo. [18]
11	cors	Cấu hình cho phép chia sẻ tài nguyên nguồn chéo giữa frontend và backend. [21]
12	dotenv	Tải và cấu hình các biến môi trường bảo mật từ file cấu hình hệ thống .env. [22]

2.6.3 Ưu điểm của Node.js

Hiệu suất xử lý cao: Nhờ sử dụng v8 engine cùng cơ chế non-blocking i/o, node.js có khả năng xử lý request nhanh và phù hợp với các ứng dụng web có nhiều người dùng truy cập cùng lúc.

Khả năng mở rộng tốt: Node.js phù hợp với các hệ thống cần khả năng mở rộng trong tương lai, đặc biệt là các ứng dụng quản lý dữ liệu trực tuyến.

Sử dụng chung ngôn ngữ javascript: Việc sử dụng javascript cho cả frontend và backend giúp giảm độ phức tạp trong quá trình phát triển và thuận tiện cho việc bảo trì hệ thống.

Cộng đồng hỗ trợ lớn: Node.js có cộng đồng phát triển mạnh cùng số lượng thư viện phong phú, giúp lập trình viên dễ dàng tìm kiếm tài liệu và giải pháp khi gặp vấn đề.

2.6.4 Nhược điểm của Node.js

Không phù hợp với các tác vụ xử lý nặng về CPU: Node.js hoạt động theo mô hình single thread nên không tối ưu cho các bài toán cần xử lý tính toán đồ họa phức tạp hoặc xử lý nén dữ liệu lớn liên tục.

Khó quản lý bất đồng bộ trong hệ thống lớn: Nếu không tổ chức mã nguồn hợp lý, việc xử lý nhiều tác vụ bất đồng bộ lồng nhau có thể dẫn đến hiện tượng khó kiểm soát mã nguồn.

Thư viện thay đổi nhanh: Các package trong hệ sinh thái NPM được cập nhật thường xuyên nên đôi khi có thể xảy ra lỗi tương thích giữa các phiên bản thư viện nếu không quản lý tốt file package.json.

2.6.5 Vai trò của Node.js trong hệ thống

Trong hệ thống quản lý công việc (TaskFlow), node.js kết hợp với framework express.js để xây dựng tầng backend và RESTful API. Node.js đảm nhận nhiều chức năng cốt lõi quan trọng như:

Tiếp nhận và xử lý nhanh chóng các request từ giao diện người dùng (Frontend).

Xây dựng và quản lý toàn bộ các endpoint API của hệ thống.

Thực hiện cơ chế xác thực bảo mật và phân quyền người dùng (Admin/User) thông qua chuỗi JWT.

Kết nối, truy vấn và thực hiện các thao tác thêm/sửa/xóa với cơ sở dữ liệu MySQL một cách an toàn.

Xử lý logic nghiệp vụ phức tạp của các chức năng quản lý công việc, dự án nhóm, thời khóa biểu và các mốc tiến độ.

Tiếp nhận và lưu trữ các tệp tin đính kèm (attachments) cũng như hệ thống bình luận (comments) trao đổi của người dùng.

Giao tiếp với API của google gemini AI để cung cấp các tính năng thông minh.

2.7. Tổng quan về Express.js

2.7.1 Tìm hiểu Express.js

Express.js là một framework mã nguồn mở [\[11\]](#) được xây dựng trên nền tảng node.js, hỗ trợ phát triển ứng dụng web và RESTful API một cách nhanh chóng và hiệu quả. Framework này cung cấp nhiều công cụ và chức năng giúp đơn giản hóa quá trình xử lý request, định tuyến URL, quản lý middleware và xây dựng hệ thống backend.

Hiện nay, express.js được xem là framework phổ biến nhất trong hệ sinh thái Node.js nhờ cấu trúc tối giản, dễ sử dụng và khả năng mở rộng tốt. Express.js giúp lập trình viên giảm đáng kể lượng mã nguồn cần viết so với việc sử dụng node.js thuần, đồng thời hỗ trợ tổ chức project theo hướng rõ ràng và dễ bảo trì hơn.

Trong hệ thống quản lý công việc (TaskFlow), express.js được sử dụng để xây dựng tầng backend và hệ thống RESTful API. Framework này đóng vai trò tiếp nhận các yêu cầu từ frontend, điều hướng request đến các chức năng xử lý tương ứng và trả dữ liệu về cho người dùng dưới dạng JSON.

2.7.2 Các đặc điểm chính của Express.js

Express.js cung cấp nhiều tính năng quan trọng hỗ trợ quá trình phát triển và vận hành hệ thống TaskFlow:

Middleware linh hoạt: Express.js cho phép sử dụng middleware để xử lý request trước khi trả response cho client. Middleware có thể dùng để xác thực người dùng, kiểm tra quyền truy cập, xử lý dữ liệu JSON, phân tích form-data, hoặc tăng cường bảo mật.

Bảng 2.3: Một số middleware được sử dụng trong hệ thống

STT	Middleware	Chức năng
1	cors	Cho phép chia sẻ tài nguyên giữa các miền (Cross-Origin Resource Sharing - CORS), giúp frontend và backend có thể giao tiếp an toàn khi chạy trên các domain hoặc cổng khác nhau.
2	express.json()	Phân tích cú pháp (parse) dữ liệu JSON được gửi từ Client trong phần body của HTTP Request, giúp backend dễ dàng truy xuất dữ liệu.
3	express.urlencoded()	Phân tích dữ liệu được mã hóa theo định dạng URL-encoded từ các biểu mẫu HTML (application/x-www-form-urlencoded).

STT	Middleware	Chức năng
4	auth (Custom Middleware)	Kiểm tra sự tồn tại và tính hợp lệ của JWT (JSON Web Token) trong header Authorization, xác thực người dùng và kiểm tra quyền truy cập trước khi xử lý yêu cầu.
5	multer	Xử lý việc tải tệp (File Upload) từ Client lên máy chủ thông qua dữ liệu multipart/form-data, đồng thời hỗ trợ lưu trữ và quản lý các tệp đính kèm.

Hệ thống định tuyến (Routing): Express.js hỗ trợ xây dựng và quản lý các route API một cách rõ ràng. Mỗi route sẽ tương ứng với một module chức năng cụ thể trong hệ thống như quản lý công việc, người dùng, trợ lý AI hay thùng rác. Ví dụ trích xuất từ mã nguồn:

```
app.use('/api/auth', authRoutes);
app.use('/api/tasks', taskRoutes);
app.use('/api/ai', aiRoutes);
app.use('/api/tasks/:taskId/comments',
commentRoutes);
```

Để dàng tích hợp thư viện: Express.js có khả năng tích hợp tốt với nhiều thư viện trong hệ sinh thái Node.js như mã hóa mật khẩu (bcryptjs), xử lý file (multer), hoặc giao tiếp với AI (@google/generative-ai). Điều này giúp quá trình phát triển TaskFlow trở nên nhanh chóng và mạnh mẽ.

2.7.3 Cấu trúc project với Express.js

Trong hệ thống TaskFlow, dự án backend được cấu trúc rõ ràng theo mô hình điều hướng-xử lý (Router-Controller-Model) chạy song song với các tiến trình Socket.IO. Express.js định tuyến toàn bộ yêu cầu, áp dụng các middleware kiểm tra bảo mật (JWT Auth), sau đó gọi tới các controller để thực thi logic nghiệp vụ và trả kết quả về dưới dạng JSON chuẩn. Hệ thống cũng sử dụng express.js để phục vụ các tài nguyên tĩnh như file upload của người dùng thông qua API định tuyến tĩnh (express.static).

2.8. Hệ quản trị cơ sở dữ liệu MySQL

2.8.1 Tổng quan MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ [12] (Relational Database Management System - RDBMS) được sử dụng rất phổ biến trong các hệ thống website và ứng dụng quản lý hiện nay. MySQL sử dụng ngôn ngữ sql để thực hiện các thao tác lưu trữ, truy vấn và quản lý dữ liệu.

Nhờ có tốc độ xử lý tốt, độ ổn định cao và dễ sử dụng, mysql thường được lựa chọn trong các hệ thống quản lý dữ liệu có quy mô từ nhỏ đến lớn. Ngoài ra, mysql còn là phần mềm mã nguồn mở nên phù hợp với các dự án học tập, nghiên cứu và phát triển hệ thống thực tế.

Trong hệ thống quản lý công việc (TaskFlow), mysql được sử dụng làm hệ quản trị cơ sở dữ liệu chính để lưu trữ toàn bộ thông tin của ứng dụng. Các dữ liệu được tổ chức theo dạng bảng và có liên kết với nhau thông qua khóa chính và khóa ngoại nhằm đảm bảo tính nhất quán của toàn bộ hệ thống.

2.8.2 Đặc điểm kỹ thuật của hệ quản trị cơ sở dữ liệu MySQL

Mô hình cơ sở dữ liệu quan hệ: Dữ liệu được tổ chức dưới dạng các bảng có mối liên kết với nhau thông qua khóa chính (Primary Key) và khóa ngoại (Foreign Key), giúp quản lý dữ liệu một cách chặt chẽ và nhất quán.

Hỗ trợ ngôn ngữ SQL: MySQL sử dụng ngôn ngữ truy vấn có cấu trúc (SQL) để thực hiện các thao tác thêm, sửa, xóa và truy vấn dữ liệu. Cú pháp đơn giản, dễ sử dụng cùng khả năng xử lý linh hoạt giúp việc phát triển và bảo trì hệ thống trở nên thuận tiện hơn.

Đảm bảo tính toàn vẹn dữ liệu: MySQL hỗ trợ cơ chế transaction theo nguyên tắc ACID, giúp đảm bảo các thao tác trên cơ sở dữ liệu được thực hiện đầy đủ và chính xác. Ví dụ, khi xóa một công việc, các bình luận hoặc tệp đính kèm liên quan có thể được xóa đồng bộ (Cascade Delete) để tránh rác dữ liệu.

Hỗ trợ Index: Hệ thống có thể tạo chỉ mục trên các cột thường xuyên được truy vấn (như user_id, due_date) nhằm cải thiện hiệu suất xử lý dữ liệu, nâng cao tốc độ phản hồi của ứng dụng.

2.8.3 Thiết kế cơ sở dữ liệu

Hệ thống quản lý công việc (TaskFlow) sử dụng các bảng cơ sở dữ liệu quan hệ để lưu trữ thông tin:

Bảng 2.4: Các bảng chính để lưu trữ và quản lý thông tin trong Taskflow

STT	Bảng dữ liệu	Vai trò / Chức năng
1	users	Lưu thông tin tài khoản người dùng (họ tên, email, mật khẩu đã mã hóa) và thông tin phân quyền (Admin/User).
2	gamification	Lưu trữ thông tin trò chơi hóa như điểm kinh nghiệm (XP), cấp độ (Level), thứ hạng và danh hiệu của người dùng.
3	tasks	Bảng trung tâm lưu trữ thông tin công việc như tiêu đề, mô tả, trạng thái, mức độ ưu tiên, thời hạn hoàn thành và các thuộc tính liên quan.
4	subtasks	Lưu các nhiệm vụ con (Checklist) thuộc một công việc chính, hỗ trợ theo dõi tiến độ thực hiện chi tiết.
5	task_dependencies	Lưu trữ mối quan hệ phụ thuộc giữa các công việc, xác định thứ tự thực hiện các nhiệm vụ.
6	task_comments	Lưu nội dung bình luận và trao đổi của người dùng trên từng công việc.
7	task_attachments	Lưu thông tin và đường dẫn của các tệp đính kèm liên quan đến công việc.
8	projects	Lưu thông tin dự án nhóm như tên dự án, mô tả và các thông tin liên quan.

STT	Bảng dữ liệu	Vai trò / Chức năng
9	project_members	Quản lý danh sách thành viên tham gia dự án và quyền của từng thành viên trong dự án.
10	project_chats	Lưu trữ các tin nhắn trao đổi theo thời gian thực giữa các thành viên trong dự án.
11	grades	Lưu thông tin điểm số các môn học như điểm chuyên cần, giữa kỳ, cuối kỳ và điểm tổng kết.
12	class_schedules	Quản lý thời khóa biểu học tập cố định của người dùng theo từng tuần.
13	habits	Lưu thông tin các thói quen cần theo dõi như tên thói quen, tần suất thực hiện và chuỗi ngày hoàn thành (streak).
14	notes	Lưu trữ các ghi chú cá nhân và nội dung học tập để hỗ trợ ôn tập.
15	flashcard_decks	Quản lý các bộ thẻ ghi nhớ (Flashcard Deck) phục vụ việc học tập và ôn luyện kiến thức.

2.8.4 Ưu điểm của MySQL

Tốc độ xử lý dữ liệu nhanh và ổn định: MySQL được tối ưu cho các thao tác truy vấn giúp hệ thống phản hồi nhanh ngay cả khi khối lượng dữ liệu tăng lên.

Đảm bảo tính toàn vẹn dữ liệu: Cơ chế khóa ngoại giúp ngăn chặn việc xóa nhầm các dữ liệu liên quan ở các bảng con.

Không tốn chi phí bản quyền: MySQL là phần mềm mã nguồn mở hoàn toàn miễn phí.

Dễ dàng tích hợp với node.js: Kết nối thuận lợi thông qua thư viện mysql2, hỗ trợ các truy vấn bất đồng bộ dựa trên cơ chế promise.

2.8.5 Nhược điểm của MySQL

Chưa tối ưu cho dữ liệu phi cấu trúc: Không linh hoạt bằng các cơ sở dữ liệu NoSQL đối với các dạng dữ liệu thay đổi thuộc tính liên tục.

Yêu cầu thiết kế tối ưu khi mở rộng: Khi hệ thống phát triển lớn với hàng chục ngàn nhiệm vụ và dự án, việc liên kết (Join) nhiều bảng phức tạp đòi hỏi cấu hình chỉ mục (Index) kỹ lưỡng để tránh suy giảm hiệu năng.

2.9. Tổng quan về HTML, CSS và JavaScript

2.9.1 Tìm hiểu về HTML (HyperText Markup Language)

HTML là ngôn ngữ đánh dấu tiêu chuẩn [\[3\]](#) được sử dụng để xây dựng cấu trúc cho các trang web. HTML định hình bố cục trang web, phân tách các khu vực văn bản, hình ảnh, bảng số liệu, biểu mẫu nhập liệu và các nút bấm tương tác. Trong hệ thống TaskFlow, HTML được tích hợp trực tiếp dưới dạng các cú pháp template của Vue, làm nhiệm vụ hiển thị khung biểu diễn cho các thành phần giao diện.

2.9.2 Tìm hiểu về CSS (Cascading Style Sheets)

CSS đóng vai trò quyết định tính thẩm mỹ [\[4\]](#) và giao diện trực quan của TaskFlow (UI/UX). Hệ thống ứng dụng CSS thuần để tùy biến phong cách hiển thị bao gồm:

Phân màu theo trạng thái: Badge báo độ ưu tiên màu đỏ (Cao), màu vàng (Trung bình) và màu xanh (Thấp); cảnh báo đỏ cho công việc quá hạn hoặc dời hạn.

Thiết kế bố cục hiện đại: Sử dụng CSS Grid và flexbox để xây dựng cấu trúc dạng thẻ (card-based), thanh bên (Sidebar) thu gọn và hỗ trợ responsive tự động thích ứng với nhiều kích thước màn hình từ desktop đến mobile.

Hiệu ứng chuyển động: Cung cấp hiệu ứng hover, transition mượt mà giúp trải nghiệm kéo thả kanban và pomodoro trực quan, sống động hơn.

2.9.3 Tìm hiểu về JavaScript

JavaScript là ngôn ngữ lập trình chạy trên trình duyệt [\[5\]](#) đảm nhận chức năng xử lý các tương tác logic động bất đồng bộ ở phía client. JavaScript thực hiện kết nối

với backend thông qua thư viện gửi request axios, cập nhật giao diện thời gian thực và quản lý các luồng xử lý chính:

Xử lý bất đồng bộ: Gọi các API lấy dữ liệu mà không cần tải lại trang web (Single Page Application).

Trực quan biểu đồ: Tương tác trực tiếp với thư viện chart.js để vẽ biểu đồ tiến độ dạng tròn và cột trên dashboard và report.

Tương tác kéo thả: Quản lý hành động drag & drop trên kanban board và timebox planner.

Hội thoại với gemini AI: Xử lý gửi câu hỏi và nhận về nội dung phản hồi từ mô hình AI thời gian thực.

Xuất file PDF: Sử dụng thư viện html2pdf.js để tự động chụp giao diện HTML báo cáo thành ảnh canvas rồi chuyển đổi thành tài liệu PDF A4.

2.9.4 Vai trò của HTML, CSS và JavaScript trong hệ thống

Sự kết hợp đồng bộ giữa HTML (bộ khung cấu trúc), CSS (giao diện đồ họa trực quan) và javascript (hành động logic và tương tác API) đã tạo nên một ứng dụng khách (Frontend Client) hoạt động độc lập, mượt mà, phản hồi nhanh chóng trước mọi thao tác quản trị thời gian và công việc của người dùng.

2.10. Giao diện Vue.js và Single Page Application (SPA)

Vue.js (phiên bản Vue 3) là một framework [\[6\]](#) javascript lũy tiến được lựa chọn để xây dựng giao diện người dùng cho TaskFlow dưới dạng ứng dụng đơn trang (Single Page Application - SPA).

Kiến trúc SPA: Với SPA, trình duyệt chỉ tải trang index.html một lần duy nhất. Khi người dùng chuyển trang hoặc lọc công việc, hệ thống định tuyến client (Vue Router) sẽ hoán đổi linh hoạt các component và render trực tiếp trên client mà không cần load lại trình duyệt từ server, tạo tốc độ truy cập nhanh tức thì.

Composition API: Vue 3 sử dụng composition API giúp tổ chức mã nguồn theo chức năng, dễ dàng tái sử dụng logic (Composables) và tối ưu hóa hệ thống reactive.

Reactivity System: Vue 3 sử dụng cơ chế reactive dựa trên ES6 Proxy để theo dõi các biến số. Khi dữ liệu của một công việc thay đổi, giao diện sẽ tự động cập nhật ngay lập tức mà không cần can thiệp DOM thủ công.

2.11. Quản lý trạng thái toàn cục với Pinia

Đối với một hệ thống quản lý công việc phức tạp như TaskFlow, việc chia sẻ trạng thái người dùng đăng nhập, danh sách nhiệm vụ và thông báo giữa các trang là vô cùng cần thiết. Hệ thống đã ứng dụng thư viện quản lý trạng thái tập trung pinia để giải quyết vấn đề này.

Nguyên lý hoạt động: Pinia đóng vai trò như một kho chứa dữ liệu chung [\[7\]](#) (Centralized Store) chạy độc lập với các vue component. Mọi component đều có thể truy xuất hoặc kích hoạt các hàm cập nhật dữ liệu (Actions) trong store.

Các Store chính:

authStore: Lưu trữ thông tin phiên đăng nhập của người dùng, token xác thực JWT và vai trò phân quyền. taskStore: Quản lý danh sách nhiệm vụ, bộ lọc tìm kiếm và các hoạt động thay đổi trạng thái công việc.

friendStore: Quản lý thông tin kết bạn và danh sách trực tuyến.

2.12. Socket.IO và Giao tiếp thời gian thực

RESTful API thông thường hoạt động trên nguyên lý một chiều (Client gọi, Server trả lời). Để hiện thực hóa các tính năng cộng tác nhóm trực tuyến như chat dự án và thông báo tức thời, hệ thống sử dụng socket.io dựa trên giao thức WebSocket. [\[15\]](#)

Giao thức WebSocket: Thiết lập một kênh kết nối song công (bi-directional), liên tục và duy nhất giữa trình duyệt của client và máy chủ node.js.

Cơ chế Event-Driven: Khi một thành viên dự án gửi tin nhắn chat, frontend phát ra sự kiện (emit). Máy chủ backend nhận tín hiệu và phát tán tức thời (broadcast) tin nhắn đó tới tất cả các thành viên cùng phòng chat đang trực tuyến mà không có độ trễ.

2.13. Trí tuệ nhân tạo (AI) tạo sinh và mô hình Google Gemini

Một trong những điểm đổi mới của hệ thống TaskFlow là việc ứng dụng trí tuệ nhân tạo để làm cố vấn năng suất cho người dùng thông qua việc tích hợp google gemini API (sử dụng SDK `@google/generative-ai` ở Backend). [\[18\]](#)

Mô hình Generative AI: Sử dụng các prompt thiết kế cấu trúc để hướng dẫn mô hình ngôn ngữ lớn xử lý thông tin.

Ứng dụng thực tế trong hệ thống:

AI phân rã công việc (decompose): Tự động phân tích một công việc phức tạp thành danh sách các bước thực hiện nhỏ hơn (subtasks) dưới định dạng JSON có cấu trúc.

AI Coach: Cho phép trò chuyện thông minh về giải pháp hoàn thành công việc dựa trên ngữ cảnh công việc hiện có của người dùng.

AI tạo Quiz: Quét tài liệu ghi chú học tập của sinh viên và tự động sinh ra các câu hỏi trắc nghiệm ôn tập nhanh.

2.14. Cơ chế bảo mật JWT và mã hóa bcryptjs

JSON Web Token (JWT): Là một phương thức xác thực [\[16\]](#) không lưu trạng thái (Stateless Authentication). Khi người dùng đăng nhập thành công, server tạo ra một chuỗi token mã hóa chứa thông tin định danh và ký số bảo mật. Client lưu token này ở bộ nhớ cục bộ và gửi kèm trong header authorization của mỗi request API tiếp theo. Điều này giúp bảo vệ các API khỏi truy cập trái phép mà không làm nặng bộ nhớ máy chủ.

Mã hóa bcryptjs: Mật khẩu người dùng không bao giờ được lưu trữ dưới dạng văn bản thô trong mysql. Khi đăng ký hoặc đổi mật khẩu, hệ thống áp dụng thuật

toán băm mật khẩu bcryptjs [\[17\]](#) kết hợp với chuỗi muối ngẫu nhiên (Salt rounds) để lưu trữ chuỗi mã hóa một chiều, đảm bảo an toàn tuyệt đối chống lại các cuộc tấn công khai thác cơ sở dữ liệu.

CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Mô tả bài toán

3.1.1 Bối cảnh nghiên cứu

Trong kỷ nguyên số hóa, nhu cầu quản lý công việc và tối ưu hóa hiệu suất làm việc cá nhân cũng như làm việc nhóm đang trở nên cấp thiết hơn bao giờ hết. Khối lượng công việc ngày càng gia tăng đòi hỏi sự tiện lợi, nhanh chóng và tính đồng bộ cao trong việc theo dõi tiến độ. Tuy nhiên, việc ứng dụng trí tuệ nhân tạo (AI) để tối ưu hóa quy trình phân tích và gợi ý hướng giải quyết công việc tại Việt Nam vẫn còn ở giai đoạn sơ khởi, đa số người dùng vẫn chỉ đang sử dụng các công cụ ghi chú truyền thống thiếu tính liên kết.

3.1.2 Thực trạng quản lý công việc

Qua khảo sát thực tế, quy trình tổ lý và theo dõi công việc theo phương thức truyền thống hoặc qua các nền tảng tin nhắn hiện có vẫn tồn tại những bất cập cốt lõi sau:

Sự rời rạc của thông tin: Dữ liệu về nhiệm vụ, tệp đính kèm và lịch sử trao đổi thường phân tán ở nhiều nguồn khác nhau (Zalo, email, sổ tay), thiếu sự tổng hợp tập trung, gây khó khăn và mất thời gian cho việc tra cứu lại.

Thiếu tính hỗ trợ thông minh: Các công cụ quản lý hiện nay thường chỉ đóng vai trò ghi chép tĩnh, chưa thể tương tác hoặc phân tích nội dung để đưa ra gợi ý, chia nhỏ công việc phức tạp thành các bước đơn giản hơn.

Bài toán quản lý sự phụ thuộc: Việc sắp xếp thứ tự công việc và theo dõi sự ràng buộc lẫn nhau (công việc B chỉ được làm khi công việc A hoàn thành) thường bị bỏ ngỏ hoặc theo dõi thủ công, dễ dẫn đến tình trạng chồng chéo và trễ hạn (Overdue).

Hạn chế trong quản lý và phân tích: Người dùng cá nhân hoặc nhà quản lý thiếu đi một bảng điều khiển trực quan (Dashboard) để định lượng hiệu suất, theo dõi tỷ lệ hoàn thành và phân tích xu hướng làm việc dựa trên dữ liệu thực tế.

3.1.3 Mục tiêu

Nhằm giải quyết các vấn đề trên, đề tài tập trung xây dựng nền tảng Hệ thống Quản lý công việc (Taskflow). Hệ thống được thiết kế để giải quyết bài toán thông qua các trục tính năng chính:

3.1.3.1 Đối với người dùng:

Người dùng có thể tạo, chỉnh sửa và xóa công việc cá nhân một cách toàn diện bằng cách nhập tiêu đề, mô tả, danh mục, mức ưu tiên và thời hạn, đồng thời có thể đặt màu nền cho thẻ công việc để dễ nhận diện. Hệ thống hỗ trợ bốn trạng thái tiến độ: Cần làm, đang làm, đã xong và quá hạn; khi thời hạn vượt qua, một tác vụ cron trên backend sẽ tự động chuyển trạng thái sang Quá hạn. Các công việc được phân loại theo danh mục (ví dụ: Học tập, công việc, cá nhân, sức khỏe...) và mức ưu tiên (Thấp, trung bình, cao), cho phép người dùng lọc nhanh trên dashboard. Người dùng có thể đính kèm các tệp tin như PDF, hình ảnh, video hoặc tài liệu office bằng cách kéo - thả vào modal đính kèm; hệ thống sẽ lưu tên tệp, đường dẫn, kích thước và định dạng MIME trong bảng `task_attachments`. Mỗi công việc có thể có nhiều sub-task dưới dạng checklist; khi người dùng đánh dấu một sub-task là hoàn thành, phần trăm tiến độ của công việc sẽ được tính toán tự động và cập nhật trên giao diện. Mọi thao tác tạo, sửa, xóa hoặc thay đổi trạng thái đều được ghi lại trong bảng `task_activities` và hiển thị trong tab “Hoạt động” dưới dạng một dòng thời gian màu sắc, giúp người dùng theo dõi lịch sử thay đổi chi tiết.

Khi cần liên kết các công việc với nhau, người dùng có thể xác định một công việc phụ thuộc vào một công việc khác; hệ thống sẽ lưu các cặp phụ thuộc trong bảng `task_dependencies` và thực hiện kiểm tra vòng lặp bằng thuật toán duyệt đồ thị có hướng. Nếu phát hiện vòng lặp, một hộp thoại cảnh báo sẽ hiện ra và ngăn việc lưu liên kết không hợp lệ. Đồ thị phụ thuộc được vẽ bằng thư viện `cytoscape.js`, với các mũi tên xanh cho liên kết hợp lệ và màu đỏ cho liên kết bị chặn. Khi công việc phụ thuộc được hoàn thành, hệ thống sẽ tự động gửi thông báo push cho các công việc đang chờ, giúp người dùng biết ngay khi có thể tiến hành.

TaskFlow tích hợp trợ lý AI google gemini để hỗ trợ người dùng tạo và phân rã công việc một cách thông minh. Khi người dùng nhập câu lệnh tiếng Việt như “Lên kế hoạch học vue 3 trong 2 tuần”, backend sẽ gọi API Gemini, nhận về một JSON mô tả các task và sub-task, và tự động tạo chúng trong hệ thống. Trợ lý AI cũng có thể nhận mô tả công việc lớn và trả về danh sách các bước chi tiết, đồng thời đề xuất độ ưu tiên và thời gian dự kiến cho từng bước. Dựa trên lịch biểu và khối lượng công việc hiện tại, AI sẽ gợi ý lịch làm việc tối ưu, cân bằng tải và giảm áp lực. Nó còn phân tích hiệu suất làm việc, tính tỷ lệ hoàn thành và thời gian trễ, đưa ra đề xuất cải thiện như “Bạn nên giảm số nhiệm vụ trong ngày thứ hai”. Khi người dùng ghi chú học tập, AI có thể sinh câu hỏi trắc nghiệm hoặc flashcards để hỗ trợ ôn tập.

Đối với làm việc nhóm, người dùng có thể tạo dự án, mời thành viên bằng email hoặc mã QR và phân quyền owner hoặc member. Mỗi dự án có bảng kanban riêng, các task được gắn với project_id để phân biệt. Người dùng có thể định nghĩa các mốc quan trọng (Milestones) với deadline, hệ thống sẽ gửi thông báo một ngày và một giờ trước khi mốc đến. Chat thời gian thực được cung cấp qua socket.io; mỗi kênh chat dự án cho phép gửi tin nhắn, tệp đính kèm và emoji. Dashboard dự án hiển thị biểu đồ burndown, cumulative flow và số task hoàn thành theo từng thành viên, giúp theo dõi tiến độ nhóm một cách rõ ràng.

Về quản lý thời gian và học tập, giao diện lịch tuần cho phép kéo-thả task vào khung giờ và đồng bộ với google calendar nếu cần. Người dùng có thể đặt khung thời gian (Timebox Planner) từ 15 đến 60 phút cho mỗi task, và timer sẽ đếm ngược, lưu số lần pomodoro trong bảng hoạt động. Pomodoro timer mặc định 25 phút làm việc và 5 phút nghỉ, kèm âm thanh nền giúp tăng tập trung. Habit tracker cho phép định nghĩa thói quen hàng ngày, mỗi lần hoàn thành được ghi lại trong habit_logs và hiển thị dưới dạng heatmap với chuỗi ngày liên tục (streak) cùng phần thưởng XP. Hệ thống còn hỗ trợ quản lý điểm số môn học, tính GPA tự động và xuất báo cáo PDF cho kết quả học tập. Flashcards và OKR cho phép người dùng tạo câu hỏi và mục tiêu, hệ thống áp dụng spaced-repetition để tối ưu quá trình ôn tập.

Phân tích và báo cáo được thể hiện qua dashboard trực quan, với các biểu đồ tròn, cột và đường thể hiện số task theo trạng thái, ưu tiên, danh mục, tỷ lệ hoàn thành và số giờ làm việc. Dữ liệu được lấy qua API /api/stats và vẽ bằng Chart.js.

Khi người dùng nhấn “Xuất PDF”, component Reports.vue sử dụng html2pdf.js để chuyển nội dung dashboard thành file PDF ngay trên trình duyệt, không cần qua server. Đối với quản trị viên, backend cung cấp API /api/admin/export-csv trả về chuỗi CSV (text/csv) cho các bảng người dùng, task và hoạt động, cho phép tải xuống và phân tích bằng excel. Hệ thống còn có cơ chế cảnh báo thông minh: nếu tỷ lệ overdue vượt quá 20 %, một thông báo toast sẽ xuất hiện và ghi lại trong nhật ký hoạt động.

TaskFlow hỗ trợ tương tác xã hội và gamification. Người dùng có thể gửi lời mời kết bạn, xem hồ sơ của nhau với avatar, XP, level và danh hiệu. Mỗi hành động như tạo task, hoàn thành sub-task, comment hoặc tải lên tệp sẽ tăng XP; khi XP đạt ngưỡng, level sẽ tăng và người dùng nhận badge. Avatar RPG được thiết kế phong cách trò nhập vai, cấp độ và trang bị (skin) mở khóa khi đạt các milestones như số task đã hoàn thành hoặc chuỗi ngày liên tục. Bảng xếp hạng (Leaderboard) hiển thị top 10 người dùng dựa trên XP và số task hoàn thành trong tuần. Quản trị viên có thể tạo thử thách cộng đồng (ví dụ “Hoàn thành 30 task trong 7 ngày”) và người dùng đăng ký tham gia để nhận bonus XP khi hoàn thành. Tất cả các tính năng này được đồng bộ giữa frontend Vue 3 + Pinia và backend Node / Express + MySQL, với dữ liệu cập nhật real-time qua socket.io và các tác vụ nền chạy trên server, đảm bảo trải nghiệm mượt mà, thông minh và có yếu tố động lực cao cho người dùng.

3.1.3.2 Đối với quản trị viên (Admin):

Quản trị viên có quyền truy cập toàn bộ giao diện quản lý hệ thống và thực hiện các chức năng sau: quản lý người dùng và phân quyền, cho phép tìm kiếm, xem chi tiết, cập nhật thông tin (tên, email, vai trò) và xóa tài khoản; khi thay đổi trường role (user - admin) hệ thống tự động áp dụng quyền truy cập mới cho người dùng tương ứng. quản lý danh mục và môn học, cho phép thêm, sửa, xóa các danh mục công việc (ví dụ: Học tập, công việc, cá nhân, sức khỏe...) và các môn học được sử

dụng trong tính năng quản lý điểm số, các thay đổi này ngay lập tức phản ánh trên giao diện người dùng để họ có đủ lựa chọn khi lập kế hoạch. quản lý thông báo toàn hệ thống, tạo thông báo chung (cập nhật phiên bản, bảo trì, sự kiện quan trọng) thông qua form nhập tiêu đề và nội dung; khi lưu, hệ thống gửi notification tới tất cả người dùng đang đăng nhập và lưu bản sao vào bảng `system_announcements` để người dùng có thể xem lại lịch sử. theo dõi nhật ký hoạt động, mọi hành động quan trọng của người dùng (đăng nhập, tạo/chỉnh sửa/xóa công việc, thay đổi trạng thái, tải lên tệp, ...) được ghi vào bảng `task_activities/admin_logs`; admin có thể lọc theo thời gian, người thực hiện hoặc loại hành động và xuất báo cáo dưới dạng CSV để kiểm tra, audit hoặc xử lý sự cố. cấu hình hệ thống, cho phép chỉnh sửa các thông tin chung như tên trường, logo, email liên hệ và các tham số vận hành (thời gian timeout JWT, mức giới hạn tải file); các thay đổi được lưu trong bảng `system_settings` và áp dụng ngay mà không cần khởi động lại dịch vụ. Các chức năng này đáp ứng đầy đủ yêu cầu quản trị, bảo trì và cấu hình của hệ thống TaskFlow, đồng thời đảm bảo tính gọn gàng, an toàn và có khả năng truy xuất mọi thao tác của người dùng.

3.2. Phân tích hệ thống

3.2.1 Đặc tả yêu cầu hệ thống

3.2.1.1 Yêu cầu chức năng

Hệ thống cho phép người dùng quản lý công việc cá nhân một cách toàn diện. Người dùng có thể tạo, cập nhật và theo dõi các công việc với bốn trạng thái rõ ràng: Cần làm, đang làm, đã xong và quá hạn. Mỗi công việc được phân loại theo danh mục và độ ưu tiên, đồng thời hỗ trợ thêm checklist con, bình luận, tệp đính kèm và theo dõi lịch sử thay đổi chi tiết thông qua nhật ký hoạt động. Khi cần sắp xếp thứ tự thực hiện, người dùng có thể liên kết các công việc lại với nhau bằng sơ đồ phụ thuộc; hệ thống sẽ tự động kiểm tra và ngăn chặn mọi vòng lặp phụ thuộc để tránh lỗi logic.

Đặc biệt, hệ thống tích hợp trợ lý AI của google gemini, giúp người dùng tạo công việc một cách nhanh chóng từ các câu lệnh tiếng Việt, phân rã các công việc lớn thành các bước nhỏ, tư vấn cách sắp xếp thời gian hợp lý và phân tích hiệu suất

làm việc. Ngoài ra, AI còn có khả năng sinh câu hỏi ôn tập tự động từ những ghi chú học tập, hỗ trợ người dùng trong việc học tập hiệu quả.

Trong môi trường làm việc nhóm, người dùng có thể tạo dự án, quản lý thành viên, theo dõi các mốc quan trọng và trao đổi thông tin thời gian thực qua chat nhóm nhờ socket.io. Hệ thống còn cho phép người dùng kết bạn, xem hồ sơ của nhau và tương tác xã hội, tạo nên một cộng đồng năng động.

Về quản lý thời gian và học tập, hệ thống cung cấp lịch biểu tuần, công cụ lập kế hoạch theo khung giờ (Timebox Planner), bộ hẹn giờ pomodoro kèm nhạc nền giúp duy trì sự tập trung, và công cụ habit tracker để theo dõi thói quen hàng ngày. Người dùng cũng có thể quản lý điểm số các môn học, tạo flashcards để ôn tập và lên kế hoạch học tập theo phương pháp OKR, tất cả đều được tích hợp trong một giao diện thống nhất.

Để người dùng nắm bắt được tiến độ công việc, dashboard hiển thị tổng quan công việc, biểu đồ tròn thể hiện trạng thái, biểu đồ cột thể hiện tiến độ theo tuần và danh mục, cùng các thống kê nhanh. Hệ thống hỗ trợ xuất báo cáo dưới dạng PDF (được tạo ngay trên trình duyệt bằng html2pdf.js) và xuất dữ liệu CSV từ backend cho việc phân tích sâu hơn. Cuối cùng, hệ thống gamification với XP, level và RPG profile tạo động lực, khuyến khích người dùng hoàn thành nhiệm vụ và cảm thấy hứng thú hơn trong quá trình làm việc.

3.2.1.2 Đối với quản trị viên

Quản trị viên có quyền truy cập vào giao diện quản lý toàn bộ tài khoản người dùng. Họ có thể xem, tìm kiếm, cập nhật thông tin (tên, email, vai trò) và xóa tài khoản khi cần thiết. Ngoài ra, admin còn có thể xuất danh sách người dùng dưới dạng CSV để phục vụ công việc báo cáo và lưu trữ.

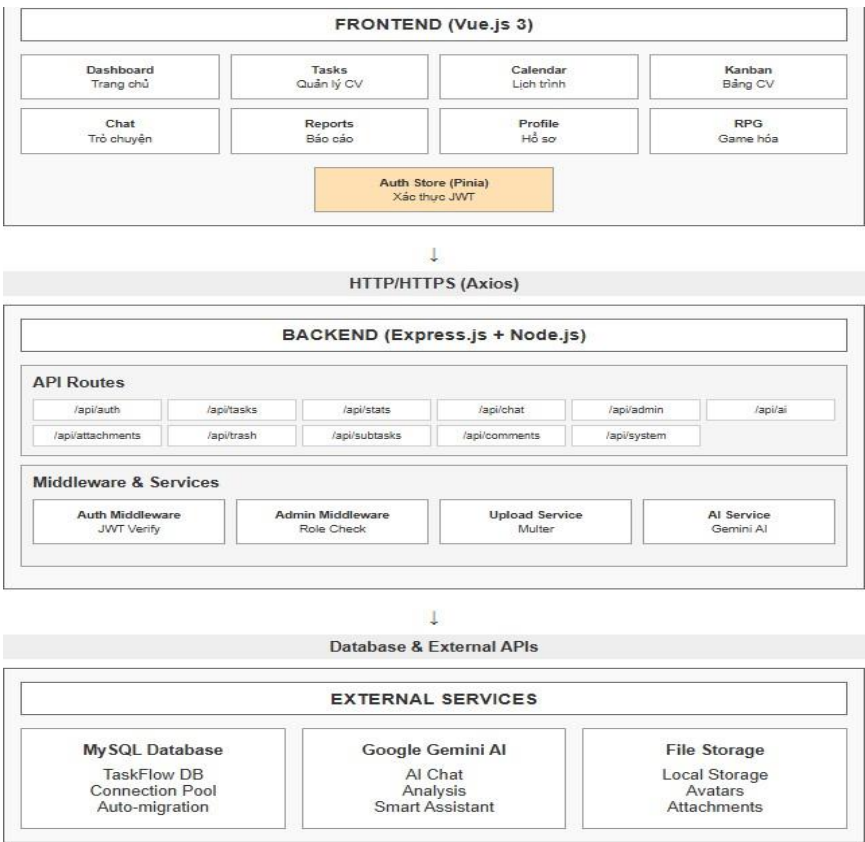
Về cấu trúc dữ liệu, admin chịu trách nhiệm quản lý các danh mục công việc và các môn học, có thể thêm, sửa hoặc xóa chúng để đảm bảo người dùng luôn có đủ lựa chọn khi lập kế hoạch. Khi có thông báo quan trọng như cập nhật phiên bản, bảo trì hoặc các sự kiện quan trọng, admin tạo và gửi thông báo toàn hệ thống, thông tin

này sẽ được đẩy tới tất cả người dùng đang online và cũng lưu lại để người dùng có thể xem lại lịch sử.

Để giám sát hoạt động của hệ thống, admin theo dõi nhật ký hoạt động, tức là toàn bộ các thao tác của người dùng như đăng nhập, tạo, chỉnh sửa hoặc xóa công việc, thay đổi trạng thái và tải lên tệp. Các nhật ký này được lưu trữ chi tiết và admin có thể lọc, tìm kiếm và xuất ra file CSV để kiểm tra, audit hoặc xử lý sự cố khi cần.

Cuối cùng, admin có thể cấu hình các thông tin chung của hệ thống như tên trường, logo, email liên hệ và các tham số vận hành (ví dụ thời gian timeout JWT, giới hạn tải file). Những thay đổi này được lưu trong bảng cấu hình và có hiệu lực ngay lập tức mà không cần khởi động lại dịch vụ. Nhờ những chức năng trên, admin có thể duy trì, bảo trì và phát triển hệ thống một cách hiệu quả, đồng thời đảm bảo tính ổn định và an toàn cho toàn bộ người dùng.

3.2.2 Kiến trúc hệ thống



Hình 3.1: Kiến trúc hệ thống

Hệ thống được thiết kế và xây dựng theo mô hình kiến trúc 3 lớp tiêu chuẩn, bao gồm: tầng hiển thị (Frontend), tầng xử lý nghiệp vụ (Backend) và tầng lưu trữ dữ liệu (Database). Việc giao tiếp giữa các tầng được thực hiện thông qua chuẩn giao diện lập trình ứng dụng RESTful API (Representational State Transfer), đảm bảo tính hoạt động độc lập và tính đóng gói cao giữa các phân hệ chức năng. Bên cạnh đó, hệ thống còn tích hợp dịch vụ trí tuệ nhân tạo (AI) tạo sinh và các dịch vụ mở rộng từ bên ngoài nhằm tối ưu hóa trải nghiệm người dùng, đồng thời mang lại hiệu năng xử lý linh hoạt và toàn diện.

3.2.3 Các tầng thành phần chính trong kiến trúc

Tầng đầu tiên là tầng hiển thị (Presentation layer), đóng vai trò là giao diện trực tiếp tương tác và tiếp nhận phản hồi từ phía người dùng, được phát triển bằng khung làm việc vue.js. Tầng này chịu trách nhiệm hiển thị toàn bộ các thành phần giao diện bao gồm: trang tổng quan (Dashboard thống kê), bảng quản lý danh sách công việc (Kanban/Task List), cửa sổ chi tiết nhiệm vụ (Task Modal), giao diện trò chuyện với trí tuệ nhân tạo (AI Chatbox) và bảng điều khiển dành cho quản trị viên. Việc quản lý trạng thái của ứng dụng (State Management) được điều phối thông qua ngữ cảnh xác thực tài khoản sử dụng thư viện pinia kết hợp cùng hệ thống định tuyến Vue Router.

Tầng nghiệp vụ (Business logic layer) vận hành trên môi trường thực thi node.js kết hợp cùng khung điều phối express.js [\[11\]](#). Tầng này quản lý các nhóm tuyến đường (Routes) giao diện lập trình ứng dụng cốt lõi nhằm tiếp nhận và xử lý các yêu cầu gửi về từ tầng hiển thị, bao gồm các nhóm: xác thực (auth), công việc (tasks), công việc con (subtasks), bình luận (comments), tệp đính kèm (attachments), thống kê (stats) và tương tác trí tuệ nhân tạo (ai). Các lớp phần mềm trung gian (Middleware) được cài đặt tại đây để thực thi việc xác thực chuỗi JWT Token, cấu hình chia sẻ tài nguyên chéo (CORS), phân tích dữ liệu đa phương tiện để tải tệp lên (Multer) và kiểm soát xử lý lỗi ngầm.

Tầng thứ ba là tầng dữ liệu (Data Access Layer), bảo toàn tính lưu trữ thông tin ứng dụng thông qua hệ quản trị cơ sở dữ liệu quan hệ mysql. Dữ liệu tại đây được

tổ chức khoa học thành các bảng (Tables) có tính liên kết chặt chẽ bao gồm: bảng người dùng (users), bảng công việc (tasks), bảng nhiệm vụ con (subtasks), bảng bình luận (task_comments), bảng tập tin đính kèm (task_attachments) và bảng lưu trữ sự phụ thuộc (task_dependencies). Quá trình kết nối, ràng buộc cấu trúc và thao tác với cơ sở dữ liệu được thực hiện thông qua thư viện mysql2 để thi hành các câu lệnh sql tối ưu hóa tốc độ truy vấn.

Tầng dịch vụ mở rộng từ bên ngoài đóng vai trò kết nối ứng dụng với các nền tảng công nghệ toàn cầu. Hệ thống tích hợp dịch vụ trí tuệ nhân tạo thông qua giao diện lập trình ứng dụng của google (Generative AI - mô hình Gemini) để vận hành bộ não của trợ lý ảo, cung cấp khả năng tư vấn hướng giải quyết công việc, phân tích ngữ cảnh và phản hồi ngôn ngữ tự nhiên

3.2.4 Mô tả luồng xử lý dữ liệu

Luồng đăng nhập và xác thực tài khoản: Thực hiện nhiệm vụ xác thực danh tính thành viên và thiết lập phiên làm việc bảo mật trên toàn hệ thống. Khi người dùng cung cấp tài khoản từ giao diện tầng hiển thị, hệ thống sẽ tiếp nhận yêu cầu thông qua phương thức POST gửi tới tuyến đường /api/auth/login. Quá trình xử lý tại máy chủ bao gồm việc truy xuất dữ liệu trong cơ sở dữ liệu mysql và áp dụng giải thuật đối chiếu mật khẩu thông qua thư viện băm dữ liệu bcryptjs. Nếu thông số hoàn toàn chuẩn xác, hệ thống sẽ khởi tạo một mã xác thực điện tử (JWT) chứa thông tin định danh và quyền hạn của tài khoản rồi gửi trả về cho tầng hiển thị để lưu trữ trực tiếp vào bộ nhớ cục bộ (Local Storage), giúp duy trì trạng thái đăng nhập hợp lệ.

Luồng trợ lý ảo trí tuệ nhân tạo (Gemini AI): Chịu trách nhiệm hỗ trợ tư vấn, phân nhỏ nhiệm vụ và hướng dẫn giải quyết công việc một cách tự động. Đầu vào của tiến trình là đoạn văn bản câu hỏi của người dùng gửi từ giao diện chatbox, được máy chủ tiếp nhận thông qua phương thức POST. Hệ thống thực hiện thu thập trước các ngữ cảnh liên quan (nếu có) để truyền vào Prompts. Sau đó, hệ thống gửi dữ liệu đến máy chủ của google để gọi mô hình gemini thực hiện xử lý ngôn ngữ tự nhiên. Kết quả đầu ra là chuỗi văn bản định dạng markdown được trả về tầng hiển thị để render ngay lập tức thành văn bản trực quan có cấu trúc trên cửa sổ trò chuyện.

Luồng khởi tạo và phân rã công việc (Task & Subtask Creation): Cho phép người dùng tạo lập kế hoạch và phân bổ các bước thực hiện chi tiết. Thông tin đầu vào bao gồm tiêu đề, mô tả, danh mục, độ ưu tiên và thời hạn (Due Date) được gửi qua phương thức POST. Quá trình xử lý tại tầng nghiệp vụ tiến hành xác thực tính hợp lệ của dữ liệu, tự động đánh dấu trạng thái khởi tạo (To do) và ghi nhận bản ghi mới vào bảng tasks. Khi người dùng tiến hành chẻ nhỏ công việc thành các bước nhỏ hơn, hệ thống sẽ tiếp tục ghi nhận các bản ghi vào bảng subtasks với khóa ngoại liên kết trực tiếp tới công việc gốc, đem lại khả năng theo dõi tiến độ chi tiết và trực quan.

Luồng thiết lập ràng buộc và ngăn chặn vòng lặp phụ thuộc (Circular Dependency Check): Nhằm mục đích quản lý thứ tự thực hiện khi một công việc B bắt buộc phải chờ công việc A hoàn thành. Khi thiết lập mối quan hệ này, máy chủ sẽ tiếp nhận mã định danh của hai công việc. Nhằm ngăn chặn lỗi logic dẫn đến tình trạng khóa chéo (công việc A chờ B, nhưng B lại chờ A), hệ thống áp dụng thuật toán duyệt đồ thị có hướng (Directed Graph Traversal - Breadth First Search) để dò tìm toàn bộ các liên kết đã có trong cơ sở dữ liệu. Nếu phát hiện vòng lặp, hệ thống lập tức từ chối và cảnh báo người dùng. Nếu an toàn, hệ thống ghi nhận mối quan hệ vào bảng task_dependencies.

Luồng xử lý dữ liệu thống kê (Dashboard & Analytics): Đóng vai trò cung cấp dữ liệu định lượng để người dùng đưa ra cái nhìn tổng quan về hiệu suất làm việc. Khi truy cập trang chủ, giao diện gửi yêu cầu GET tới tuyến đường /api/stats. Hệ thống tại backend sẽ thực thi chuỗi câu lệnh SQL đếm số lượng công việc theo từng nhóm (Đã hoàn thành, đang thực hiện, trễ hạn), nhóm theo độ ưu tiên và thống kê theo từng danh mục. Kết quả trả về được định dạng JSON, sau đó tầng frontend tiếp nhận và sử dụng thư viện chart.js để render thành các biểu đồ trực quan (pie chart, bar chart).

Luồng cập nhật trạng thái tự động theo thời gian thực: Để đảm bảo tiến độ luôn được phản ánh chính xác, mỗi khi có yêu cầu truy xuất dữ liệu danh sách công việc, hệ thống luôn tự động chạy một dịch vụ ngầm. Dịch vụ này thực thi câu lệnh SQL so sánh mốc thời gian thời hạn (Due Date) của tất cả công việc với thời gian

thực tế hiện tại của máy chủ. Bất kỳ công việc nào chưa hoàn thành nhưng đã vượt quá thời hạn sẽ được tự động cập nhật trạng thái sang Trễ hạn (overdue), giúp người dùng luôn nắm bắt được các nhiệm vụ cấp bách cần giải quyết ngay lập tức.

3.3. Phân tích thiết kế hệ thống

Các tác nhân của hệ thống

Bảng 3.1: Các tác nhân của hệ thống

Tác nhân	Mô tả chức năng hệ thống
Khách (Guest)	Người dùng chưa đăng nhập. Chỉ có thể thực hiện thao tác: Đăng nhập, đăng ký tài khoản mới và khôi phục mật khẩu. (Hệ thống yêu cầu xác thực bắt buộc để bảo mật dữ liệu công việc).
Người dùng (User)	Quản lý công việc: Thêm, sửa, xóa, đổi trạng thái/ưu tiên, tạo công việc con (subtasks) và thiết lập ràng buộc phụ thuộc. Tương tác: Thêm bình luận và tải lên tệp đính kèm vào từng công việc. Trợ lý AI: Chat với AI (Gemini) để nhận tư vấn, gợi ý cách giải quyết công việc. Tiện ích: Xem biểu đồ thống kê cá nhân (Dashboard), quản lý hồ sơ và khôi phục dữ liệu từ thùng rác.
Admin	Quản lý tài khoản: Xem danh sách người dùng toàn hệ thống, khóa/xóa tài khoản vi phạm và phân quyền admin. Giám sát hệ thống: Xem báo cáo thống kê tổng quan (tổng lượng người dùng, tổng số công việc, hiệu suất chung của nền tảng).

3.3.1 Sơ đồ UseCase



Hình 3.2: Sơ đồ UseCase

Sơ đồ thể hiện cái nhìn toàn cảnh về toàn bộ các chức năng cốt lõi và mối quan hệ tương tác giữa ba tác nhân chính bao gồm khách, người dùng cuối và quản trị viên với nền tảng. Việc thiết lập sơ đồ tổng quát này đóng vai trò định hình cấu trúc vận hành, giúp liên kết các nhóm tính năng từ tuyến đầu của người dùng đến khu vực điều hành chuyên sâu của quản trị viên thành một thể thống nhất trước khi đi vào phân rã chi tiết từng tiến trình xử lý dữ liệu.

Đối với tác nhân khách (Guest), do đặc thù của hệ thống quản lý công việc đòi hỏi tính bảo mật dữ liệu cá nhân cao, hệ thống sẽ thiết lập rào cản truy cập ngay từ tuyến ngoài cùng. Chức năng cốt lõi của khách xoay quanh chuỗi hành vi chuyển

đổi thành viên. Khách được cung cấp chức năng đăng ký để khởi tạo không gian làm việc mới, chức năng khôi phục mật khẩu trong trường hợp thất lạc thông tin, và chức năng đăng nhập nhằm thiết lập quyền truy cập bảo mật vào hệ thống. Mọi truy cập vào các trang nội bộ đều bị chặn đứng nếu khách chưa hoàn tất tiến trình đăng nhập hợp lệ.

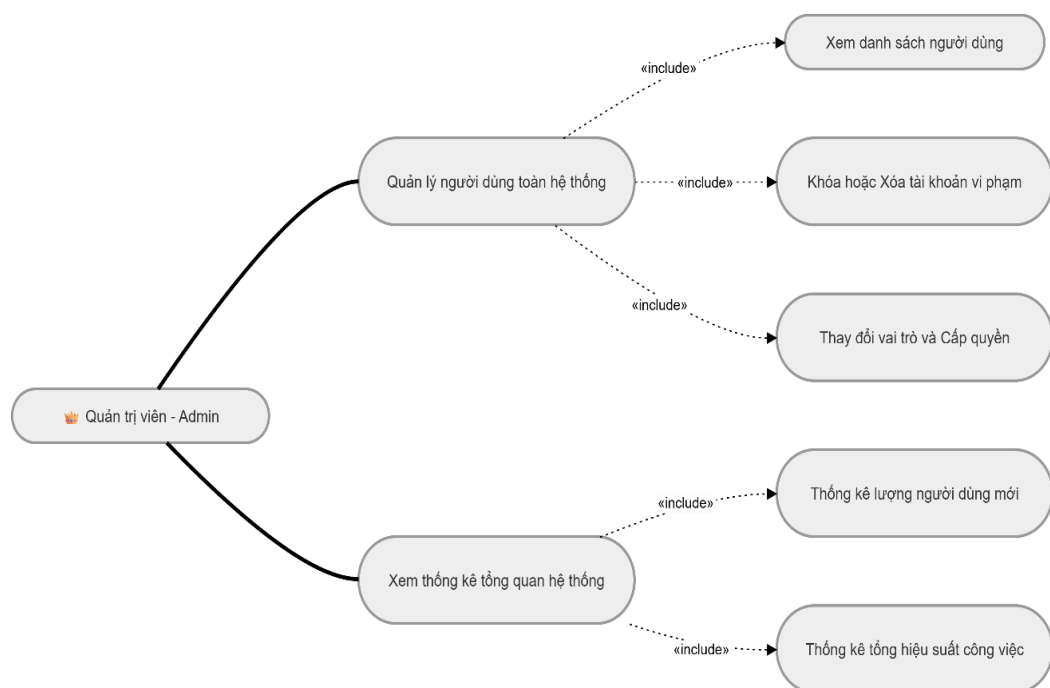
Tiếp nối luồng vận hành, khi khách thực hiện đăng nhập thành công sẽ tự động chuyển đổi vai trò thành tác nhân người dùng cuối (User) để mở khóa các đặc quyền quản lý công việc cá nhân. Người dùng có thể tương tác với chức năng cập nhật hồ sơ, giúp thay đổi các thông tin cá nhân cơ bản và đổi mật khẩu. Tính năng trung tâm của người dùng là quản lý danh sách công việc (Tasks), tiến trình này bắt buộc bao hàm các chức năng tác vụ cốt lõi là thêm mới, chỉnh sửa nội dung, thay đổi trạng thái (To do, In progress, Done) và xóa công việc. Để tối ưu hóa quy trình làm việc, người dùng được cung cấp chức năng quản lý chi tiết, luồng xử lý này bao hàm việc chia nhỏ công việc thành các nhiệm vụ phụ (Subtasks) và thiết lập các mối ràng buộc phụ thuộc (Dependencies) chặt chẽ giữa các nhiệm vụ.

Nhằm tăng cường khả năng lưu trữ thông tin, chức năng quản lý tương tác cho phép người dùng đính kèm các tệp tài liệu quan trọng và viết bình luận thảo luận trực tiếp ngay bên trong từng thẻ công việc. Nổi bật nhất trong hệ thống là chức năng trò chuyện với trợ lý ảo (Gemini AI), tiến trình tương tác này bắt buộc bao hàm chức năng phân tích ngữ cảnh, giúp người dùng nhận được các gợi ý chuyên sâu về cách thức giải quyết hoặc cách phân bổ thời gian hợp lý cho một công việc cụ thể. Cuối cùng, người dùng có quyền truy cập chức năng xem thống kê (Dashboard) để theo dõi biểu đồ hiệu suất làm việc của chính mình, và chức năng quản lý thùng rác giúp khôi phục các công việc lỡ xóa hoặc tiến hành xóa vĩnh viễn để dọn dẹp bộ nhớ.

Ở cấp độ điều phối và giám sát tối cao, tác nhân quản trị viên (Admin) thực hiện kết nối vào phân hệ điều hành, kế thừa toàn bộ các đặc quyền của người dùng thông thường và được mở khóa thêm các tính năng kiểm soát hệ thống. Chức năng đầu tiên của quản trị viên là xem thống kê hệ thống, giúp truy xuất dữ liệu tự động để hiển thị các biểu đồ tổng quan về hiệu suất hoạt động, số lượng tài khoản mới

đăng ký và tổng lượng công việc đang lưu trữ trên toàn bộ nền tảng. Tiếp đến, chức năng quản lý người dùng giúp điều phối danh sách thành viên trên toàn hệ thống, tiến trình xử lý này bắt buộc bao hàm hai chức năng tác vụ cốt lõi là phân quyền tài khoản (cấp hoặc thu hồi quyền admin) và khóa/xóa người dùng đối với các tài khoản có dấu hiệu bất thường. Mọi chức năng tổng quát của quản trị viên đều hướng đến việc bảo vệ tính toàn vẹn dữ liệu, kiểm soát luồng truy cập và duy trì sự ổn định cho toàn bộ nền tảng ứng dụng Taskflow.

3.3.2 Tác nhân admin



Hình 3.3: Tác nhân admin

Trong hệ thống Taskflow, quản trị viên đóng vai trò điều phối và giám sát toàn bộ hoạt động của nền tảng. Các chức năng mà quản trị viên được phân quyền bao gồm: quản lý tài khoản người dùng, phân quyền hệ thống, quản lý danh mục công việc mặc định, giám sát cấu hình trợ lý ảo AI và theo dõi bảng điều khiển thống kê tổng quan. Mỗi chức năng này đều được thiết kế nhằm đảm bảo tính toàn vẹn dữ liệu, hiệu năng hoạt động và độ bảo mật cao nhất cho toàn bộ hệ thống.

Chức năng đăng nhập hệ thống quản trị là bước khởi đầu bắt buộc. Đầu vào bao gồm thông tin đăng nhập của quản trị viên (email và mật khẩu). Sau khi được hệ

thống xác thực, một lớp phần mềm trung gian (Middleware) sẽ tiến hành giải mã chuỗi JWT Token để kiểm tra và xác nhận quyền hạn tài khoản bắt buộc phải mang nhân admin. Nếu đáp ứng đầy đủ điều kiện, phiên làm việc quản trị tối cao mới được thiết lập. Cơ chế này nhằm đảm bảo rằng chỉ những cá nhân được cấp quyền hợp lệ mới có thể truy cập vào khu vực điều hành chuyên sâu.

Chức năng quản lý tài khoản người dùng, cho phép kiểm soát toàn bộ thành viên tham gia nền tảng, đặc biệt là khi xảy ra các hành vi vi phạm chính sách hoặc cần hỗ trợ kỹ thuật. Quản trị viên có thể tìm kiếm người dùng theo tên hoặc email để xem lịch sử hoạt động cơ bản. Dữ liệu đầu vào là thông tin định danh người dùng, đầu ra là việc cập nhật trạng thái tài khoản trực tiếp trong cơ sở dữ liệu mysql (ví dụ: khóa tạm thời hoặc xóa vĩnh viễn tài khoản).

Gắn liền với việc quản lý tài khoản là chức năng phân quyền hệ thống. Quản trị viên có thể thay đổi vai trò (Role) của một người dùng thông thường thành quản trị viên cấp cao hoặc ngược lại. Việc phân quyền chặt chẽ giúp tổ chức và điều phối nhân sự quản lý nền tảng một cách linh hoạt khi hệ thống mở rộng quy mô.

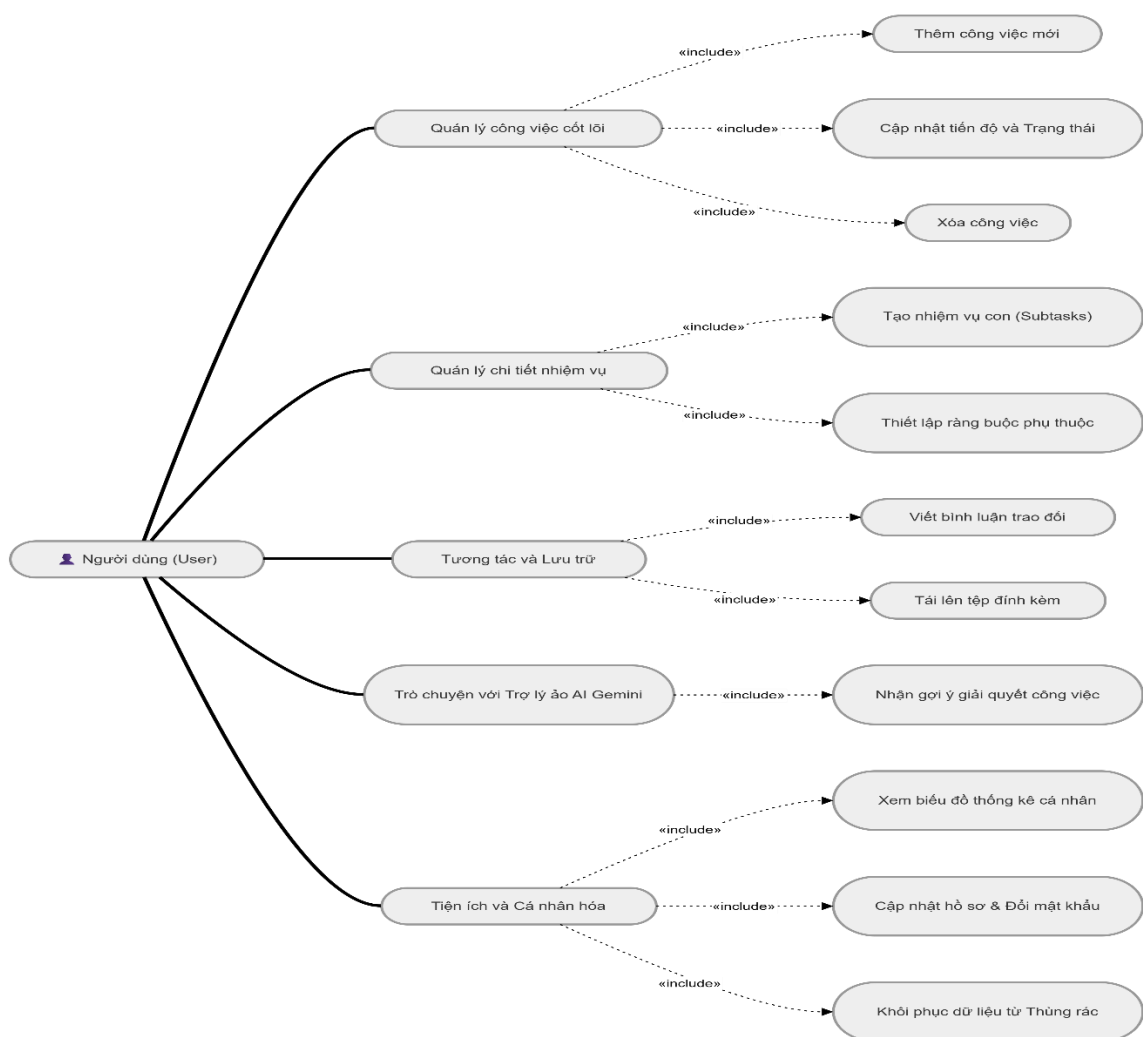
Hệ thống cung cấp chức năng quản lý danh mục công việc. Tại đây, quản trị viên có thể tổ chức và thiết lập các bộ danh mục mặc định cho toàn hệ thống (ví dụ: Học tập, công việc, cá nhân, sức khỏe). Đầu vào là tên danh mục, hệ thống sẽ thực hiện tạo mới hoặc cập nhật vào kho dữ liệu. Việc phân loại khoa học từ cấp hệ thống giúp chuẩn hóa dữ liệu và tối ưu hóa trải nghiệm sử dụng bộ lọc của người dùng cuối.

Đối với xu hướng tự động hóa, hệ thống cung cấp chức năng quản lý cấu hình trợ lý ảo AI (Gemini). Tại đây, quản trị viên có thể giám sát các tham số hệ thống, theo dõi tần suất gọi API từ phía người dùng để tránh tình trạng quá tải. Ngoài ra, quản trị viên còn có thể tinh chỉnh các câu lệnh hệ thống (System Prompts) mặc định. Điều này giúp mô hình trí tuệ nhân tạo ngày càng hiểu rõ bối cảnh công việc hơn, từ đó đưa ra các tư vấn phân rã nhiệm vụ hoặc sắp xếp thời gian có độ chính xác cao.

Chức năng xem bảng điều khiển thống kê (Dashboard) đóng vai trò cung cấp cái nhìn toàn cảnh về “sức khỏe” của nền tảng. Thông qua các biểu đồ trực quan, quản trị viên có thể theo dõi các chỉ số quan trọng như: tổng số lượng tài khoản mới

đăng ký, tổng số lượng công việc (tasks) được tạo ra trên toàn hệ thống, và tỷ lệ hoàn thành công việc trung bình theo mốc thời gian. Dựa trên dữ liệu thực tế này, ban quản trị có thể phân tích chính xác xu hướng sử dụng của người dùng để đưa ra các quyết định nâng cấp máy chủ hoặc phát triển thêm tính năng mới kịp thời. Quá trình thống kê này không yêu cầu đầu vào trực tiếp mà được thực hiện thông qua việc truy vấn và tổng hợp dữ liệu tự động từ các bảng trong cơ sở dữ liệu.

3.3.3 Tác nhân người dùng



Hình 3.4: Tác nhân người dùng

Khi người dùng đã hoàn tất đăng ký và đăng nhập thành công, họ sẽ được cấp quyền truy cập vào không gian làm việc cá nhân hóa với các chức năng cao cấp như: quản lý công việc, quản lý tiến độ, thiết lập các mối ràng buộc phụ thuộc, tương tác tài liệu và trò chuyện với Trợ lý ảo AI.

Chức năng cơ bản đầu tiên là đăng nhập hệ thống. Người dùng sẽ nhập địa chỉ email và mật khẩu đã đăng ký trước đó. Hệ thống tiếp nhận thông tin, tiến hành băm dữ liệu bằng thư viện bcryptjs và so sánh với chuỗi mã hóa lưu trữ trong cơ sở dữ liệu mysql. Nếu hợp lệ, hệ thống tạo phiên làm việc mới, cấp mã xác thực điện tử (JWT Token) trả về bộ nhớ cục bộ (Local Storage) của trình duyệt và cho phép truy cập vào khu vực làm việc (Dashboard) với các đặc quyền riêng tư.

Tiếp đến, quản lý công việc là chức năng trung tâm của hệ thống. Người dùng khởi tạo một nhiệm vụ mới bằng cách nhập các thông tin cơ bản gồm: tiêu đề công việc, mô tả, danh mục, mức độ ưu tiên (Thấp/Trung bình/Cao) và thời hạn hoàn thành (Due Date). Hệ thống tiếp nhận và ghi nhận một bản ghi mới vào bảng tasks với trạng thái mặc định ban đầu là “Cần làm” (To do). Sau đó, người dùng có thể tự do tìm kiếm, chia nhỏ công việc này thành các nhiệm vụ con (Subtasks) để dễ dàng thực hiện từng bước.

Kèm theo đó là chức năng theo dõi và cập nhật tiến độ, cho phép người dùng xem lại toàn bộ danh sách kế hoạch đã lập (dưới dạng danh sách hoặc bảng Kanban), thực hiện chỉnh sửa nội dung, hoặc chuyển các công việc không còn cần thiết vào thùng rác. Đồng thời, người dùng có thể linh hoạt cập nhật trạng thái công việc tiến triển dần từ “Cần làm” (To do) sang “Đang thực hiện” (In progress) hoặc “Đã hoàn thành” (Done) tùy theo thực tế công việc. Mỗi thao tác thay đổi đều được hệ thống tự động lưu vết vào bảng nhật ký hoạt động (task_activities).

Một tính năng mở rộng đặc biệt để chống chồng chéo công việc là thiết lập ràng buộc phụ thuộc (Dependencies). Khi người dùng thiết lập quy tắc “Công việc B chỉ được bắt đầu khi Công việc A hoàn thành”, hệ thống sẽ chạy thuật toán ngàm duyệt đồ thị có hướng (Directed Graph Traversal). Tại đây, hệ thống tính toán để tự động dò tìm và ngăn chặn lỗi vòng lặp phụ thuộc (Circular dependency - khóa chéo lẫn nhau), đảm bảo luồng công việc luôn được sắp xếp theo một trình tự logic và khả thi nhất.

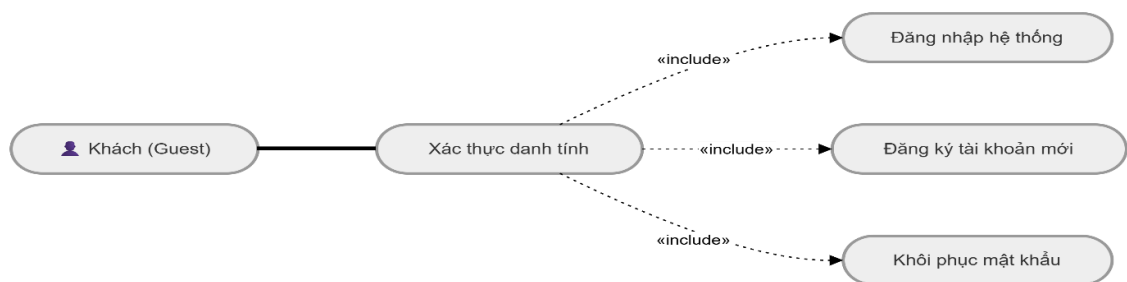
Trong quá trình giải quyết công việc, người dùng còn có thể sử dụng chức năng tương tác và lưu trữ. Tại cửa sổ chi tiết của mỗi công việc, người dùng có thể

tải lên các tệp tài liệu đính kèm (Attachments) hoặc viết các bình luận (Comments) ghi chú lại quá trình làm việc. Thao tác này giúp tập trung toàn bộ dữ liệu quan trọng vào một nơi, tránh thất lạc thông tin như khi sử dụng các phần mềm nhắn tin thông thường.

Đặc biệt, hệ thống cung cấp chức năng trò chuyện với trợ lý ảo AI (Gemini). Giao diện chatbox sẽ trực tiếp kết nối với API của google, cho phép người dùng nhập các câu hỏi bằng ngôn ngữ tự nhiên. Trợ lý ảo có khả năng phân tích ngữ cảnh, tư vấn cách giải quyết một bài toán học búa, hoặc đưa ra lời khuyên về cách phân bổ thời gian hợp lý khi khối lượng công việc quá tải. Văn bản trả về từ AI được định dạng theo cấu trúc markdown, giúp người dùng dễ dàng đọc hiểu và sao chép để áp dụng ngay vào bảng công việc của mình.

Cuối cùng, chức năng xem bảng điều khiển (Dashboard) giúp người dùng tự đánh giá năng suất cá nhân. Hệ thống tự động thu thập dữ liệu và vẽ các biểu đồ trực quan thống kê số lượng công việc đã hoàn thành, số lượng công việc đang trễ hạn, giúp người dùng luôn nắm bắt được tổng quan tình hình và điều chỉnh nhịp độ làm việc hiệu quả.

3.3.4 Tác nhân khách



Hình 3.5: Tác nhân khách

Trong hệ thống Taskflow, dữ liệu công việc mang tính chất riêng tư và bảo mật cao. Do đó, người dùng ở trạng thái khách (chưa đăng nhập) sẽ bị giới hạn quyền truy cập và chỉ được phép sử dụng cụm chức năng xác thực danh tính. Cụm chức năng này đóng vai trò như một “cánh cửa” bảo vệ hệ thống, nhằm thu hút người dùng mới đăng ký và hỗ trợ họ thiết lập quyền truy cập một cách an toàn nhất.

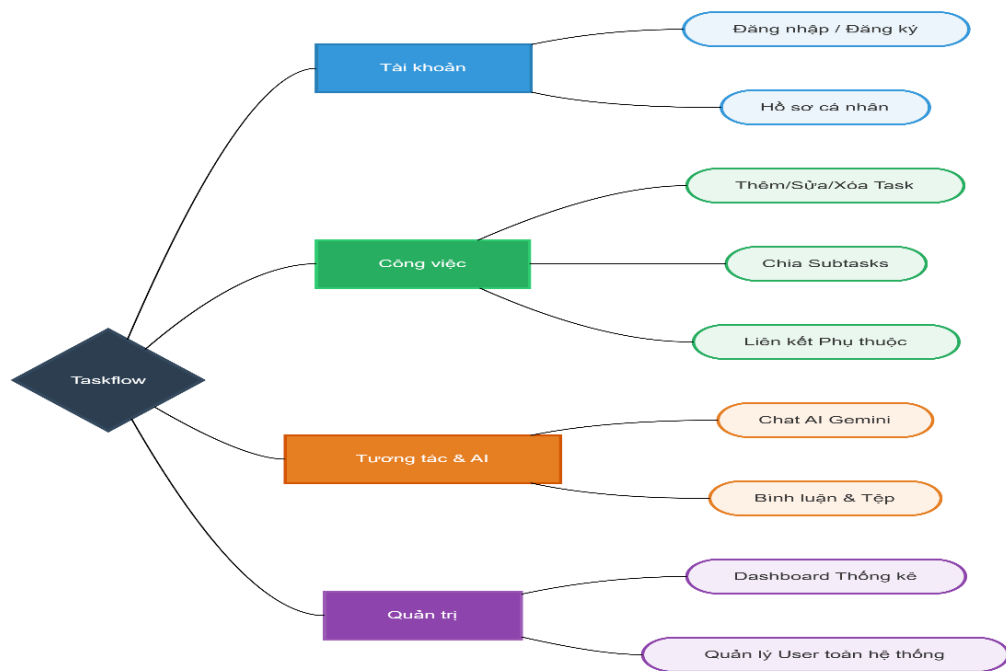
Chức năng đầu tiên và quan trọng nhất đối với khách là đăng ký tài khoản mới. Chức năng này giúp chuyển đổi người dùng từ trạng thái khách vắng lai sang người dùng thực thụ của hệ thống. Đầu vào yêu cầu khách cung cấp các thông tin cơ bản như: Tên hiển thị, địa chỉ email hợp lệ và mật khẩu. Sau khi tiếp nhận, hệ thống tại backend tiến hành xác thực định dạng dữ liệu, kiểm tra tính duy nhất của email (tránh đăng ký trùng lặp). Tiếp đó, mật khẩu sẽ được mã hóa bảo mật thông qua thư viện băm dữ liệu bcryptjs trước khi ghi nhận vào cơ sở dữ liệu mysql. Kết quả đầu ra là một tài khoản mới được khởi tạo thành công.

Sau khi có tài khoản, khách sẽ sử dụng chức năng đăng nhập hệ thống. Mục tiêu của chức năng này là thiết lập phiên làm việc và cấp quyền truy cập. Khách nhập địa chỉ email và mật khẩu. Hệ thống sẽ truy xuất dữ liệu, giải mã và đối chiếu mật khẩu. Nếu hợp lệ, hệ thống khởi tạo một mã xác thực điện tử (JWT Token), gửi trả về trình duyệt và tự động điều hướng khách vào giao diện bảng điều khiển (Dashboard), chính thức chuyển đổi vai trò của họ thành Người dùng (User).

Ngoài ra, nhằm hỗ trợ các trường hợp khách không thể truy cập tài khoản, hệ thống cung cấp chức năng khôi phục mật khẩu (Quên mật khẩu). Khách hàng chỉ cần nhập địa chỉ email đã đăng ký. Hệ thống sẽ kiểm tra sự tồn tại của tài khoản trong cơ sở dữ liệu. Nếu khớp, hệ thống sẽ thực hiện luồng xử lý cấp lại quyền truy cập (thông qua mã OTP hoặc liên kết xác nhận gửi vào email), cho phép khách tiến hành thiết lập lại một mật khẩu mới đã được mã hóa an toàn để tiếp tục sử dụng dịch vụ.

Mọi nỗ lực truy cập vào các đường dẫn (URL) quản lý công việc, bảng thông kê hay giao diện chat với AI từ phía khách đều sẽ bị phần mềm trung gian (Middleware) của hệ thống chặn lại và lập tức điều hướng (Redirect) trả về trang đăng nhập, nhằm đảm bảo tính toàn vẹn và bảo mật tuyệt đối cho nền tảng Taskflow

3.4. Sơ đồ website



Hình 3.6: Sơ đồ website

Sơ đồ phân cấp chức năng của hệ thống được xây dựng theo mô hình phân quyền rõ ràng, phân rã thành các phân hệ lớn tương ứng với ba nhóm đối tượng tương tác trực tiếp bao gồm: Khách, người dùng và quản trị viên. Hệ thống chính đóng vai trò là hạt nhân trung tâm điều phối và phân luồng dữ liệu, giúp chuyển tiếp chính xác các yêu cầu từ giao diện người dùng đến các cấu trúc xử lý nghiệp vụ cơ sở bên dưới.

Đối với nhóm đối tượng là khách (Guest), do yêu cầu bảo mật nghiêm ngặt đối với dữ liệu công việc cá nhân, hệ thống đóng kín các cổng chức năng nội bộ và chỉ mở ra phân hệ xác thực danh tính tại tuyến đầu. Luồng chức năng khởi đầu từ việc đăng ký tài khoản mới nhằm thu thập thông tin và kích hoạt hành vi chuyển đổi thành viên. Cùng với đó, chức năng đăng nhập hệ thống chính là cửa ngõ cốt lõi giúp khách cung cấp thông tin xác thực để chính thức nâng cấp quyền hạn. Trong trường hợp thất lạc thông tin, chức năng khôi phục mật khẩu đóng vai trò như một tác vụ hỗ trợ trực tiếp, giúp gửi mã xác nhận và khôi phục lại quyền truy cập một cách an toàn.

Sau khi xác thực thành công, tài khoản chuyển sang trạng thái người dùng (User) và được mở khóa chuỗi chức năng cá nhân hóa nâng cao nhằm phục vụ mục tiêu quản lý và tối ưu hóa hiệu suất làm việc. Tính năng hạt nhân của phân hệ này là

Quản lý công việc (Tasks), cho phép người dùng thêm mới, chỉnh sửa, cập nhật trạng thái và Xóa các bản ghi công việc. Tính năng này kết hợp chặt chẽ với chức năng chia nhỏ nhiệm vụ phụ (Subtasks) và đặc biệt là chức năng thiết lập ràng buộc phụ thuộc (Dependencies). Bằng việc áp dụng thuật toán duyệt đồ thị ngầm (Graph Traversal), hệ thống tự động kiểm tra và ngăn chặn các lỗi khóa chéo phụ thuộc, đảm bảo trình tự thực hiện công việc luôn logic và thông suốt. Quá trình làm việc còn được hỗ trợ lưu trữ bởi chức năng tương tác, cho phép người dùng viết bình luận và tải lên tệp đính kèm ngay bên trong thẻ công việc.

Bên cạnh đó, nhằm tăng cường mức độ tự động hóa, người dùng được cấp quyền sử dụng chức năng trò chuyện với trợ lý ảo AI (Gemini). Khác với các hệ thống thông thường, trợ lý AI tại đây có khả năng nhận gợi ý cá nhân hóa để đưa ra tư vấn về cách giải quyết một nhiệm vụ khó hoặc cách phân bổ thời gian hợp lý. Quá trình làm việc của người dùng sẽ được tự động tổng hợp thông qua chức năng xem dashboard thống kê cá nhân và được sao lưu an toàn bằng chức năng quản lý thùng rác (nhằm khôi phục dữ liệu lỡ xóa).

Cuối cùng, quản trị viên (Admin) được phân quyền tiếp cận hệ thống bảng điều khiển toàn diện để quản lý tài nguyên, kiểm duyệt thành viên và giám sát nền tảng ứng dụng. Chức năng đầu tiên là xem bảng điều khiển thống kê hệ thống (System Dashboard), tự động tổng hợp dữ liệu để kết xuất các biểu đồ trực quan về lượng thành viên đăng ký mới và tổng khối lượng công việc đang lưu trữ, phục vụ việc đưa ra định hướng mở rộng máy chủ kịp thời. Đối với việc quản lý nhân sự, quản trị viên sử dụng chức năng quản lý người dùng để rà soát danh sách tài khoản, thực hiện phân quyền vai trò (cấp quyền Admin) hoặc khóa/xóa người dùng đối với các trường hợp vi phạm chính sách của nền tảng. Mọi luồng xử lý của quản trị viên đều hướng đến mục tiêu tối thượng là duy trì tính toàn vẹn dữ liệu, kiểm soát lưu lượng truy cập và đảm bảo không gian làm việc ổn định, an toàn cho toàn bộ hệ thống.

3.5. Cơ sở dữ liệu

Hệ thống sử dụng hệ quản trị cơ sở dữ liệu quan hệ mysql với 7 bảng dữ liệu (Tables) cốt lõi bao gồm:

users (Quản lý thông tin tài khoản, mật khẩu mã hóa và phân quyền hệ thống).

tasks (Lưu trữ dữ liệu công việc chính bao gồm tiêu đề, mức độ ưu tiên, trạng thái và thời hạn). subtasks (Quản lý các nhiệm vụ con được phân rã chi tiết từ công việc chính

để dễ dàng theo dõi). task_comments (Ghi nhận các bình luận và lịch sử trao đổi thảo luận nội bộ

bên trong từng công việc).

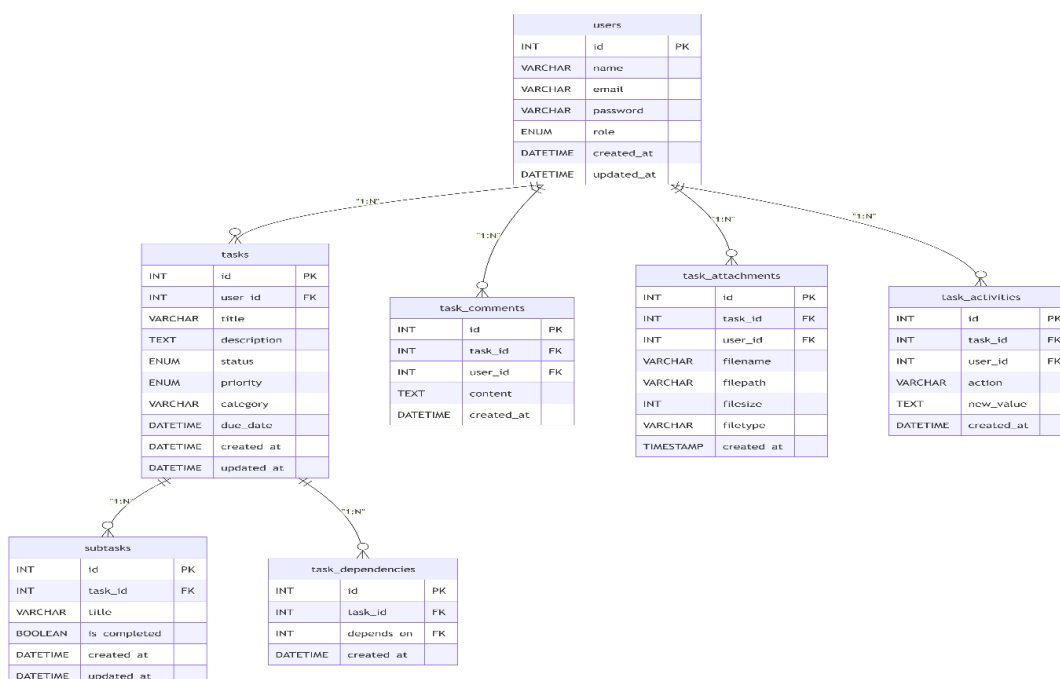
task_attachments (Lưu trữ thông tin cấu trúc và đường dẫn tham chiếu của các tệp tài liệu đính kèm).

task_dependencies (Quản lý các mối ràng buộc phụ thuộc nghiêm ngặt giữa các công việc nhằm chống khóa chéo). task_activities (Ghi nhật ký hệ thống về mọi

hành vi tương tác và thay đổi

tiến độ để phục vụ việc kiểm toán).

3.5.1 Mô hình ERD



Hình 3.7: Mô hình ERD

3.5.2 Bảng user (Người dùng)

Tên thực thể: users (Người dùng)

Mô tả: Lưu trữ thông tin tài khoản, thông tin cá nhân và vai trò phân quyền của người dùng trên hệ thống TaskFlow.

Bảng 3.2: Trường dữ liệu tập hợp người dùng (User)

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
id	INT (PK)	Định danh duy nhất, khóa chính tự tăng của người dùng trong hệ thống.
name	VARCHAR	Họ tên đầy đủ, dùng để hiển thị và nhận diện tài khoản trên giao diện.
email	VARCHAR	Địa chỉ thư điện tử duy nhất, dùng làm tài khoản đăng nhập và xác thực.
password	VARCHAR	Mật khẩu đã qua mã hóa bảo mật bcrypt, tuyệt đối không trả về phía máy khách.
role	ENUM	Quyền hạn tài khoản, nhận giá trị quy định là 'user' hoặc 'admin'.
created_at	DATETIME	Mốc thời gian khởi tạo tài khoản thành viên, tự động sinh bởi cơ sở dữ liệu.
updated_at	DATETIME	Mốc thời gian cập nhật thông tin tài khoản gần nhất, tự động cập nhật.

Mối kết hợp:

- Quan hệ 1-N với tasks: Một người dùng sở hữu nhiều công việc.
- Quan hệ 1-N với task_comments: Một người dùng viết nhiều bình luận.
- Quan hệ 1-N với task_attachments: Một người dùng tải lên nhiều tệp đính kèm.

- Quan hệ 1-N với task_activities: Một người dùng phát sinh nhiều nhật ký hoạt động.

3.5.3 Bảng công việc chính

Tên thực thể: tasks (Công việc)

Mô tả: Là bảng cốt lõi quản lý toàn bộ kho dữ liệu công việc. Cung cấp dữ liệu để vẽ bảng theo dõi tiến độ, tính toán cảnh báo trễ hạn, làm nguồn ngữ cảnh cho trợ lý AI phân tích và tư vấn.

Bảng 3.3: Chi tiết các trường dữ liệu bảng công việc

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
id	INT (PK)	Định danh duy nhất, khóa chính tự tăng của bản ghi công việc.
user_id	INT (FK)	Khóa ngoại liên kết đến bảng users để xác định chủ sở hữu công việc.
title	VARCHAR	Tiêu đề tên gọi đại diện của nhiệm vụ, trường bắt buộc nhập.
description	TEXT	Nội dung văn bản mô tả thông tin chi tiết về yêu cầu của công việc.
status	ENUM	Trạng thái tiến độ hiện tại (todo, in_progress, done, overdue).
priority	ENUM	Mức độ ưu tiên của công việc (low, medium, high).
category	VARCHAR	Phân loại danh mục công việc (Học tập, cá nhân, văn phòng...).
due_date	DATETIME	Mốc thời gian giới hạn cuối cùng (Deadline) phải hoàn thành công việc.
created_at	DATETIME	Mốc thời gian khởi tạo bản ghi dữ liệu, tự động sinh.

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
updated_at	DATETIME	Mốc thời gian làm mới thông tin tiến độ gần nhất, tự động cập nhật.

Mối kết hợp:

- Quan hệ N-1 với users: Nhiều công việc thuộc về một người dùng.
- Quan hệ 1-N với subtasks: Một công việc có nhiều nhiệm vụ con.
- Quan hệ 1-N với task_comments: Một công việc có nhiều bình luận.
- Quan hệ 1-N với task_attachments: Một công việc có nhiều tệp đính kèm.
- Quan hệ 1-N với task_dependencies: Một công việc có nhiều ràng buộc phụ thuộc.
- Quan hệ 1-N với task_activities: Một công việc có nhiều nhật ký hoạt động.

3.5.4 Bảng subtasks (Nhiệm vụ con)

Tên thực thể: subtasks (Nhiệm vụ con)

Mô tả: Lưu trữ các bước xử lý nhỏ được phân rã từ công việc gốc, hỗ trợ người dùng theo dõi tiến độ chi tiết theo từng bước.

Bảng 3.4: Chi tiết các trường dữ liệu bảng nhiệm vụ con

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
id	INT (PK)	Định danh duy nhất, khóa chính tự tăng của nhiệm vụ con.
task_id	INT (FK)	Khóa ngoại liên kết trực tiếp đến bảng tasks (công việc gốc).
title	VARCHAR	Tiêu đề mô tả bước thực hiện hoặc nhiệm vụ nhỏ cần xử lý.
is_completed	BOOLEAN	Cờ trạng thái đánh dấu nhiệm vụ con đã hoàn thành (True/False).

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
created_at	DATETIME	Mốc thời gian khởi tạo nhiệm vụ con, tự động sinh.
updated_at	DATETIME	Mốc thời gian đánh dấu hoàn thành gần nhất, tự động sinh.

Mối kết hợp:

- Quan hệ N-1 với tasks: Nhiều nhiệm vụ con thuộc về một công việc.

3.5.5 Bảng task_comments (Bình luận)

Tên thực thể: task_comments (Bình luận)

Mô tả: Lưu trữ các đoạn hội thoại, trao đổi, ghi chú cá nhân gắn liền trực tiếp với từng công việc.

Bảng 3.5: Chi tiết các trường dữ liệu bảng bình luận

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
id	INT (PK)	Định danh duy nhất, khóa chính tự tăng của bài bình luận.
task_id	INT (FK)	Khóa ngoại liên kết trực tiếp đến công việc đang được thảo luận.
user_id	INT (FK)	Khóa ngoại định danh người dùng viết bình luận.
content	TEXT	Nội dung văn bản ghi chú hoặc trao đổi chi tiết về tiến độ.
created_at	DATETIME	Mốc thời gian bình luận được đăng tải thành công, tự động sinh.

Mối kết hợp:

- Quan hệ N-1 với tasks: Nhiều bình luận gắn với một công việc.
- Quan hệ N-1 với users: Nhiều bình luận do một người dùng viết.

3.5.6 Bảng task_attachment (Tập đính kèm công việc)

Tên thực thể: task_attachments (Tập đính kèm)

Mô tả: Quản lý tài nguyên số, lưu trữ hình ảnh minh họa, tài liệu học tập hoặc báo cáo liên quan trực tiếp tới từng đầu việc.

Bảng 3.6: Chi tiết các trường dữ liệu bảng teepk đính kèm

STT	Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
1	id (PK)	INT	Khóa chính tự tăng, định danh duy nhất của mỗi tập đính kèm trong hệ thống.
2	task_id (FK)	INT	Khóa ngoại tham chiếu đến trường id của bảng tasks, xác định công việc chứa tập đính kèm.
3	user_id (FK)	INT	Khóa ngoại tham chiếu đến bảng users, xác định người dùng đã tải tập lên hệ thống.
4	filename	VARCHAR(255)	Lưu tên gốc của tập do người dùng tải lên (ví dụ: tailieu_on_tap.pdf).
5	filepath	VARCHAR(500)	Lưu đường dẫn vật lý của tập trên máy chủ (ví dụ: uploads/attachments/file_123.pdf).
6	filesize	INT	Lưu kích thước thực tế của tập đính kèm, đơn vị tính là Byte.

STT	Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
7	filetype	VARCHAR(100)	Lưu định dạng (MIME Type) của tệp, ví dụ: image/png, application/pdf, application/zip.
8	created_at	TIMESTAMP	Lưu thời điểm tệp được tải lên hệ thống; giá trị được tạo tự động khi bản ghi được thêm mới.

Mối kết hợp:

- Quan hệ N-1 với tasks: Nhiều tệp gắn với một công việc (xóa task tự động xóa file qua ON DELETE CASCADE).
- Quan hệ N-1 với users: Nhiều tệp do một người dùng tải lên.

3.5.7 Bảng task_dependencies (Ràng buộc phụ thuộc)

Tên thực thể: task_dependencies (Ràng buộc phụ thuộc)

Mô tả: Bảng trung gian thiết lập mối quan hệ phụ thuộc giữa hai công việc trong hệ thống.

Bảng 3.7: Chi tiết các trường dữ liệu bảng ràng buộc phụ thuộc

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
id	INT (PK)	Định danh duy nhất, khóa chính của bản ghi thiết lập ràng buộc.
task_id	INT (FK)	Khóa ngoại chỉ định mã công việc B (công việc bị khóa).

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
depends_on	INT (FK)	Khóa ngoại chỉ định mã công việc A (công việc phải hoàn thành trước).
created_at	DATETIME	Mốc thời gian thiết lập mối quan hệ phụ thuộc.

Mối kết hợp:

- Quan hệ N-1 với tasks (qua task_id): Nhiều ràng buộc 指向 đến một công việc.
- Quan hệ N-1 với tasks (qua depends_on): Nhiều ràng buộc chỉ định một công việc phải hoàn thành trước.

3.5.8 Bảng task_activities (Nhật ký hệ thống)

Tên thực thể: task_activities (Nhật ký hệ thống)

Mô tả: Hệ thống giám sát ngầm (Log System), lưu vết toàn bộ dấu chân tương tác từ lúc công việc được tạo ra cho đến khi hoàn tất.

Bảng 3.8: Chi tiết các trường dữ liệu bảng nhật ký hệ thống

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
id	INT (PK)	Định danh duy nhất, khóa chính của bảng ghi nhật ký.
task_id	INT (FK)	Khóa ngoại tham chiếu đến công việc vừa phát sinh thay đổi.
action	VARCHAR	Đoạn văn bản tự động sinh mô tả hành động (vd: "Đã chuyển sang In Progress").

Tên trường	Kiểu dữ liệu	Mô tả chi tiết chức năng
created_at	DATETIME	Mốc thời gian chính xác hệ thống ghi nhận sự thay đổi tiến độ.

Mối kết hợp:

- Quan hệ N-1 với tasks: Nhiều nhật ký gắn với một công việc.
- Quan hệ N-1 với users: Nhiều nhật ký do một người dùng phát sinh.

CHƯƠNG 4. KẾT QUẢ NGHIÊN CỨU

Sau khi đã hoàn thiện các bản thiết kế chi tiết tại Chương 3, Chương 4 trình bày quá trình hiện thực hóa hệ thống TVU TaskFlow (Hệ thống hỗ trợ quản lý học tập và nhiệm vụ cá nhân cho sinh viên). Nội dung chương này bao gồm việc thiết lập môi trường phát triển thực tế, triển khai mã nguồn cho phân hệ giao diện frontend và phân hệ máy chủ backend, tích hợp các mô hình trí tuệ nhân tạo thông qua gemini API SDK và cuối cùng là đánh giá chất lượng hệ thống thông qua các kịch bản kiểm thử đơn vị, kiểm thử tích hợp, chức năng, hiệu năng và bảo mật.

4.1. Môi trường phát triển

Để đảm bảo tính nhất quán, hiệu năng cao và khả năng mở rộng của hệ thống, việc lựa chọn công nghệ và cấu hình môi trường đóng vai trò quyết định. Đề tài lựa chọn kiến trúc single page application (SPA) với frontend chạy trên vue 3 và backend chạy trên node.js/express kết hợp cơ sở dữ liệu quan hệ mysql.

4.1.1 Công cụ phát triển phần mềm

Môi trường lập trình (IDE và Editor) Hệ thống được phát triển trên các môi trường lập trình tối ưu sau để quản lý mã nguồn hiệu quả:

Bảng 4.1: Các công cụ dùng cho lập trình

Công cụ	Phiên bản	Vai trò và Mục đích
Visual Studio Code	1.85+	Môi trường lập trình chính (IDE) phát triển cả frontend (Vue 3) và backend (Node.js/Express). Tận dụng hệ sinh thái extension phong phú giúp tăng tốc độ viết code.
MySQL Workbench / phpMyAdmin	8.0 / 5.2	Quản trị cơ sở dữ liệu quan hệ mysql, thiết kế bảng dữ liệu, thực thi và kiểm tra các truy vấn sql.

Công cụ	Phiên bản	Vai trò và Mục đích
Postman	10.x	Thiết kế, phát triển và kiểm thử tự động các API endpoint của backend độc lập trước khi tích hợp vào frontend. [29]
Git & GitHub	2.43+	Quản lý phiên bản mã nguồn phân tán, thực hiện các thao tác branching, commit, và push/pull để theo dõi lịch sử phát triển dự án. [27] [28]

Các tiện ích mở rộng hỗ trợ (VS Code Extensions) Việc tích hợp các tiện ích giúp chuẩn hóa định dạng mã nguồn và giảm thiểu lỗi cú pháp:

Volar (Vue Official): Hỗ trợ cú pháp vue 3, gợi ý code thông minh cho single file component (SFC) sử dụng composition API và <script setup>.

CSS IntelliSense (auto-complete): Hỗ trợ tự động hoàn thành (auto-complete) các lớp tiện ích giúp thiết kế giao diện nhanh và chính xác.

Prettier - Code formatter: Tự động định dạng mã nguồn (HTML, CSS, JS, Vue) theo các tiêu chuẩn viết code thống nhất (styleguide).

ESLint: Kiểm tra tĩnh mã nguồn để phát hiện sớm các lỗi logic, cú pháp và các biến chưa được định nghĩa.

4.1.2 Cấu hình môi trường hệ thống

Hệ thống yêu cầu một cấu hình phần cứng và phần mềm đủ tốt để có thể vận hành đồng thời môi trường phát triển frontend (Vite), backend (Node.js/Express Server) và cơ sở dữ liệu MySQL.

Yêu cầu về tài nguyên hệ thống (System Requirements)

Bảng 4.2: Cấu hình môi trường hệ thống

Thành phần	Yêu cầu tối thiểu	Khấu hình khuyến nghị
Hệ điều hành	Windows 10 (64-bit)	Windows 11 hoặc macOS
Bộ nhớ RAM	8 GB	16 GB (Để chạy mượt các dịch vụ và IDE đồng thời)
Bộ vi xử lý	4 Cores (Intel Core i5 / AMD Ryzen 5)	8 Cores (Intel Core i7 / AMD Ryzen 7 hoặc Apple Silicon)
Lưu trữ	10 GB trống	20 GB SSD (Ưu tiên tốc độ đọc ghi dữ liệu)

4.1.3 Cài đặt các nền tảng thực thi (Runtimes)

Sử dụng các phiên bản ổn định lâu dài (LTS) để đảm bảo tính tương thích và hiệu năng của hệ thống:

Node.js v18.x hoặc v20.x (LTS): Môi trường thực thi Javascript ở phía máy chủ (Backend) và cung cấp công cụ build nhanh Vite ở phía frontend. [\[10\]](#)

MySQL 8.0: Hệ quản trị cơ sở dữ liệu quan hệ chính của hệ thống để lưu trữ thông tin người dùng, công việc, thời khóa biểu, điểm số.

4.1.4 Cấu hình biến môi trường và bảo mật

Hệ thống tuân thủ nguyên tắc không lưu trữ cứng thông tin nhạy cảm trong mã nguồn. Mọi thông tin cấu hình cổng dịch vụ, tài khoản CSDL, JWT secret key, và khóa API trí tuệ nhân tạo đều được quản lý thông qua tệp biến môi trường .env ở backend:

```
PORT=3001
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
```

```
DB_NAME=taskflow
DB_PORT=3306
JWT_SECRET=your_jwt_super_secret_key_here
JWT_EXPIRES_IN=7d
GEMINI_API_KEY=AIzaSy...your_gemini_api_key...
```

Việc thiết lập biến môi trường bài bản này giúp hệ thống dễ dàng được chuyển đổi cấu hình giữa môi trường phát triển (Local Development) và môi trường triển khai thực tế (Production Cloud / Docker Container) mà không cần can thiệp chỉnh sửa mã nguồn.

4.2. Triển khai phân hệ Frontend (Vue 3 Application)

Phân hệ frontend được xây dựng dựa trên kiến trúc single page application (SPA) hiện đại của vue 3 với công cụ đóng gói vite [\[9\]](#). Phân hệ này tập trung vào tính mô-đun hóa cao, khả năng phản hồi tức thời (reactivity) và trải nghiệm người dùng mượt mà theo phong cách sang trọng luxury obsidian & gold glassmorphism.

4.2.1 Tổ chức cấu trúc dự án

Mã nguồn frontend được tổ chức theo cấu trúc dạng lớp (Layered Directory Structure) giúp phân tách rõ ràng giữa giao diện, logic xử lý trạng thái và các API dịch vụ:

src/assets/: Chứa các tài nguyên tĩnh, đặc biệt là file main.css định nghĩa toàn bộ hệ thống màu sắc chủ đạo (Obsidian & Gold), kiểu chữ (typography), các hiệu ứng kính mờ (glassmorphism), chuyển cảnh (transitions) và hỗ trợ chế độ Light Mode/Dark Mode.

src/api/: Định nghĩa lớp dịch vụ axios để giao tiếp với backend. File tiêu biểu: taskApi.js cấu hình base URL, đính kèm token xác thực và định nghĩa các API gọi backend.

src/components/: Chứa các thành phần giao diện nhỏ có tính tái sử dụng cao
src/views/: Chứa các thành phần đại diện cho toàn bộ trang của ứng dụng

src/stores/: Quản lý trạng thái toàn cục của ứng dụng bằng Pinia. Bao gồm authStore.js (quản lý phiên đăng nhập của sinh viên) và taskStore.js (quản lý danh sách nhiệm vụ, bộ lọc và các thông tin đồng bộ).

src/router/: Định nghĩa định tuyến trong index.js, cấu hình các Navigation Guards để bảo vệ các trang yêu cầu đăng nhập (yêu cầu JWT token).

4.2.2 Triển khai các tính năng kỹ thuật trọng tâm

a) Quản lý trạng thái xác thực (Authentication Store)

Sử dụng Pinia (useAuthStore) kết hợp với localStorage để duy trì trạng thái đăng nhập của sinh viên. Khi đăng nhập thành công, thông tin sinh viên và JWT Token sẽ được lưu vào localStorage. Khi khởi tạo ứng dụng, store tự động tải thông tin này lên để khôi phục phiên làm việc mà không cần sinh viên phải đăng nhập lại (kể cả khi tải lại trang F5).

b) Lớp dịch vụ API với axios interceptors

Mọi yêu cầu HTTP gửi đến backend (Port 3001) đều đi qua một instance Axios duy nhất. Hệ thống sử dụng Request Interceptor để tự động kiểm tra sự tồn tại của JWT Token trong localStorage và đính kèm nó vào Header của yêu cầu dưới định dạng Authorization: Bearer <token>. Đồng thời, response interceptor được cấu hình để bắt tập trung lỗi 401 unauthorized (khi token hết hạn hoặc không hợp lệ) để tự động xóa token và chuyển hướng sinh viên về trang đăng nhập (/login).

c) Giao diệnkanban board kéo thả linh hoạt

Giao diện Kanban (Kanban.vue) hiển thị các công việc theo 3 cột trạng thái: Chưa làm (Todo), Đang làm (In Progress), và Hoàn thành (Done). Tính năng kéo thả được lập trình bằng HTML5 Drag and Drop API thông qua các sự kiện @dragstart trên thẻ công việc và @dragover.prevent, @drop trên các cột trạng thái. Khi thả một thẻ công việc vào cột mới, store sẽ kích hoạt hàm updateTask để gửi yêu cầu bất đồng bộ (Axios) cập nhật ngay trạng thái công việc dưới cơ sở dữ liệu MySQL mà không cần tải lại trang.

d) Kế hoạch ngày (Timebox Planner) và khóa khung giờ học trên lớp

Module timebox planner (TimeboxPlanner.vue) cho phép sinh viên lên lịch chi tiết công việc trong ngày theo từng khung giờ (từ 0h đến 23h).

Trực quan hóa Thời khóa biểu: Hệ thống tự động truy vấn lịch học cố định trên lớp của sinh viên (từ module thời khóa biểu). Các khung giờ có lịch học sẽ tự động được hiển thị dưới dạng một khối thông báo màu đỏ: lớp học

Khóa khung giờ học: Để tránh việc sinh viên lập lịch tự học trùng với giờ học chính khóa trên lớp, hệ thống sẽ kiểm tra trạng thái khung giờ bằng hàm getClassForSlot(hour). Nếu khung giờ đó trùng lịch học chính khóa, sự kiện @drop sẽ bị vô hiệu hóa, ngăn chặn việc kéo thả công việc vào khung giờ đó.

4.2.3 Thiết kế đáp ứng và tối ưu hóa giao diện (Responsive Design)

Để đảm bảo hệ thống hoạt động tốt trên cả máy tính và thiết bị di động (Mobile-First Approach), dự án tận dụng triệt để sức mạnh của CSS thuần/CSS Variables kết hợp với CSS Variables linh hoạt:

Hệ thống lưới (CSS Grid & Flexbox): Tự động co giãn theo chiều rộng màn hình. Trên màn hình máy tính hiển thị bố cục 2 cột (ví dụ: backlog bên trái, timeline bên phải trên trang Timebox), sang màn hình di động sẽ tự động xếp chồng thành 1 cột dọc giúp dễ tương tác bằng thao tác vuốt chạm.

Sidebar thích ứng: Trên màn hình nhỏ (Mobile), Sidebar được ẩn hoàn toàn để tối ưu diện tích hiển thị và chỉ xuất hiện dưới dạng menu trượt ra (slide-out drawer) khi người dùng bấm vào biểu tượng Hamburger Menu.

Chế độ sáng/tối (Theme Toggle): Toàn bộ giao diện sử dụng các biến CSS (--bg-primary, --text-primary,...) được định nghĩa lại ở lớp .light-theme. Việc chuyển đổi giao diện sáng/tối diễn ra tức thì thông qua việc thêm/xóa class light-theme ở thẻ <html> gốc.

4.3. Triển khai phân hệ Backend Node.js & Express (Core & AI Service)

Phân hệ backend đóng vai trò là hạt nhân điều phối tri thức, xử lý nghiệp vụ, giao tiếp cơ sở dữ liệu MySQL và tích hợp trí tuệ nhân tạo. Backend được xây dựng

bằng node.js và framework express gọn nhẹ, hiệu năng cao, sử dụng mô hình lập trình bất đồng bộ (async/await) để tránh nghẽn luồng xử lý (Non-blocking I/O).

4.3.1 Cấu trúc tổ chức dự án

Mã nguồn Backend được tổ chức theo kiến trúc phân tầng dạng MVC rút gọn, giúp tách biệt rõ ràng giữa các luồng dữ liệu và logic nghiệp vụ:

server.js: Điểm khởi đầu của ứng dụng backend. Cấu hình middleware (CORS, Express JSON parser, static upload folders), ánh xạ các cụm định tuyến API chính và khởi chạy máy chủ tại cổng 3001.

src/config/: Thiết lập cấu hình hệ thống. File database.js khởi tạo kết nối MySQL connection Pool (mysql2/promise) để tối ưu số lượng kết nối đồng thời và chứa kịch bản tự động kiểm tra và nâng cấp cấu trúc bảng (Auto-migrations) khi khởi chạy server.

src/models/: Định nghĩa các lớp mô hình dữ liệu (User, task, grade, classSchedule, project, milestone). Các lớp này chứa các phương thức tĩnh viết sql trực tiếp để tương tác với mysql.

src/controllers/: Chứa các controller xử lý logic nghiệp vụ chi tiết cho từng đối tượng (authController, taskController, gradeController, classScheduleController, aiController, attachmentController).

src/routes/: Định nghĩa các Endpoint REST API và ánh xạ chúng tới các phương thức xử lý tương ứng trong Controller.

src/middleware/: Chứa các bộ lọc kiểm soát yêu cầu đầu vào. Ví dụ: auth.js là middleware xác thực JWT Token của người dùng trước khi cho phép đi vào các API nghiệp vụ.

4.3.2 Triển khai phân hệ AI Service (Tích hợp trong Backend Express)

Hệ thống TaskFlow loại bỏ kiến trúc đa máy chủ phức tạp (như Spring Boot phối hợp FastAPI) mà tích hợp trực tiếp lớp dịch vụ trí tuệ nhân tạo (AI Service) vào

backend Node.js/Express. Giải pháp này giúp tối ưu tài nguyên máy chủ, giảm độ trễ giao tiếp liên mạng và đơn giản hóa quy trình đóng gói ứng dụng.

4.3.3 Tổ chức API và gọi Google Generative AI (Gemini SDK)

Backend sử dụng thư viện SDK chính thức `@google/generative-ai` [18] để kết nối trực tiếp với API của Google Gemini. Trong `aiController.js`, hệ thống khởi tạo đối tượng kết nối `GoogleGenerativeAI` với khóa API `GEMINI_API_KEY`. Để tối ưu hóa chi phí và hiệu năng phản hồi, hệ thống ưu tiên sử dụng mô hình thế hệ mới `gemini-2.0-flash` và `gemini-1.5-flash` (các mô hình có tốc độ xử lý cực nhanh và hiểu ngữ cảnh tiếng Việt rất tốt). Mọi yêu cầu gọi API bên ngoài đều được thực hiện thông qua cơ chế bất đồng bộ (`async/await`) để không gây nghẽn tiến trình chạy chính của máy chủ Node.js.

4.3.4 Các chức năng thông minh và Cơ chế dự phòng (Fallbacks)

Để đảm bảo hệ thống hoạt động ổn định 100% ngay cả khi API Key của Gemini bị cạn hạn định mức (Rate Limit) hoặc mất kết nối mạng Internet, toàn bộ các tính năng AI đều được triển khai song song cơ chế Dự phòng cục bộ bằng mã nguồn (Local Rule-based Fallbacks):

a) Tự động phân tích câu lệnh tiếng Việt thành Task cấu trúc (`/api/ai/quick-task`)

Luồng AI chính: Khi sinh viên nhập một câu nói tự nhiên (Ví dụ: “lập trình giao diện web vào chiều mai lúc 14h ưu tiên cao”), hệ thống gửi câu lệnh kèm Prompt yêu cầu Gemini phân tích ngữ nghĩa và trả về duy nhất một chuỗi định dạng JSON gồm các trường: `title`, `description`, `priority`, `category`, `due_date`. Dữ liệu JSON sau đó được parse và tự động ghi vào database.

Cơ chế dự phòng cục bộ (`localParseTask`): Nếu cuộc gọi API Gemini thất bại, hệ thống tự động kích hoạt hàm phân tích cục bộ. Hàm này sử dụng các biểu thức chính quy (Regular Expressions) để tìm kiếm từ khóa:

Độ ưu tiên: Tìm các từ khóa “gấp”, “quan trọng”, “cao” để gán `high`, từ khóa “thấp”, “rảnh” để gán `low`.

Danh mục: Quét các từ “học”, “bài tập”, “khóa luận” để phân loại vào danh mục “Học tập”; các từ “mua”, “ăn”, “cá nhân” gán vào danh mục “Cá nhân”.

Thời gian: Nhận diện từ “mai” (ngày hiện tại + 1), “mốt” (ngày hiện tại + 2) và giờ học dạng “14h”, “8h tối” để tính toán ra chuỗi thời gian ISO thích hợp.

Tiêu đề sạch: Loại bỏ các cụm từ chỉ thời gian và độ ưu tiên ra khỏi câu lệnh ban đầu để lọc ra tiêu đề sạch nhất cho công việc.

b) Phân rã công việc lớn thành các subtasks (/api/ai/decompose)

Luồng AI chính: Sinh viên bấm nút “AI Tách Việc” trên một công việc lớn. Hệ thống sẽ gửi tiêu đề và mô tả công việc sang Gemini để phân tách thành 4 - 6 đầu việc con chi tiết dưới dạng một mảng JSON. Các công việc con này sẽ được tự động lưu vào bảng subtasks của cơ sở dữ liệu.

Cơ chế dự phòng cục bộ: Khi không có kết nối API AI, hệ thống rà soát từ khóa trong tiêu đề công việc lớn:

Nếu chứa từ khóa liên quan đến viết báo cáo/khóa luận, hệ thống trả về bộ khung: Xây dựng đề cương - Thu thập tài liệu - Viết bản thảo lý thuyết - Viết chương phân tích - Căn chỉnh định dạng A4 - Chuẩn bị slide bảo vệ.

Nếu chứa từ khóa liên quan đến lập trình/phần mềm: Thiết kế ERD -> Thiết lập thư mục và Git - Viết API backend - Thiết kế UI frontend - Kết nối API và kiểm thử - Viết hướng dẫn chạy.

Nếu chứa từ khóa liên quan đến ôn thi: Tập hợp slide - Tóm tắt lý thuyết - Giải đề thi năm trước - Học nhóm - Ôn luyện kiến thức yếu.

c) Trợ lý Cố vấn hiệu suất AI Coach cá nhân hóa (/api/ai/coach)

Đây là tính năng độc đáo giúp sinh viên tương tác trực tiếp với một chuyên gia năng suất thông qua giao diện chat.

Tích hợp dữ liệu thời gian thực (Context-Aware Chat): Trước khi gửi tin nhắn của sinh viên sang gemini, controller sẽ tự động thực hiện các truy vấn sql để đếm tổng số lượng công việc, số việc đang làm, chờ làm, đặc biệt là danh sách chi tiết các

công việc đã bị quá hạn (Overdue) và tỷ lệ hoàn thành công việc hiện tại của sinh viên đó. Các thông tin thực tế này được chèn trực tiếp vào phần prompt hệ thống để gemini nắm rõ trạng thái học tập của sinh viên, từ đó đưa ra lời khuyên cá nhân hóa chính xác

Lệnh tạo công việc tự động từ chatbot: Nếu trong nội dung hội thoại, sinh viên yêu cầu trợ lý tạo một công việc mới gemini sẽ chèn thêm nhãn khối JSON đặc biệt [CREATE_TASK_JSON: {...}] ở cuối phản hồi. Backend sẽ bắt lấy nhãn này, tự động trích xuất dữ liệu bên trong để insert trực tiếp một công việc mới vào MySQL cho sinh viên, sau đó xóa khối JSON đó đi trước khi trả tin nhắn phản hồi về giao diện chat.

Hỗ trợ điều hướng bằng Route Links: AI Coach có thể trả về các liên kết điều hướng có cấu trúc [Tên Nút Bấm](route://TênRoute). Khi hiển thị ở Frontend, các liên kết này sẽ được render thành các nút bấm đẹp mắt, cho phép sinh viên bấm vào là chuyển hướng ngay sang trang Kanban, Lịch trình hoặc Focus Room mà không phải thao tác thủ công.

Cơ chế dự phòng cục bộ: Nếu gemini ngoại tuyến, AI Coach sẽ sử dụng bộ phân tích từ khóa cục bộ để tự động nhận dạng các ý định phổ biến của người dùng về: Tạo task nhanh, Thống kê hiệu suất thực tế, Hỏi về phương pháp sắp xếp ưu tiên (Eisenhower, eat that frog), Hỏi cách giảm áp lực stress khi quá tải bài tập, hoặc Hỏi về kỹ thuật quản lý thời gian (Pomodoro, time blocking) để trả về các cẩm nang hướng dẫn khoa học đã được cấu hình sẵn.

d) Triển khai chatbot dạng tác nhân thông minh với khả năng gọi hàm (Gemini Function Calling - /api/chat)

Để nâng cấp chatbot thành một trợ lý tác nhân (AI Agent) thực thụ, phân hệ chatbot triển khai kỹ thuật gọi hàm (Function Calling) của mô hình gemini thông qua tệp chatcontroller.js.

Định nghĩa bộ công cụ (Tools Declaration): Hệ thống định nghĩa một mảng các đặc tả hàm (Function Declarations) dưới dạng JSON và truyền cho gemini khi khởi tạo cuộc hội thoại:

`get_tasks(status, priority)`: Lấy danh sách nhiệm vụ của sinh viên kèm theo bộ lọc trạng thái và độ ưu tiên. `create_task(title, description, priority, category, due_date)`: Thêm một nhiệm

vụ học tập mới vào MySQL. `update_task_status(task_id, status)`: Cập nhật trạng thái nhiệm vụ (chờ làm, đang làm, hoàn thành).

`delete_task(task_id)`: Xóa một nhiệm vụ học tập (được cấu hình yêu cầu xác nhận trước khi thực hiện).

`get_stats()`: Lấy số liệu thống kê công việc thời gian thực bao gồm tỷ lệ hoàn thành công việc, số nhiệm vụ quá hạn và danh sách việc sắp tới trong 24 giờ.

Vòng lặp thực thi tác nhân (Agentic Loop): Khi sinh viên gửi tin nhắn (Ví dụ: “Đánh dấu hoàn thành giúp tôi công việc ID 15” hoặc “Có những công việc nào quá hạn không?”), Gemini sẽ tự động nhận diện ý định và không trả về văn bản thông thường, mà trả về một yêu cầu gọi hàm `functionCall` cùng các tham số tương ứng. Backend nhận được yêu cầu gọi hàm này, tự động ánh xạ và thực thi hàm tương ứng với cơ sở dữ liệu MySQL (thông qua hàm `executeTool`). Sau khi nhận kết quả truy vấn từ MySQL, backend đóng gói kết quả thành phản hồi hàm `functionResponse` gửi ngược lại cho Gemini. Chu trình này được chạy lặp lại tối đa 5 lần trong một vòng lặp (for (let i = 0; i < 5; i++)) cho đến khi Gemini có đủ thông tin để sinh ra câu trả lời cuối cùng bằng ngôn ngữ tự nhiên gửi về cho người dùng.

Cơ chế chống lỗi Rate Limit: Để tránh lỗi nghẽn hoặc cạn hạn ngạch khi sử dụng mô hình Gemini, hệ thống triển khai vòng lặp thử lại tối đa 3 lần (`maxRetries = 3`) kết hợp thuật toán giãn cách thời gian tăng dần (Exponential Backoff) nếu nhận diện mã trạng thái lỗi 429 (Rate limit).

4.4. Triển khai lưu trữ và quản lý tài nguyên đính kèm

Để thay thế cho hệ thống RAG phức tạp và không thực sự tối ưu cho nhu cầu lưu trữ tài liệu học tập cá nhân của sinh viên, hệ thống triển khai Module quản lý tài nguyên đính kèm trực quan liên kết trực tiếp với từng nhiệm vụ cụ thể.

4.4.1 Xử lý tải tệp tin bằng Multer và lưu trữ trên Server

Hệ thống hỗ trợ sinh viên đính kèm các tệp tin học tập (PDF, word, excel, slide thuyết trình powerpoint, ảnh bài tập hoặc file nén ZIP) trực tiếp từ máy tính lên hệ thống.

Bộ lọc tải tệp (File Upload Middleware): Backend sử dụng thư viện multer[20] để xử lý các yêu cầu đa phần (multipart/form-data). Tệp tin tải lên được cấu hình lưu trữ trong thư mục cục bộ uploads/attachments/ trên đĩa cứng server. Để tránh việc trùng tên tệp tin tải lên, tên file được đổi tự động bằng cách kết hợp thời gian thực (timestamp) và số ngẫu nhiên.

Kiểm soát kích thước tệp: Middleware được cấu hình giới hạn kích thước tối đa của mỗi tệp tải lên là 10MB để tránh tràn dung lượng bộ nhớ máy chủ.

4.4.2 Quản lý liên kết đính kèm của Task

Dữ liệu về tệp đính kèm và các liên kết tài nguyên ngoài (ví dụ: Link thư mục Google Drive của nhóm, Link bài đọc tham khảo trên website) được quản lý tập trung trong bảng task_attachments của MySQL với cấu trúc: id, task_id, filename (Tên hiển thị), filepath (Đường dẫn tương đối trên server hoặc link URL ngoài), filetype (Định dạng file như pdf, docx hoặc url).

Khi sinh viên xóa một nhiệm vụ, controller sẽ kích hoạt cơ chế xóa dây chuyền (Cascade Delete): hệ thống tự động quét tìm tất cả các bản ghi đính kèm của nhiệm vụ đó, thực hiện xóa vật lý tệp tin trên đĩa cứng server bằng hàm fs.unlinkSync() trước khi xóa bản ghi trong CSDL, giúp làm sạch dung lượng đĩa cứng tránh rác hệ thống.

4.5. Tích hợp và giao tiếp hệ thống (System Integration)

Sự kết nối chặt chẽ và nhất quán về mặt dữ liệu giữa giao diện người dùng và máy chủ dịch vụ quyết định sự thành công trong vận hành của hệ thống TaskFlow.

Tích hợp giữa vue 3 frontend và node.js express backend:

Frontend và backend giao tiếp hoàn toàn thông qua kiến trúc RESTful API sử dụng định dạng truyền tải dữ liệu chuẩn JSON.

Cấu hình CORS (Cross-Origin Resource Sharing): Do ở môi trường phát triển, Frontend chạy trên Vite (Port 5173) và Backend chạy trên express (Port 3001), backend cấu hình middleware cors [\[21\]](#) để cho phép các yêu cầu có nguồn gốc từ <http://localhost:5173> được phép truy cập tài nguyên hệ thống và gửi kèm thông tin xác thực (Credentials).

Đồng bộ hóa dữ liệu thời gian thực: Khi sinh viên thực hiện một thao tác trên giao diện (ví dụ: kéo thẻ nhiệm vụ từ đang làm sang hoàn thành), luồng tích hợp diễn ra như sau:

Frontend kích hoạt action trong pinia taskstore.js.

Pinia store gọi hàm trong lớp dịch vụ taskApi.js để gửi yêu cầu PUT `/api/tasks/:id` chứa trạng thái mới.

Backend nhận yêu cầu, đi qua middleware authMiddleware.js để xác thực quyền sở hữu nhiệm vụ của sinh viên.

Backend thực thi câu lệnh sql cập nhật trạng thái trong mysql và ghi nhận lịch sử vào bảng task_activities.

Backend phản hồi kết quả thành công kèm dữ liệu đã cập nhật.

Pinia store nhận phản hồi, cập nhật lại mảng dữ liệu trong bộ nhớ frontend, giao diện vue 3 tự động render lại phần thay đổi tức thì mà không cần tải lại toàn bộ trang web.

4.6. Kiểm thử hệ thống

Kiểm thử là giai đoạn quan trọng nhằm xác minh tính đúng đắn của toàn bộ các tính năng đã phát triển, đảm bảo hệ thống vận hành ổn định, an toàn và đáp ứng các tiêu chí phi chức năng về hiệu năng và bảo mật.

4.6.1 Kiểm thử đơn vị

Kiểm thử đơn vị tập trung vào việc cô lập các hàm logic tính toán lỗi để kiểm tra tính chính xác của các giá trị đầu ra:

Kiểm thử logic tự phân tích cú pháp công việc cục bộ (localParseTask): Chạy thử hàm với các đầu vào chứa các từ khóa phức tạp khác nhau để đảm bảo hàm luôn trích xuất đúng tiêu đề sạch, đúng độ ưu tiên và ngày hạn chót tương ứng.

Kiểm thử thuật toán dự đoán điểm thi cuối kỳ: Truyền các bộ dữ liệu điểm chuyên cần, giữa kỳ và trọng số giả lập để kiểm tra xem hàm có tính toán chính xác điểm thi cuối kỳ cần đạt tối thiểu theo đúng công thức hay không, đồng thời đảm bảo xử lý đúng các trường hợp đặc biệt (ví dụ: điểm yêu cầu ra kết quả âm hoặc lớn hơn 10.0).

4.6.2 Kiểm thử tích hợp

Kiểm thử tích hợp tập trung vào việc xác minh sự tương tác chính xác giữa API Layer, business logic layer, database layer và security layer:

Kiểm thử luồng bảo mật JWT: Giả lập gửi các yêu cầu HTTP tới API lấy danh sách công việc (GET /api/tasks) với 3 kịch bản: Không gửi token, gửi token sai chữ ký, và gửi token đã hết hạn. Xác nhận backend luôn chặn thành công và trả lời chính xác mã lỗi 401 Unauthorized.

Kiểm thử thuật toán phát hiện liên kết vòng: Thực hiện gọi liên tiếp chuỗi API thêm liên kết phụ thuộc: A phụ thuộc B, B phụ thuộc C, và C phụ thuộc A. Xác nhận hệ thống ở bước cuối cùng luôn phát hiện ra chu trình và chặn đứng thao tác, bảo vệ tính toàn vẹn cho sơ đồ phụ thuộc.

4.6.3 Kiểm thử chức năng

Thực hiện kiểm thử hộp đen (Black-box Testing) để kiểm tra toàn bộ các tính năng tương tác thực tế của hệ thống đối chiếu với yêu cầu thiết kế ban đầu:

Bảng 4.3: Kiểm thử chức năng

ID	Chức năng kiểm thử	Kịch bản kiểm thử	Kết quả mong đợi	Trạng thái
TC01	Đăng ký tài khoản	Nhập đầy đủ thông tin hợp lệ, mật khẩu ≥ 6 ký tự).	Tạo tài khoản thành công, mật khẩu được băm bcrypt trong MySQL, trả về JWT Token và tự động đăng nhập.	Đạt
TC02	Đăng nhập hệ thống	Nhập sai định dạng email hoặc sai mật khẩu.	Hệ thống chặn lại, không cho đăng nhập và hiển thị thông báo lỗi thân thiện trên form.	Đạt
TC03	Tạo công việc nhanh bằng AI	Gõ câu lệnh tiếng Việt: “Nộp đề cương khóa luận vào chiều mai lúc 14h gấp”.	Hệ thống tự động khởi tạo công việc mới với tiêu đề sạch, hạn chót đúng ngày mai lúc 14:00, ưu tiên cao.	Đạt

ID	Chức năng kiểm thử	Kịch bản kiểm thử	Kết quả mong đợi	Trạng thái
TC04	Cảnh báo trùng lịch học	Tạo hạn chót công việc trùng vào khung giờ học lớp học cố định đã khai báo trong Thời khóa biểu.	Hiển thị cảnh báo đỏ nổi bật ngay trên form báo trùng lịch học môn tương ứng trước khi người dùng bấm lưu.	Đạt
TC05	Khóa giờ trên Timebox Planner	Kéo thả một nhiệm vụ từ cột Backlog vào khung giờ đang có lịch học trên lớp chính khóa.	Hệ thống chặn thao tác kéo thả (khóa đỏ khung giờ), không cho phép lập lịch tự học trùng giờ lên lớp.	Đạt
TC06	Pomodoro & Tài liệu đi kèm	Bật phiên tập trung Pomodoro cho một nhiệm vụ có đính kèm file PDF bài giảng và liên kết Drive.	Đồng hồ đếm ngược hoạt động mượt mà, widget tài liệu bên phải hiển thị đúng danh sách file/link đính kèm để mở nhanh.	Đạt

ID	Chức năng kiểm thử	Kịch bản kiểm thử	Kết quả mong đợi	Trạng thái
TC07	Dự toán GPA & Đếm ngược	Nhập điểm chuyên cần 9.0, giữa kỳ 6.0, chọn mục tiêu đạt điểm A (≥ 8.5) môn Lập trình Web (trọng số 10-30-60).	Hệ thống hiển thị đếm ngược chính xác số ngày đến lịch thi môn học và dự toán cần thi cuối kỳ đạt $\geq 9.67đ$.	Đạt
TC08	Sơ đồ Dependency Graph	Thêm liên kết phụ thuộc giữa các công việc trên giao diện đồ thị trực quan.	Đồ thị D3/Vis.js tự động vẽ lại các mũi tên liên kết chính xác, hiển thị trực quan các tiến trình công việc liên đới.	Đạt

4.6.4 Kiểm thử hiệu năng

Tốc độ phản hồi (Response Time): Sử dụng các công cụ đo lường mạng của trình duyệt (Browser Developer Tools) để rà soát. Các yêu cầu API nghiệp vụ thông thường (CRUD công việc, thời khóa biểu, điểm số) có tốc độ phản hồi trung bình cực nhanh ($< 80ms$) nhờ cơ chế Connection Pool của MySQL. Các tác vụ gọi AI sang máy chủ Google Gemini có độ trễ dao động từ 1.2s đến 2.5s tùy thuộc vào kích thước prompt và tốc độ sinh nội dung của mô hình, hoàn toàn nằm trong mức chấp nhận được đối với tác vụ trí tuệ nhân tạo. Khi API Gemini gặp sự cố, cơ chế dự phòng cục bộ phản hồi ngay lập tức ($< 15ms$), đảm bảo trải nghiệm không bị gián đoạn.

Kiểm thử chịu tải (Load Testing): Sử dụng apache jmeter giả lập 100 người dùng ảo đồng thời thực hiện các thao tác gửi tin nhắn tới AI coach và cập nhật trạng thái công việc trong thời gian 5 phút. Kết quả kiểm thử cho thấy máy chủ node.js duy trì sự ổn định cao với tỷ lệ lỗi (Error Rate) bằng 0% và tài nguyên CPU/RAM tiêu hao rất thấp nhờ cơ chế xử lý phi tuần tự của node.js.

4.6.5 Kiểm thử bảo mật

Phòng chống SQL Injection: Mọi truy vấn cơ sở dữ liệu trong các lớp models/ của Backend đều sử dụng phương thức `pool.execute()` của thư viện `mysql2/promise` (Prepared Statements). Các tham số do người dùng truyền lên được gửi riêng biệt với câu lệnh sql gốc, loại bỏ hoàn toàn nguy cơ bị khai thác lỗ hổng tiêm câu lệnh sql độc hại.

Bảo mật dữ liệu đường truyền: Mật khẩu của sinh viên được băm bằng thuật toán `bcrypt` [17] một chiều không thể đảo ngược. Các tệp tin tải lên qua `multer` được kiểm duyệt phần mở rộng kỹ càng, không cho phép lưu trữ hoặc thực thi các file mã nguồn độc hại (.exe, .bat, .php, .js) trên thư mục upload của máy chủ.

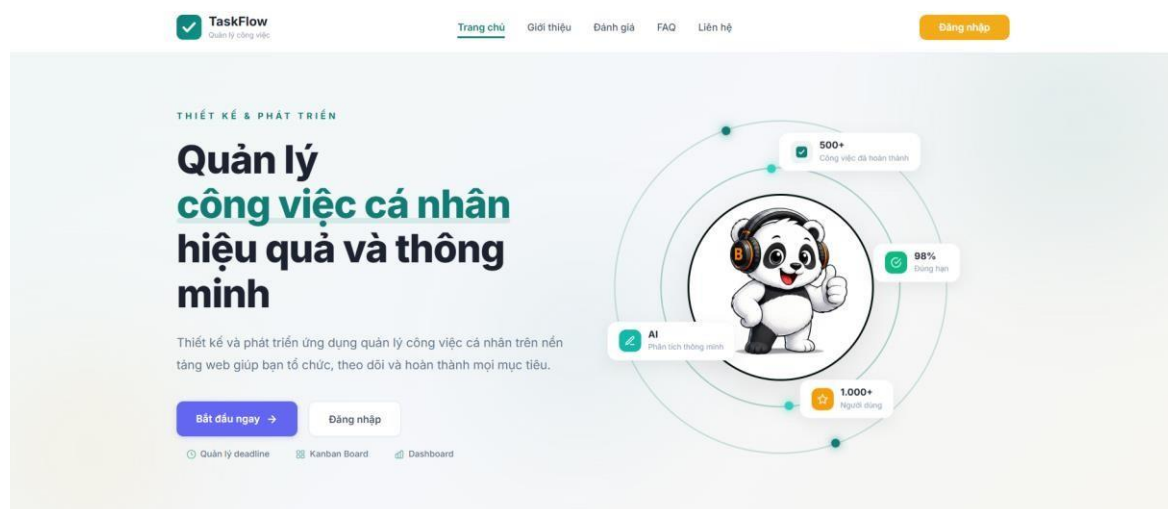
4.7. Kết quả triển khai hệ thống

Sau quá trình thiết kế hệ thống và thực thi mã nguồn, ứng dụng TaskFlow đã được hoàn thiện giao diện và vận hành ổn định trên các môi trường chạy thử nghiệm.

4.7.1 Giao diện trang chủ

Trang dashboard là trang chính của TaskFlow, hiển thị tổng quan toàn bộ hoạt động học tập và công việc của người dùng. Đầu trang có lời chào theo ngày tháng, kèm các nút làm mới và AI Phân tích để cập nhật dữ liệu hoặc nhận gợi ý thông minh. Tiếp theo là thống kê nhanh gồm 4 card mang tính chất clickable hiển thị số lượng công việc cần làm, đang làm, đã xong và quá hạn. Khu vực biểu đồ trình bày phân bố trạng thái bằng thanh ngang và phân loại công việc theo danh mục bằng biểu đồ cột. Phần phân tích hiệu suất mang xu hướng 7 ngày qua và công cụ phân tích tri hoãn với chỉ số nguy cơ, lời khuyên cá nhân hóa và khung giờ vàng hoạt động hiệu quả nhất. Bên dưới là các widget tiện ích như danh sách sắp đến hạn, vòng tròn hiệu

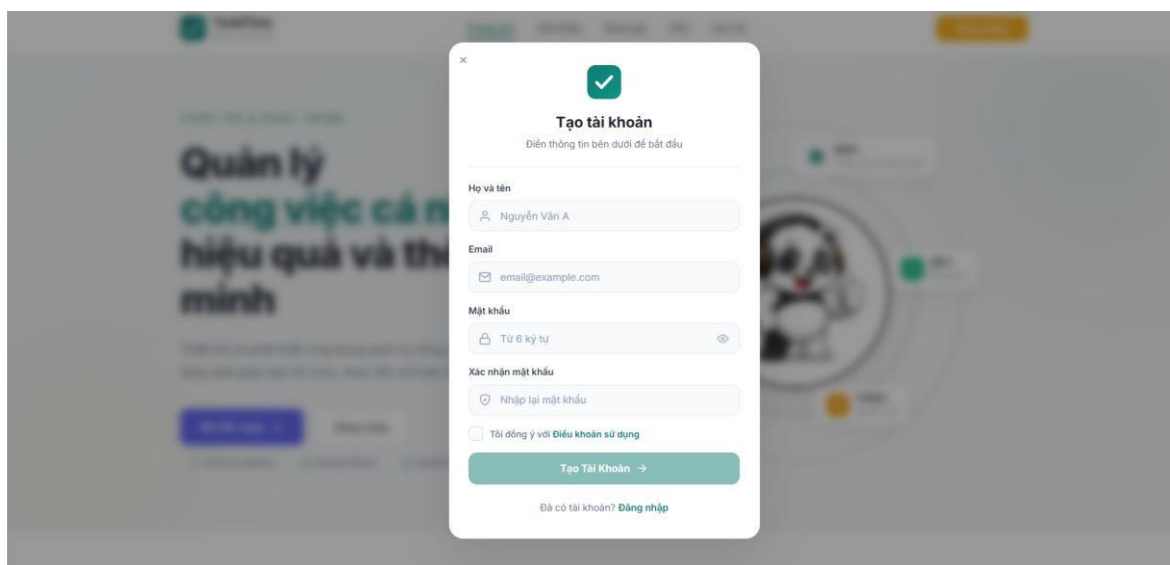
suất hôm nay, tiến độ OKRs và thống kê nhanh tổng hợp. Cuối cùng là hệ sinh thái liên kết cung cấp shortcut đến ghi chú, lịch, thời gian theo dõi và insights thông minh gợi ý cải thiện năng suất.



Hình 4.1: Giao diện trang chủ

4.7.2 Giao diện trang đăng ký

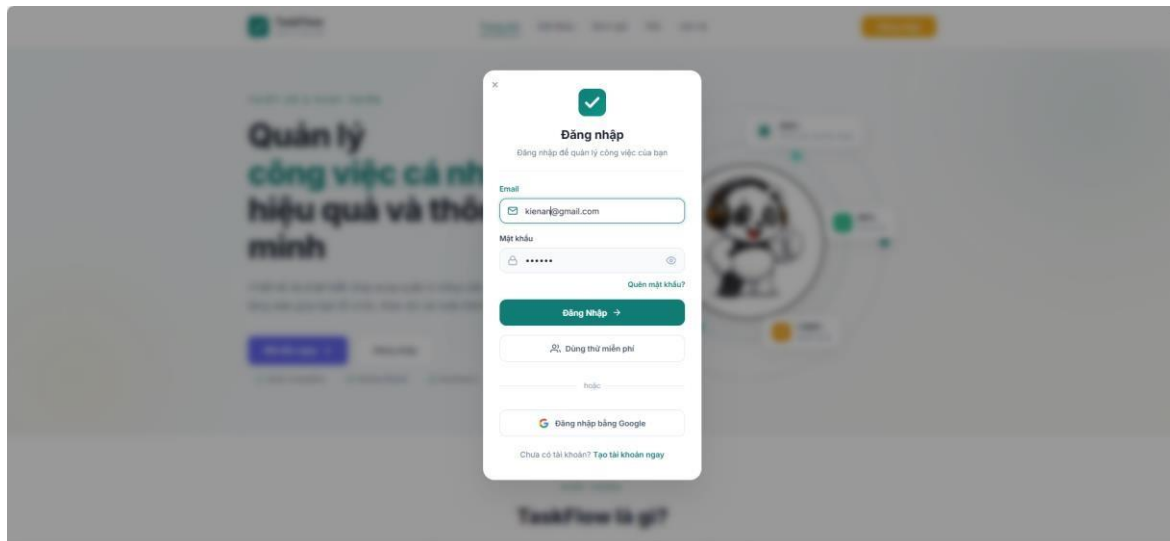
Trang đăng nhập và đăng ký (/register) của TaskFlow được thiết kế với giao diện landing page chuyên nghiệp, trình bày các tính năng nổi bật, đánh giá người dùng và FAQ để giới thiệu sản phẩm. Trang đăng nhập cung cấp form nhập email và mật khẩu với khả năng hiển thị/ẩn mật khẩu, kèm nút “Quên mật khẩu” để gửi yêu cầu đặt lại qua email. Người dùng có thể đăng nhập bằng tài khoản thông thường, dùng thử miễn phí với tài khoản demo, hoặc đăng nhập bằng google oauth. Trang đăng ký yêu cầu họ tên, email, mật khẩu và xác nhận mật khẩu với chỉ số độ mạnh mật khẩu động (4 cấp độ) và bắt buộc đồng ý điều khoản sử dụng trước khi tạo tài khoản. Cả hai trang đều có modal điều khoản sử dụng riêng, validate dữ liệu ở clientside, thông báo lỗi rõ ràng, và tự động chuyển hướng về dashboard sau khi đăng nhập hoặc đăng ký thành công. Hệ thống đăng nhập/đăng ký được bảo mật bằng JWT, bcrypt và tích hợp sẵn passport google oauth 2.0 cho xác thực bên thứ ba. Một modal cấu hình server đặc biệt cũng xuất hiện khi chạy trên ứng dụng di động (Capacitor) để người dùng nhập địa chỉ IP máy chủ backend.



Hình 4.2: Giao diện trang đăng ký

4.7.3 Giao diện trang đăng nhập

Đây là bước xác thực đầu tiên trước khi sử dụng hệ thống, giúp nhận diện đúng người dùng, bảo vệ dữ liệu cá nhân và mở quyền truy cập tương ứng với từng tài khoản.

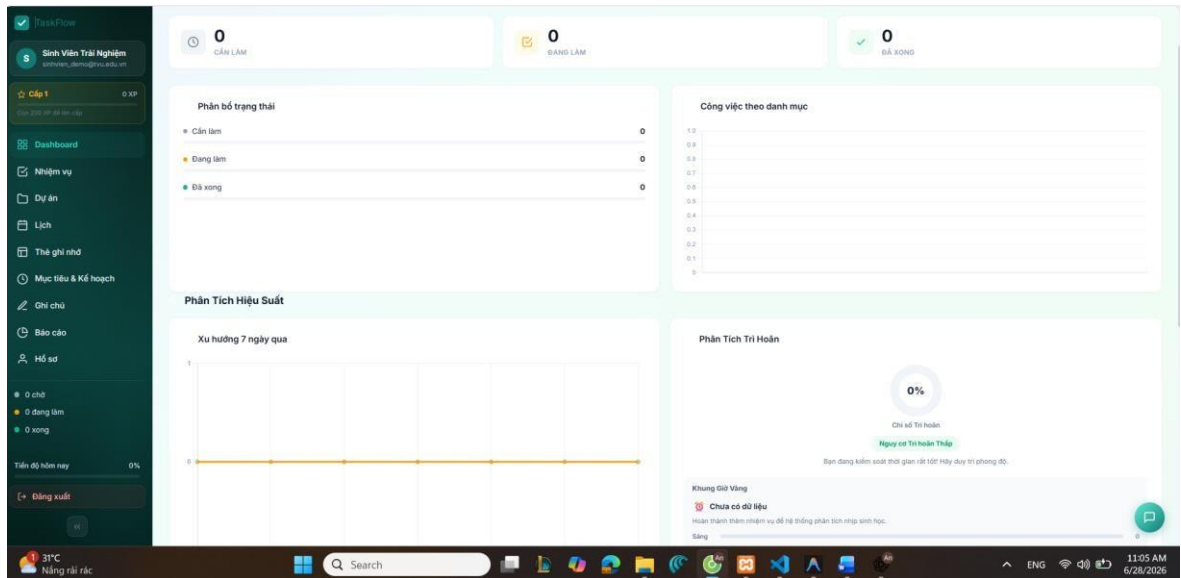


Hình 4.3: Giao diện trang đăng nhập

4.7.4 Giao diện trang dashboard người dùng

Trang dashboard là trang tổng quan chính, hiển thị thống kê công việc theo trạng thái (Cần làm/Đang làm/Đã xong/Quá hạn) dạng card clickable để lọc nhanh, biểu đồ thanh ngang phân bổ trạng thái và biểu đồ cột theo danh mục. Phần phân tích xu hướng 7 ngày so sánh task tạo mới vs hoàn thành, cùng công cụ phân tích Trì hoãn

với chỉ số %, mức độ rủi ro, lời khuyên cá nhân hóa và khung giờ vàng hoạt động. Các widget hiển thị sắp đến hạn, hiệu suất hôm nay (vòng tròn), tiến độ OKRs và thống kê nhanh. Hệ sinh thái liên kết cung cấp shortcut đến ghi chú, lịch, thời gian với insight thông minh. Banner lời mời nhóm học tập, nút làm mới và AI phân tích tích hợp chatbot.

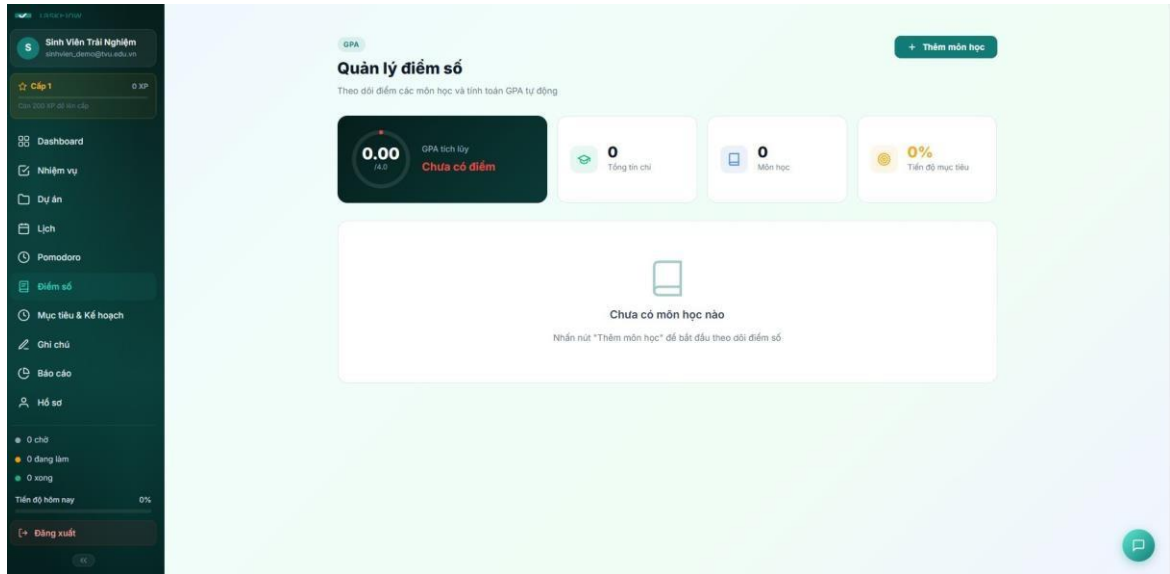


Hình 4.4: Giao diện trang dashboard

4.7.5 Giao diện trang quản lý điểm số

Chức năng “Quản lý điểm số” (GPA) cho phép sinh viên theo dõi và quản lý điểm các môn học trong quá trình học tập. Tại giao diện chính, hệ thống hiển thị 4 thông tin tổng quan: GPA tích lũy (thang điểm 4.0) với vòng tròn trực quan, tổng số tín chỉ đã học, số lượng môn học, và phần trăm tiến độ đạt mục tiêu GPA mà sinh viên đặt ra.

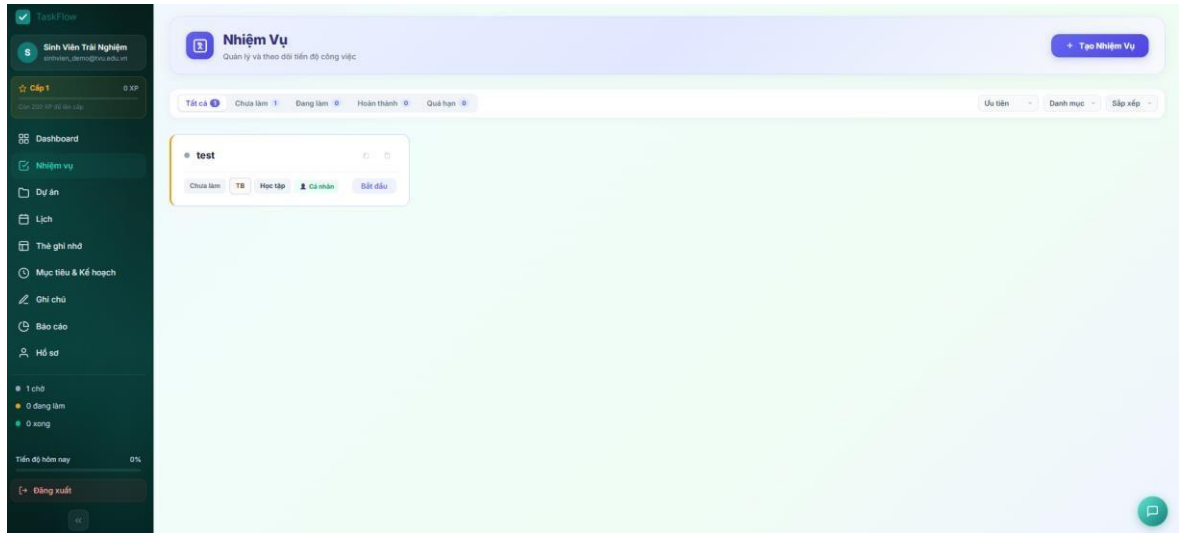
Khi thêm môn học mới, sinh viên nhập tên môn, số tín chỉ, ngày thi, cùng điểm thành phần bao gồm chuyên cần, giữa kỳ và cuối kỳ. Mỗi loại điểm có trọng số tương ứng (mặc định 10%, 30%, 60%), hệ thống tự động tính điểm tổng kết theo công thức weighted average và quy đổi sang thang điểm chữ (A+, A, B+, B, C+, C, D, F) cùng điểm hệ 4.0.



Hình 4.5: Giao diện trang quản lý điểm số

4.7.6 Giao diện trang nhiệm vụ

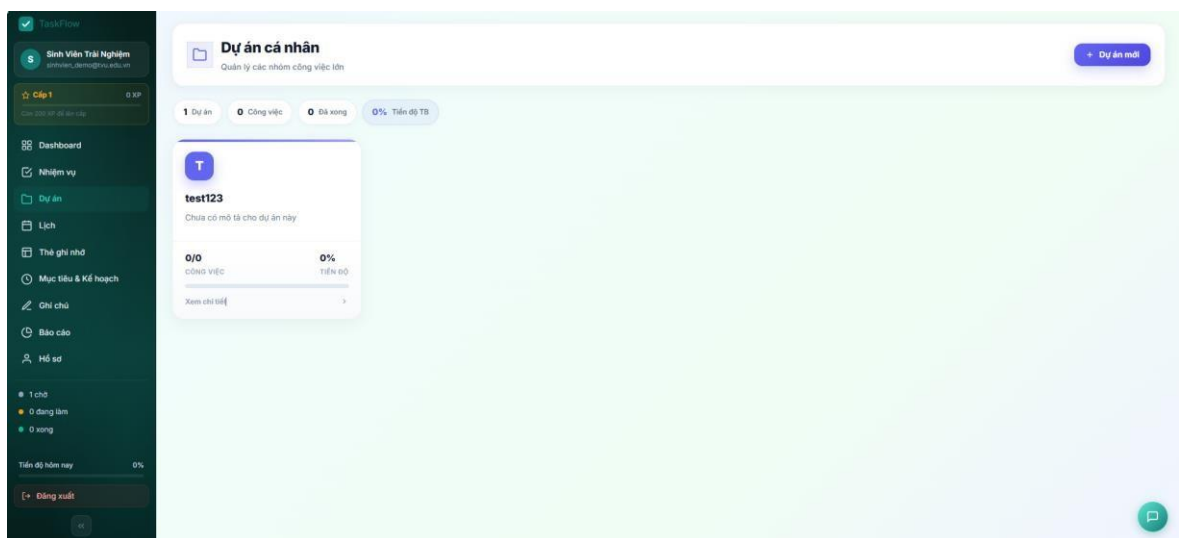
Trang nhiệm vụ (/tasks) là khu vực quản lý công việc trung tâm với header gradient tím chứa icon danh sách, tiêu đề “Nhiệm vụ” và nút “Tạo nhiệm vụ” nổi bật để thêm công việc mới. Thanh lọc chia 2 khu vực: bên trái là segmented buttons chọn trạng thái (Tất cả/Chưa làm/Đang làm/Hoàn thành/Quá hạn) mỗi badge hiển thị số lượng real-time, bên phải là dropdown lọc độ ưu tiên, danh mục, sắp xếp theo hạn gần/xa và nút bỏ lọc reset toàn bộ. Khu vực hiển thị dạng grid responsive hiển thị TaskCard với skeleton loading 6 placeholder khi đang tải. Mỗi card có khả năng chỉnh sửa, đổi trạng thái trực tiếp và mở form chi tiết đầy đủ (tiêu đề, mô tả, ưu tiên, danh mục, deadline, ghi chú). Trang tự động hiển thị trạng thái trống với icon và nút kêu gọi tạo nhiệm vụ đầu tiên khi chưa có dữ liệu, đồng thời real-time qua pinia store với toast notification phản hồi.



Hình 4.6: Giao diện trang nhiệm vụ

4.7.7 Giao diện trang dự án

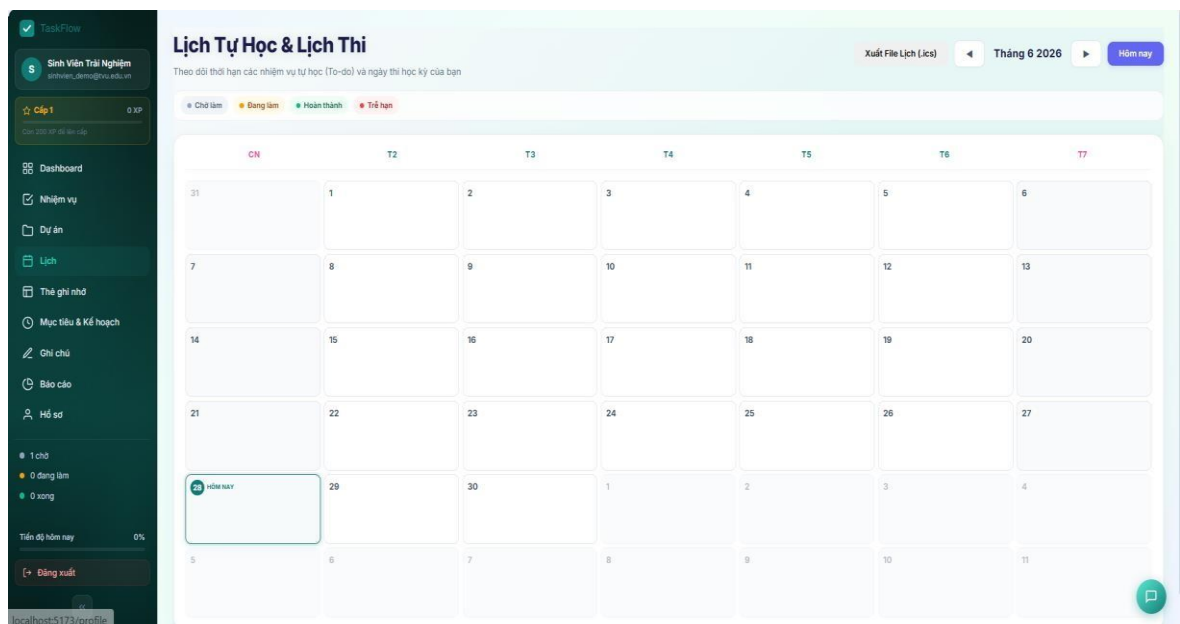
Trang Dự án (/projects) cho phép người dùng quản lý các nhóm công việc lớn dạng card grid. Header có nút “Dự án mới” để tạo project mới. Thanh thống kê hiển thị tổng số dự án, tổng công việc, đã xong và tiến độ trung bình. Mỗi project card hiển thị icon chữ cái đầu với màu tùy chọn, tên, badge môn học liên kết, mô tả, thanh progress cụ thể (số công việc hoàn thành/tổng + %), và nút “Xem chi tiết” để vào trang chi tiết. Hover card hiển thị nút Sửa/Xóa. Modal tạo/sửa dự án cho phép nhập tên, mô tả, chọn màu nhận diện từ 6 preset và liên kết môn học. Xóa dự án chỉ mất liên kết, không xóa task. Trạng thái trống hiển thị khi chưa có dự án với CTA tạo dự án đầu tiên.



Hình 4.7: Giao diện trang dự án cá nhân

4.7.8 Giao diện trang quản lịch học

Trang lịch (/calendar) giúp theo dõi thời hạn tự học và ngày thi học kỳ dạng calendar grid. Header có nút “Xuất File Lịch (.ics)” để đồng bộ google calendar, điều hướng tháng trước/sau và nút “Hôm nay”. Legend màu phân loại trạng thái task. Grid calendar hiển thị đầy đủ tháng hiện tại (được highlight border xanh) kèm ngày tháng prev/next mờ, header thứ T2-CN với cuối tuần màu hồng. Mỗi cell hiển thị tối đa 3 event badge có màu theo trạng thái (xám/vàng/xanh lá/đỏ) kèm thời gian, và "+n" khi overflow. Click mở modal chi tiết ngày với: ghi chú cá nhân (thêm/sửa/xóa lưu localStorage, highlight cell màu vàng), form tạo công việc mới (tiêu đề, mô tả, time picker, độ ưu tiên), và danh sách task trong ngày (sắp xếp quá hạn lên đầu, có badge trễ

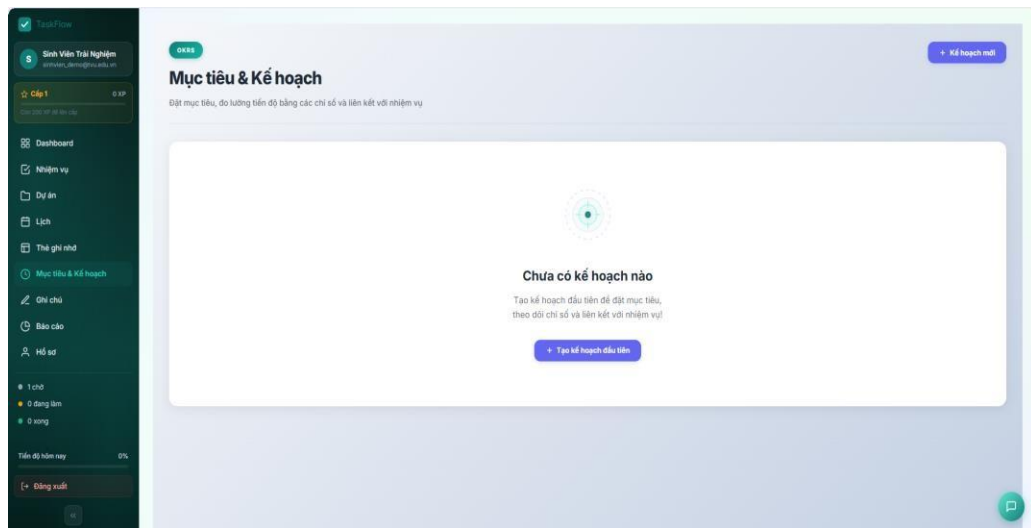


Hình 4.8: Giao diện trang lịch tự học

4.7.9 Giao diện trang mục tiêu

Trang Mục tiêu & Kế hoạch triển khai quy trình OKR (Objectives and Key Results) giúp người dùng đặt mục tiêu, đo lường tiến độ và liên kết với nhiệm vụ học tập. Trang có thể tạo các kế hoạch mới (ví dụ: kế hoạch tháng, học kỳ) với tên, ngày bắt đầu và kết thúc, theo dõi trạng thái thông qua badge Đang thực hiện/Hoàn thành/Nháp. Mỗi kế hoạch hiển thị card thống kê tổng hợp số mục tiêu, số đạt được và tổng chỉ số KRs, cùng danh sách mục tiêu (Objectives) dạng grid với độ ưu tiên và môn học liên kết. Bên trong mỗi mục tiêu có thể thêm Key Results (chỉ số đo

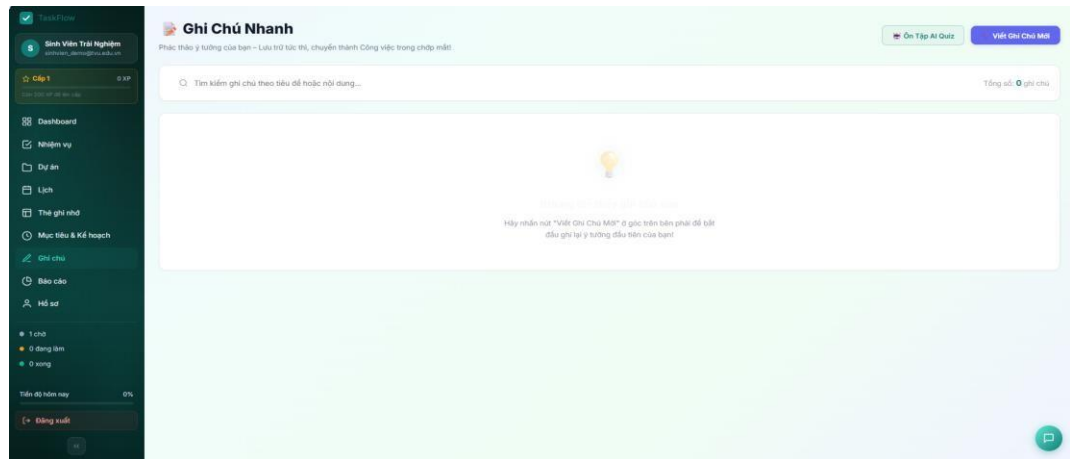
lượng) với giá trị mục tiêu, đơn vị (%)/điểm/bài/giờ/lần), theo dõi tiến độ qua progress bar và cập nhật giá trị thực tế. Đặc biệt, người dùng có thể liên kết và hủy liên kết các nhiệm vụ từ trang Nhiệm vụ với mục tiêu cụ thể, tạo thành hệ sinh thái quản lý mục tiêu - công việc thống nhất. Ngoài ra còn có module Quản lý môn học để thêm/xóa môn học.



Hình 4.9: Giao diện trang mục tiêu

4.7.10 Giao diện trang ghi chú

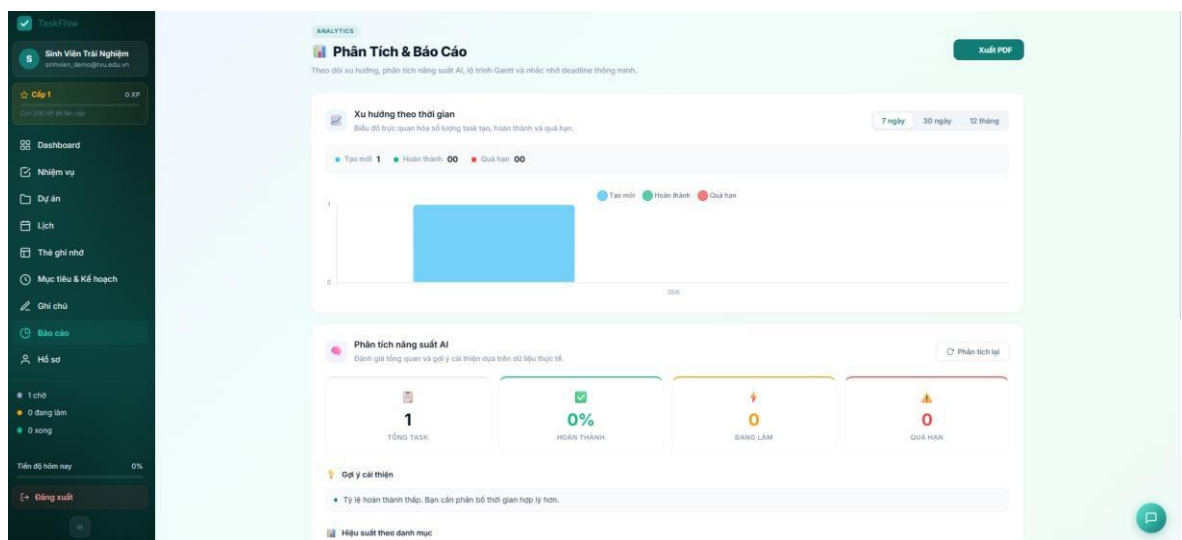
Trang ghi chú (/notes) cho phép người dùng phác thảo ý tưởng, lưu trữ tức thì và chuyển thành công việc trong chớp mắt. Header có nút “Viết ghi chú mới” tạo card ghi chú mới, và “Ôn Tập AI Quiz” mở hệ thống quiz thông minh. Thanh tìm kiếm lọc ghi chú theo tiêu đề/nội dung, hiển thị tổng số ghi chú real-time. Grid card ghi chú có thanh màu tùy chỉnh (chọn từ palette), hiển thị tiêu đề, nội dung, thời gian cập nhật và nút xóa. Chức năng “Chuyển Task” chuyển đổi ghi chú thành công việc mới với chọn độ ưu tiên, danh mục và hạn chót. Tính năng “Tạo Quiz AI” biến ghi chú thành bộ câu hỏi trắc nghiệm 5 câu, có kéo thả file PDF/TXT/MD hoặc chọn ghi chú có sẵn, load AI với hiệu ứng brain pulse, chọn 1 trong 4 đáp án A/B/C/D, hiển thị giải thích chi tiết sau mỗi câu, điểm số cuối kèm badge khen , và lưu kết quả vào flashcards để ôn tập lại bằng thuật toán spaced repetition. Modal sửa ghi chú có autosave khi typing, thay đổi màu real-time.



Hình 4.10: Giao diện trang quản lý thông báo

4.7.11 Giao diện trang báo cáo

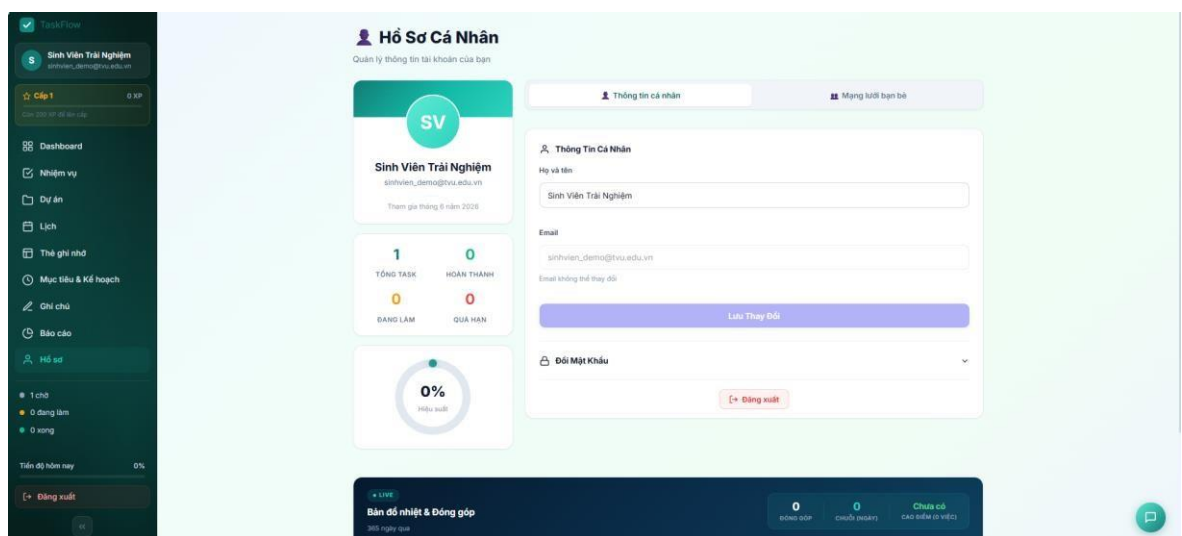
Trang báo cáo (/reports) là công cụ phân tích và báo cáo tổng hợp với 4 module chính. Xu hướng theo thời gian vẽ biểu đồ đường/bánh ngành hiển thị số task tạo mới/hoàn thành/quá hạn, người dùng có thể chuyển đổi giữa 3 chế độ xem: 7 ngày, 30 ngày hoặc 12 tháng, kèm thanh tổng kết số liệu. Phân tích năng suất AI gọi API AI để đánh giá tổng quan và đưa ra gợi ý cải thiện, hiển thị 4 card thống kê (tổng task, % hoàn thành, đang làm, quá hạn), danh sách insights và thanh hiệu suất theo từng danh mục. Biểu đồ Gantt trực quan hóa timeline các nhiệm vụ, giải thích từng thanh bar theo trạng thái, có thể scroll ngang để xem toàn bộ thời gian. Nhắc nhở deadline hiển thị các task sắp đến hạn với bộ lọc thời gian (1 giờ/6 giờ/24 giờ/3 ngày), cho phép gửi email nhắc nhở tự động trực tiếp từ trang. Tính năng xuất PDF.



Hình 4.11: Giao diện trang nhật ký hoạt động

4.7.12 Giao diện trang hồ sơ

Trang hồ Sơ (/profile) là trung tâm quản lý tài khoản cá nhân gồm 2 tab chính Thông tin cá nhân và Mạng lưới bạn bè. Ở tab Thông tin cá nhân người dùng có thể upload ảnh đại diện tối đa 5MB với hiệu ứng hover overlay và fallback initials gradient xem thông tin tên email ngày tham gia xem thống kê mini 4 ô tổng task hoàn thành đang làm quá hạn xem vòng tròn hiệu suất SVG hiển tỷ lệ % hoàn thành chỉnh sửa họ tên đổi mật khẩu với form đầy đủ có nút toggle hiện ẩn mật khẩu và thanh độ mạnh màu sắc 4 cấp yếu trung bình khá tốt mạnh và nút đăng xuất. Ở tab mạng lưới bạn bè có chức năng tìm kiếm bạn học qua email gửi lời mời kết bạn xem lời mời đang chờ với badge đỏ animation pulse chấp nhận hoặc từ chối xem danh sách bạn bè kèm trạng thái online realtime qua websocket và nút hủy kết bạn.

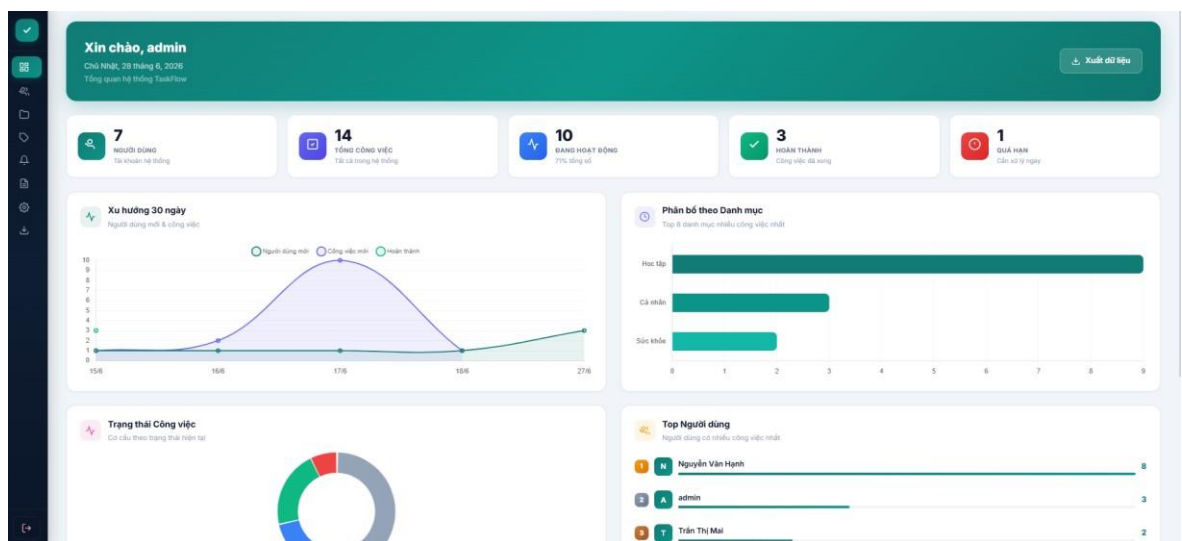


Hình 4.12: Giao diện trang cài đặt

4.7.13 Giao diện trang dashboard admin

Trang admin là khu vực quản trị hệ thống dành riêng cho tài khoản có quyền admin, được bảo vệ hoàn toàn bởi middleware kiểm tra vai trò và tách biệt hoàn toàn khỏi giao diện người dùng thường. Giao diện đăng nhập admin có gradient tím-xanh riêng, chấp nhận email hoặc username kết hợp mật khẩu; nếu người dùng thường cố truy cập sẽ tự động bị đẩy về trang login, còn admin đã đăng nhập sẽ được chuyển thẳng vào dashboard. Dashboard admin hiển thị tổng quan hệ thống với bốn card

thống kê nổi bật gồm tổng người dùng, tổng công việc, tổng dự án và tỷ lệ hoàn thành, kèm bộ biểu đồ chart.js phân tích xu hướng 30 ngày về người dùng mới và công việc, phân bố theo danh mục, trạng thái công việc và bảng xếp hạng người dùng tích cực nhất; nút xuất dữ liệu cho phép tải báo cáo tổng hợp. Module quản lý người dùng cung cấp bảng dữ liệu đầy đủ với tìm kiếm theo tên và email, lọc theo vai trò admin-manager-user có badge đếm số lượng, cho phép đổi vai trò trực tiếp qua dropdown, chặn hoặc bỏ chặn tài khoản và xóa vĩnh viễn, đồng thời bảo vệ tài khoản admin khỏi các thao tác thay đổi cấu hình. Module quản lý danh mục gồm hai cột: bên trái là form thêm danh mục mới với color picker chọn màu, bên phải là danh sách các danh mục đã tạo kèm nút xóa. Module quản lý thông báo cho phép tạo thông báo toàn hệ thống với tiêu đề, nội dung, loại thông báo thông tin-thành công-cảnh báo-lỗi, tùy chọn kích hoạt ngay và lịch sử thông báo với trạng thái bật-tắt. Module Cài đặt Hệ thống lưu các giá trị cấu hình toàn cục như tên ứng dụng, mã màu chủ đạo và dung lượng tải lên tối đa, đồng bộ toàn bộ client thông qua API. Module Xuất Dữ liệu cung cấp link tải file CSV chứa toàn bộ người dùng, công việc và subtask để phân tích hoặc báo cáo bên ngoài. Module quản lý dự án hiển thị toàn bộ dự án trên hệ thống dạng grid card với thống kê thành viên, công việc, tiến độ hoàn thành và chủ sở hữu. Module nhật ký hoạt động ghi lại chi tiết từng hành vi trên hệ thống với bảng có tìm kiếm, lọc theo loại hành động, hiển thị thời gian, người thực hiện có avatar và thông báo hệ thống.



Hình 4.13: Giao diện trang dashboard admin

4.7.14 Giao diện quản lý người dùng

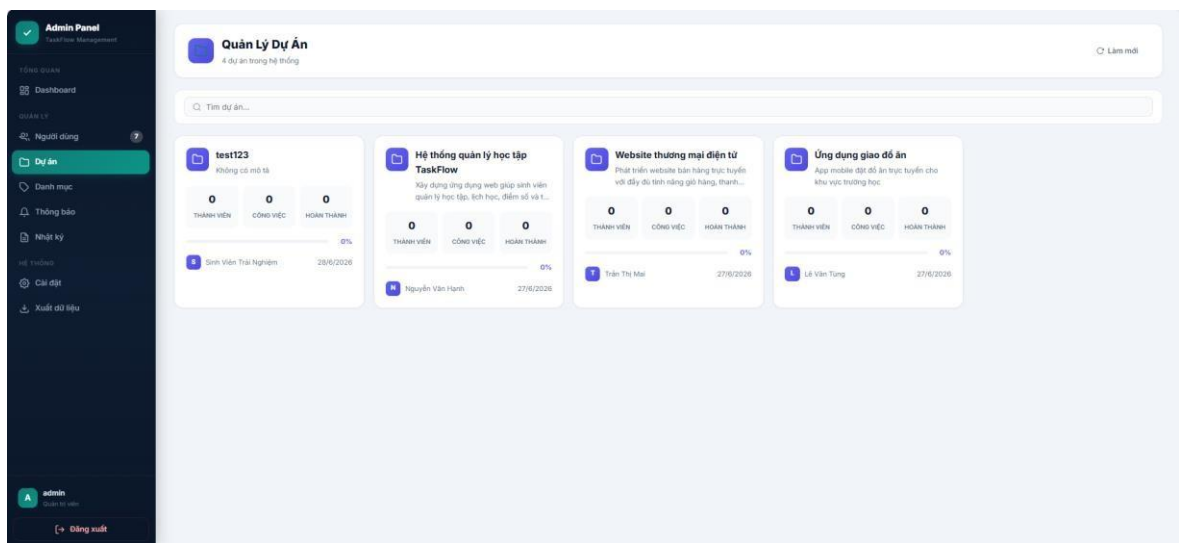
Trang quản lý người dùng trong khu vực admin cho phép admin xem và điều chỉnh toàn bộ tài khoản trên hệ thống một cách trực quan và nhanh chóng. Đầu trang có thanh tìm kiếm để lọc theo tên hoặc email, kết hợp với các nút lọc theo vai trò như tất cả, admin, manager, user mỗi loại có số lượng người dùng hiển thị ngay bên cạnh để admin nắm bắt tổng quan phổ quyền trong hệ thống. Sau khi lọc xong, admin nhìn thấy bảng danh sách người dùng với đầy đủ thông tin: ID, ảnh đại diện hoặc chữ cái đầu tên nếu chưa có ảnh, tên, email, vai trò có thể thay đổi ngay tại chỗ bằng dropdown, số lượng công việc đang giữ, và ngày tham gia hệ thống. Admin có thể nhấp vào biểu tượng mắt để mở modal xem chi tiết sâu hơn gồm thống kê công việc của người đó như tổng công việc, hoàn thành, đang làm, quá hạn, danh sách dự án đang tham gia và vài công việc gần nhất với trạng thái và độ ưu tiên rõ ràng. Về thao tác quản lý, admin có thể chặn tài khoản bằng nút cấm hình tròn gạch chéo màu đỏ, người bị chặn sẽ bị làm mờ toàn bộ hàng và có badge “Đã chặn” màu đỏ bên cạnh tên để nhận biết dễ dàng; bỏ chặn chỉ cần nhấp lại cùng nút đó. Nếu muốn xóa vĩnh viễn tài khoản, admin dùng nút thùng rác màu đỏ, hệ thống sẽ hiện hộp thoại xác nhận để tránh xóa nhầm. Riêng tài khoản của chính admin đang đăng nhập thì bị khóa hoàn toàn các thao tác đổi vai trò và xóa để bảo vệ bản thân. Cuối cùng có nút xuất CSV để tải toàn bộ danh sách người dùng đã lọc ra file excel để lưu trữ hoặc báo cáo bên ngoài.

ID	NGƯỜI DÙNG	EMAIL	VAI TRÒ	CÔNG VIỆC	NGÀY THAM GIA	THAO TÁC
#7	Sinh Viên Trải Nghiệm	sinhvien_demo@vu.edu.vn	User	1	27/6/2026	
#6	Test User	test@gmail.com	User	0	27/6/2026	
#5	admin	admin@taskflow.com	Admin	3	27/6/2026	
#4	Phạm Thị Lan	lan@gmail.com	User	0	18/6/2026	
#3	Lê Văn Tùng	tung@gmail.com	User	0	17/6/2026	
#2	Trần Thị Mai	mai@gmail.com	User	2	16/6/2026	
#1	Nguyễn Văn Hạnh	hanh@gmail.com	User	8	15/6/2026	

Hình 4.14: Giao diện trang quản lý người dùng

4.7.15 Giao diện trang quản lý dự án

Trang quản lý dự án trong admin cho phép xem và giám sát toàn bộ dự án trên hệ thống dạng grid card hiện đại. Đầu trang có thanh tìm kiếm để lọc nhanh theo tên dự án, giúp admin tìm thấy dự án cụ thể ngay cả khi có hàng trăm dự án. Mỗi project card hiển thị đầy đủ thông tin: tên dự án, mô tả ngắn hoặc “Không có mô tả” nếu trống, thống kê nhanh gồm số thành viên tham gia, tổng công việc và số công việc đã hoàn thành để admin đánh giá quy mô và tiến độ ngay từ bên ngoài. Phần progress bar ở dưới mỗi card thể hiện phần trăm hoàn thành một cách trực quan kèm con số cụ thể, giúp admin nhận biết ngay dự án nào đang trễ tiến độ. Footer mỗi card hiển thị chủ sở hữu dự án dạng avatar chữ cái đầu và ngày tạo để quản lý trách nhiệm rõ ràng. Trang tự động hiển thị trạng thái trống với icon và thông báo “Không tìm thấy dự án nào” khi kết quả tìm kiếm không khớp, đảm bảo trải nghiệm người dùng luôn nhất quán và thân thiện.

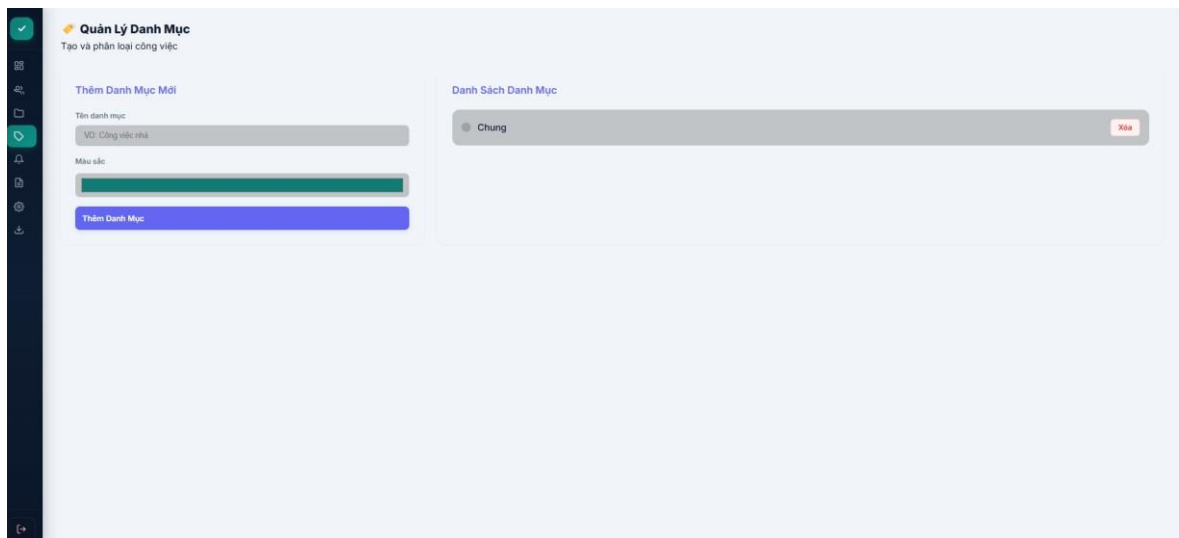


Hình 4.15: Giao diện trang quản lý dự án

4.7.16 Giao diện trang quản lý danh mục

Trang quản lý danh mục trong admin có giao diện 2 cột đơn giản và dễ sử dụng. Bên trái là form thêm danh mục mới, admin chỉ cần nhập tên danh mục và chọn màu sắc bằng color picker rồi bấm “Thêm danh mục” là xong, có thông báo toast phản hồi thành công hay lỗi. Bên phải là danh sách các danh mục hiện có trên

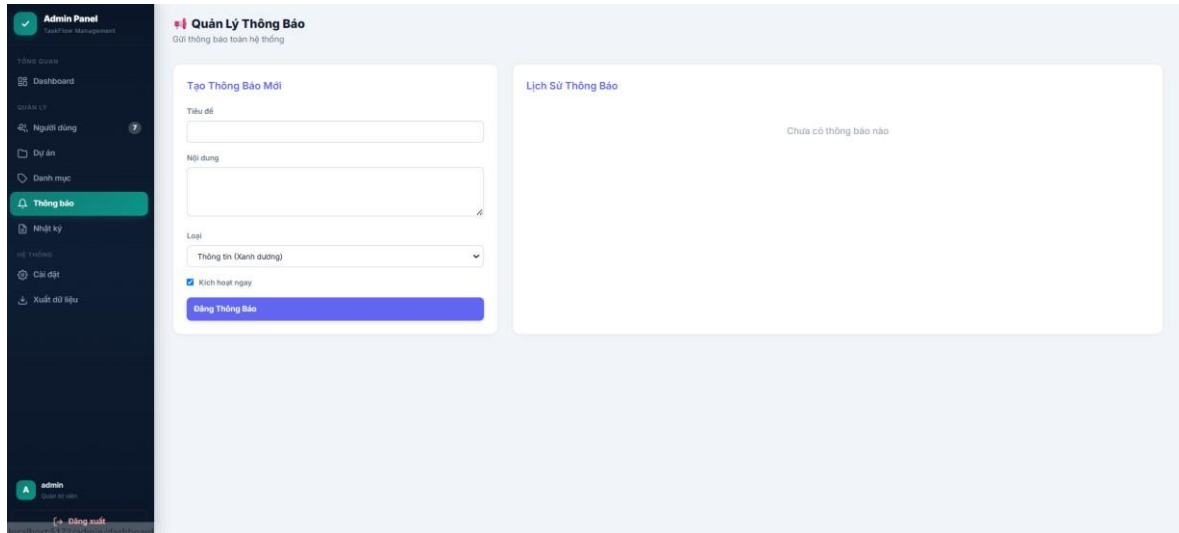
hệ thống, mỗi danh mục hiển thị một chấm màu tương ứng với màu admin đã chọn lúc tạo, kèm tên danh mục và nút “Xóa” để xóa bỏ nếu không cần dùng nữa. Khi admin xóa, hệ thống hiện hộp thoại xác nhận “Bạn có chắc muốn xóa?” để tránh xóa nhầm. Nếu chưa có danh mục nào thì hệ thống hiển thị thông báo “Chưa có danh mục nào”. Toàn trang load dữ liệu tự động khi vào và cho phép admin thêm/xóa danh mục thoải mái, giúp người dùng sau này có thể phân loại công việc theo đúng nhu cầu của tổ chức.



Hình 4.16: Giao diện trang quản lý danh mục

4.7.17 Giao diện trang quản lý thông báo

Trang quản lý thông báo trong admin có giao diện 2 cột rõ ràng để tạo và theo dõi thông báo hệ thống. Bên trái là form tạo thông báo mới, admin nhập tiêu đề và nội dung, chọn loại thông báo bằng dropdown gồm 4 màu sắc: thông tin màu xanh dương, thành công màu xanh lá, cảnh báo màu vàng và lỗi màu đỏ, đồng thời có checkbox để quyết định có kích hoạt thông báo ngay lập tức hay tạo nháp trước. Bên phải là lịch sử thông báo đã đăng, mỗi thông báo có viền màu tương ứng với loại để phân biệt nhanh, hiển thị tiêu đề, nội dung, thời gian tạo và badge trạng thái “Đang bật” màu xanh lá hoặc “Đã tắt” màu xám để admin biết thông báo nào đang hiển thị trên hệ thống. Admin có thể xóa từng thông báo bằng nút xóa, tất cả thay đổi đều có toast notification phản hồi kết quả thành công hay thất bại, giúp quản lý thông báo toàn hệ thống trở nên đơn giản và hiệu quả.



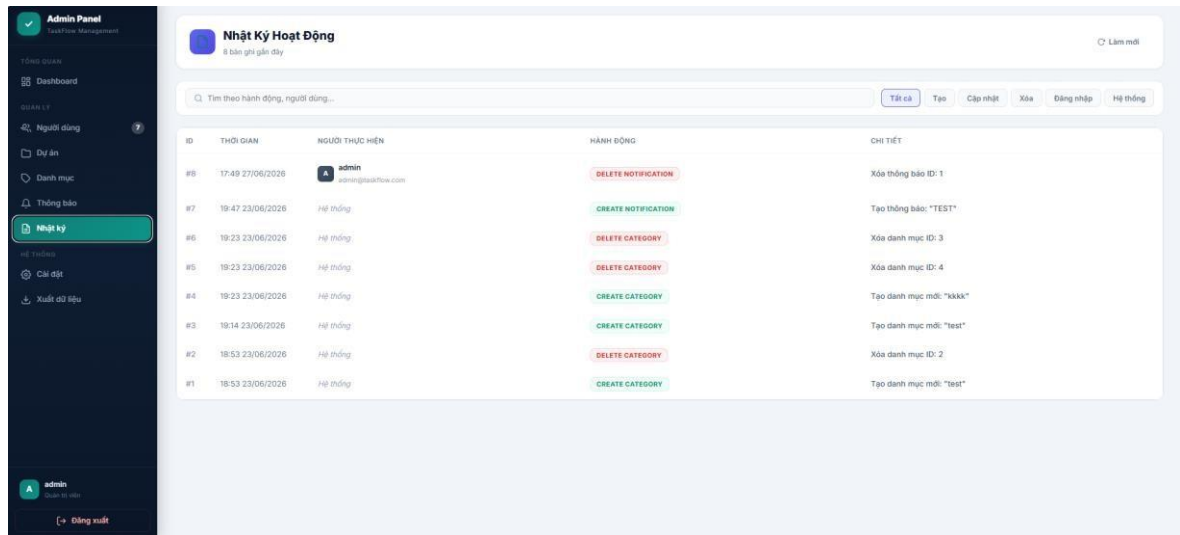
Hình 4.17: Giao diện trang quản lý thông báo

4.7.18 Giao diện trang quản lý thông báo

Chức năng nhật ký hoạt động được xây dựng nhằm giúp quản trị viên theo dõi toàn bộ các thao tác diễn ra trong hệ thống. Mỗi khi người dùng hoặc quản trị viên thực hiện một hành động quan trọng như tạo, cập nhật, xóa dữ liệu, đăng nhập hoặc thao tác với các chức năng quản trị, hệ thống sẽ tự động ghi lại thông tin để phục vụ cho việc kiểm tra và quản lý sau này.

Giao diện hiển thị các bản ghi dưới dạng bảng, bao gồm các thông tin như ID, thời gian thực hiện, người thực hiện, hành động và chi tiết. Nhờ đó, quản trị viên có thể dễ dàng biết được ai đã thực hiện thao tác, thực hiện vào thời điểm nào và nội dung thay đổi cụ thể là gì. Ví dụ, khi một danh mục được tạo hoặc xóa, hoặc khi một thông báo mới được thêm vào hệ thống, các thông tin này sẽ được lưu lại đầy đủ trong nhật ký.

Để thuận tiện cho việc tra cứu, hệ thống cung cấp ô tìm kiếm cho phép tìm theo tên người dùng hoặc nội dung hành động. Bên cạnh đó, các nút lọc như tất cả, tạo, cập nhật, xóa, đăng nhập và hệ thống giúp quản trị viên nhanh chóng lọc và xem các bản ghi theo từng loại thao tác mà không cần phải tìm kiếm thủ công.

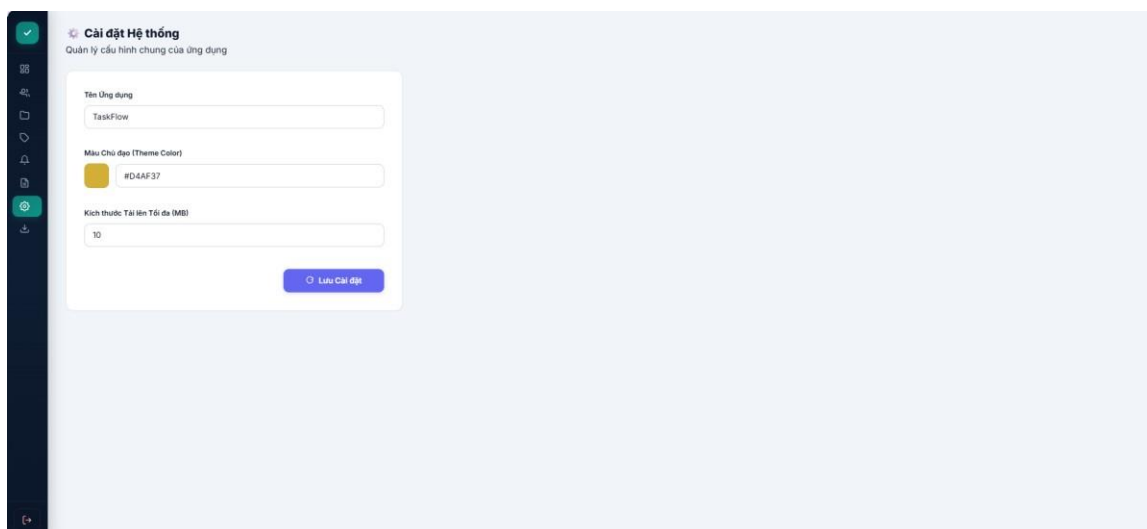


Hình 4.18: Giao diện trang nhật ký hoạt động

4.7.19 Giao diện trang quản lý thông báo

Chức năng cài đặt hệ thống được xây dựng nhằm cho phép quản trị viên cấu hình các thông tin chung của ứng dụng mà không cần can thiệp trực tiếp vào mã nguồn. Các thiết lập sau khi được lưu sẽ được áp dụng cho toàn bộ hệ thống, giúp việc quản lý và thay đổi cấu hình trở nên thuận tiện hơn.

Giao diện cài đặt bao gồm các trường thông tin cơ bản như tên ứng dụng, màu chủ đạo (Theme Color) và kích thước tải lên tối đa. Trường tên ứng dụng cho phép thay đổi tên hiển thị của hệ thống trên giao diện. Trường màu chủ đạo hỗ trợ lựa chọn màu sắc chính của ứng dụng thông qua mã màu (HEX), từ đó đồng bộ màu sắc của các thành phần giao diện như nút bấm, thanh điều hướng và các thành phần khác. Trường kích thước tải lên tối đa được sử dụng để quy định dung lượng tối đa của các tệp đính kèm mà người dùng được phép tải lên hệ thống, góp phần hạn chế việc lưu trữ các tệp có kích thước quá lớn.

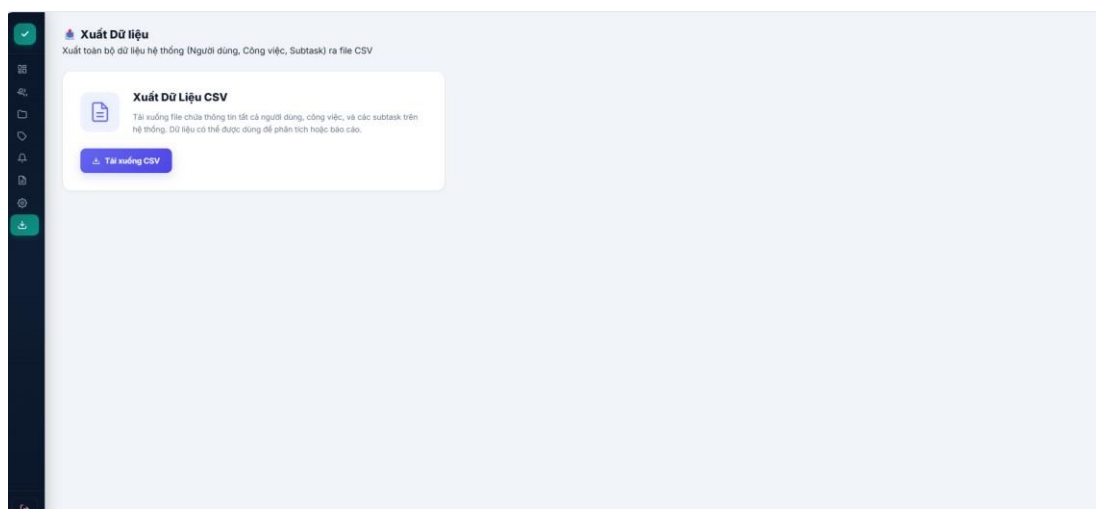


Hình 4.19: Giao diện trang cài đặt hệ thống

4.7.20 Giao diện trang quản lý thông báo

Chức năng xuất dữ liệu được xây dựng nhằm hỗ trợ quản trị viên sao lưu và trích xuất dữ liệu của hệ thống dưới định dạng CSV để phục vụ cho việc thống kê, phân tích hoặc lưu trữ. Dữ liệu được xuất bao gồm các thông tin chính như người dùng, công việc (Tasks) và các nhiệm vụ con (Subtasks).

Giao diện chức năng được thiết kế đơn giản với một thẻ thông tin giới thiệu mục đích của chức năng và nút tải xuống CSV. Khi quản trị viên nhấn nút này, hệ thống sẽ gửi yêu cầu đến backend để tổng hợp dữ liệu từ các bảng liên quan trong cơ sở dữ liệu. Sau khi xử lý hoàn tất, máy chủ sẽ tạo một tệp CSV và tự động gửi về trình duyệt để người dùng tải xuống.



Hình 4.20: Giao diện trang xuất dữ liệu

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đề tài “Thiết kế và xây dựng hệ thống quản lý công việc dựa trên nền tảng web” tashkflow đã được thực hiện thành công với mục tiêu xây dựng một ứng dụng quản lý và theo dõi tiến độ công việc, học tập tập trung, hiện đại, hỗ trợ người dùng lập kế hoạch, quản lý thời gian và nâng cao năng suất cá nhân. Sau quá trình nghiên cứu, phân tích, thiết kế và triển khai thực tế, đề tài đã đạt được các kết quả cụ thể như sau:

Về mặt lý thuyết, đề tài đã nghiên cứu và hệ thống hóa các kiến thức nền tảng về kiến trúc Client-Server ba tầng, RESTful API, vue.js 3 sử dụng composition API, node.js, express.js, MySQL, cơ chế xác thực JWT, Google OAuth 2.0 qua Passport, kết nối thời gian thực socket.io [15] cùng các thư viện hỗ trợ như bcryptjs, chart.js, pdfkit, html2pdf.js, nodemailer, multer, pinia, vue router 4 và google generative AI. Những kiến thức này đã được vận dụng hiệu quả trong suốt quá trình xây dựng hệ thống. Đặc biệt, việc tích hợp trí tuệ nhân tạo thông qua mô hình google generative AI để phát triển các tính năng như trợ lý ảo AI coach chat, phân tích năng suất cá nhân, tự động phân rã công việc và tạo quiz từ ghi chú là điểm mới mang lại giá trị thực tiễn rất cao. Việc lựa chọn công nghệ mã nguồn mở phù hợp giúp tối ưu hóa chi phí phát triển và đảm bảo khả năng mở rộng hệ thống trong tương lai.

Về mặt phân tích và thiết kế, thông qua khảo sát thực tế về thói quen và những khó khăn trong việc quản lý công việc của người dùng, đề tài đã xác định được các hạn chế chính như dữ liệu công việc bị phân tán trên nhiều công cụ thủ công, khó khăn trong lập kế hoạch dài hạn, xu hướng trì hoãn khó kiểm soát và nhu cầu về một hệ thống tích hợp duy nhất. Từ đó, hệ thống tashkflow đã được thiết kế với kiến trúc phân tầng ba lớp rõ ràng bao gồm presentation layer, application layer và data layer. Cơ sở dữ liệu được thiết kế tối ưu với 25 bảng quan hệ sử dụng InnoDB Engine, thiết lập đầy đủ các ràng buộc khóa ngoại và chỉ mục để đảm bảo toàn vẹn dữ liệu cũng như tốc độ truy vấn nhanh chóng. Hệ thống phân định rõ ràng quyền hạn của hai tác nhân chính gồm người dùng (User), quản trị viên (Admin).

Về mặt triển khai thực tế, phân hệ frontend được xây dựng dưới dạng ứng dụng đơn trang SPA thuần javascript theo cấu trúc View-based module pattern với 25 trang tính năng như dashboard, kanban, projects, calendar, timebox, eisenhower, focus room, habit tracker, timetable cùng các công cụ học tập khác. Giao diện được thiết kế hiện đại theo phong cách card-based với màu sắc chủ đạo là teal #117c75, sử dụng CSS thuần và hệ thống CSS variables để tùy biến giao diện nhanh chóng thay vì sử dụng các framework CSS cồng kềnh. Trạng thái ứng dụng được quản lý tập trung thông qua pinia store, giao tiếp dữ liệu thời gian thực qua socket.io và hỗ trợ đóng gói ứng dụng di động thông qua capacitor. Phân hệ backend chạy trên nền tảng node.js và express.js với hơn 50 endpoint API được thiết kế theo mô hình MVC, tích hợp cơ chế bảo mật chống SQL injection, mã hóa mật khẩu bcrypt và phân quyền truy cập nghiêm ngặt. Các dịch vụ gửi mail tự động, tạo báo cáo PDF và xử lý AI hoạt động mượt mà và đồng bộ.

Về mặt kiểm thử và đánh giá, hệ thống đã trải qua quy trình kiểm thử đa tầng bao gồm kiểm thử đơn vị đối với các logic cốt lõi như tính điểm gamification hay thuật toán lặp lại ngắt quãng SM-2, kiểm thử tích hợp API, kiểm thử hiệu năng bằng Apache JMeter chịu tải tốt với tỷ lệ lỗi 0% và kiểm thử bảo mật. Kết quả cho thấy hệ thống hoạt động ổn định, bảo mật tốt và đáp ứng đầy đủ các yêu cầu chức năng đề ra.

Đánh giá tổng quát, Tashkflow là một giải pháp quản lý công việc toàn diện, tích hợp AI thông minh, hỗ trợ trọn vẹn vòng đời công việc từ khi lên kế hoạch cho đến khi hoàn thành và đánh giá hiệu suất. Hệ thống góp phần giải quyết triệt để bài toán trì hoãn, thay thế các công cụ rời rạc bằng một nền tảng thống nhất và giúp người dùng đưa ra các quyết định cải thiện bản thân dựa trên số liệu thực tế.

5.2. Hướng phát triển

Mặc dù đã đạt được những kết quả tích cực, hệ thống vẫn còn một số điểm cần cải tiến và có thể mở rộng trong tương lai theo các hướng sau:

5.2.1 Phát triển ứng dụng di động đa nền tảng

Hiện tại, hệ thống đã có phiên bản đóng gói thử nghiệm trên android thông qua capacitor [26] nhưng chưa được tối ưu hóa hoàn toàn cho trải nghiệm di động. Trong tương lai, việc xây dựng ứng dụng gốc (native) cho cả hai hệ điều hành android và ios bằng react native hoặc flutter sẽ giúp người dùng nhận thông báo đẩy nhanh chóng, đồng bộ công việc mượt mà và tận dụng tốt hơn các phần cứng thiết bị như camera để quét tài liệu hoặc định vị vị trí để nhắc nhở công việc theo khu vực địa lý.

5.2.2 Tích hợp công nghệ NFC và mã QR cho quản lý thực tế

Nhằm kết nối không gian làm việc vật lý với hệ thống số, Tashkflow hướng tới việc tích hợp công nghệ quét mã QR và kết nối cận trường NFC. Người dùng có thể dán các thẻ NFC hoặc mã QR lên bàn làm việc, tài liệu học tập hoặc thiết bị cá nhân. Khi quét các mã này bằng điện thoại, hệ thống sẽ tự động kích hoạt trạng thái tập trung Pomodoro tương ứng, mở nhanh ghi chú liên quan hoặc ghi nhận thời gian làm việc thực tế cho tác vụ đó.

5.2.3 Cảnh báo thông minh dựa trên dữ liệu lịch sử cá nhân

Mặc dù hệ thống đã có tính năng phân tích hiệu suất cơ bản, hướng phát triển tiếp theo sẽ áp dụng các mô hình học máy chuyên sâu để phân tích hành vi của người dùng theo thời gian. Hệ thống có thể phát hiện các thói quen trì hoãn đối với từng nhóm công việc cụ thể, nhận diện thời điểm làm việc kém hiệu quả trong ngày để đưa ra những cảnh báo và lời khuyên mang tính chủ động, giúp người dùng điều chỉnh kế hoạch trước khi xảy ra tình trạng quá tải.

5.2.4 Mở rộng khả năng quản lý đa tổ chức và nhóm quy mô lớn

Hệ thống hiện tại đang tập trung vào cá nhân và nhóm làm việc nhỏ. Hướng phát triển tới là mở rộng kiến trúc để hỗ trợ quản lý dự án cho các tổ chức, doanh nghiệp hoặc cơ sở giáo dục lớn. Hệ thống sẽ cho phép phân chia không gian làm việc theo từng phòng ban hoặc chi nhánh khác nhau, giúp ban quản lý dễ dàng theo dõi

tiền độ tổng thể của toàn bộ hệ thống trong khi vẫn đảm bảo tính riêng tư của dữ liệu nội bộ từng nhóm.

5.2.5 Phát triển API mở và tích hợp hệ sinh thái bên thứ ba

Xây dựng cổng API mở theo chuẩn RESTful để giúp tashkflow dễ dàng kết nối và trao đổi dữ liệu với các ứng dụng phổ biến khác mà người dùng đang sử dụng hàng ngày như lịch làm việc google calendar, outlook, các phần mềm họp trực tuyến zoom, microsoft teams hoặc các hệ thống quản lý học tập phổ biến như moodle và canvas. Việc này giúp giảm thiểu thao tác nhập liệu thủ công và tạo ra một luồng công việc liền mạch.

5.2.6 Nâng cấp kiến trúc hệ thống và triển khai điện toán đám mây

Để chuẩn bị cho lượng người dùng và dữ liệu gia tăng lớn trong tương lai, hệ thống cần chuyển đổi từ kiến trúc monolithic hiện tại sang kiến trúc microservices. Khi đó, các dịch vụ như quản lý tác vụ, công cụ AI, dịch vụ thông báo và tính toán gamification sẽ hoạt động độc lập, giúp việc bảo trì và nâng cấp trở nên dễ dàng hơn. Đồng thời, việc triển khai trên các dịch vụ đám mây như AWS hoặc google cloud sẽ đảm bảo tính sẵn sàng cao và khả năng tự động co giãn theo tải thực tế.

5.2.7 Bổ sung công cụ phân tích trực quan nâng cao

Nâng cấp trang báo cáo hiệu suất với các biểu đồ trực quan hóa dữ liệu thông minh hơn. Người dùng sẽ có khả năng tự cấu hình bảng điều khiển dashboard cá nhân, thiết lập tự động xuất báo cáo định kỳ gửi qua email và theo dõi sâu các chỉ số năng suất thông qua biểu đồ phân bổ thời gian thực tế, biểu đồ năng lượng làm việc và phân tích chi tiết dòng công việc hàng tháng.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] Fielding, R. T. (2000). "Architectural Styles and the Design of Network-based Software Architectures." Doctoral dissertation, University of California, Irvine.
- [2] Richardson, L. & Ruby, S. (2007). "RESTful Web Services." O'Reilly Media, ISBN 978-0-596-52926-0.
- [3] Mozilla Developer Network. (2024). "HTML: HyperText Markup Language." [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>. [Truy cập: 15-05-2026].
- [4] Mozilla Developer Network. (2024). "CSS: Cascading Style Sheets." [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS>. [Truy cập: 15-05-2026].
- [5] Ecma International. (2024). "ECMAScript 2024 Language Specification." [Online]. Available: <https://tc39.es/ecma262/>. [Truy cập: 15-05-2026].
- [6] Vue.js. (2024). "Vue.js 3 – The Progressive JavaScript Framework." [Online]. Available: <https://vuejs.org/>. [Truy cập: 15-05-2026].
- [7] Pinia. (2024). "Pinia – The intuitive, type-safe, light and flexible Store for Vue." [Online]. Available: <https://pinia.vuejs.org/>. [Truy cập: 15-05-2026].
- [8] Vue Router. (2024). "Vue Router 4 – Official Router for Vue.js." [Online]. Available: <https://router.vuejs.org/>. [Truy cập: 15-05-2026].
- [9] Vite. (2024). "Vite – Next Generation Frontend Tooling." [Online]. Available: <https://vitejs.dev/>. [Truy cập: 15-05-2026].
- [10] Node.js. (2024). "Node.js – JavaScript Runtime Built on Chrome's V8 JavaScript Engine." [Online]. Available: <https://nodejs.org/>. [Truy cập: 15-05-2026].
- [11] Express.js. (2024). "Express – Fast, unopinionated, minimalist web framework for Node.js." [Online]. Available: <https://expressjs.com/>. [Truy cập: 15-05-2026].

- [12] MySQL. (2024). "MySQL 8.0 Reference Manual." [Online].

Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Truy cập: 15-05-2026].

- [13] Axios. (2024). "Axios – Promise based HTTP client for the browser and Node.js." [Online]. Available: <https://axios-http.com/>. [Truy cập: 15-05-2026].

- [14] Chart.js. (2024). "Chart.js – Simple yet flexible JavaScript charting library." [Online]. Available: <https://www.chartjs.org/>. [Truy cập: 15-05-2026].

- [15] Socket.IO. (2024). "Socket.IO – Bidirectional and low-latency communication for every platform." [Online]. Available: <https://socket.io/>. [Truy cập: 15-05-2026].

- [16] Auth0. (2024). "JSON Web Tokens (JWT) – An Introduction." [Online].

Available: <https://jwt.io/introduction>. [Truy cập: 15-05-2026].

- [17] bcryptjs. (2024). "bcryptjs – Optimized bcrypt in JavaScript." [Online]. Available: <https://github.com/nicolo-ribaudo/bcryptjs>. [Truy cập: 15-05-2026].

- [18] Google. (2024). "Google Generative AI – Gemini API Documentation." [Online]. Available: <https://ai.google.dev/docs>. [Truy cập: 15-05-2026].

- [19] Nodemailer. (2024). "Nodemailer – Send emails from Node.js." [Online]. Available: <https://nodemailer.com/>. [Truy cập: 15-05-2026].

- [20] Multer. (2024). "Multer – Node.js middleware for handling multipart/formdata." [Online]. Available: <https://github.com/expressjs/multer>. [Truy cập: 15-05-2026].

- [21] CORS. (2024). "cors – Node.js CORS middleware." [Online].

Available: <https://github.com/expressjs/cors>. [Truy cập: 15-05-2026].

- [22] dotenv. (2024). "dotenv – Loads environment variables from .env files." [Online]. Available: <https://github.com/motdotla/dotenv>. [Truy cập: 15-05-2026].

- [23] node-cron. (2024). "node-cron – A simple cron-like job scheduler for Node.js." [Online]. Available: <https://github.com/node-cron/node-cron>. [Truy cập: 15-052026].
- [24] Passport.js. (2024). "Passport – Simple, unobtrusive authentication for Node.js." [Online]. Available: <https://www.passportjs.org/>. [Truy cập: 15-05-2026].
- [25] html2pdf.js. (2024). "html2pdf.js – HTML to PDF converter using html2canvas and jsPDF." [Online]. Available: <https://github.com/eKoopmans/html2pdf.js>. [Truy cập: 15-05-2026].
- [26] Capacitor. (2024). "Capacitor – Cross-platform Native Runtime for Web Apps." [Online]. Available: <https://capacitorjs.com/>. [Truy cập: 15-05-2026].
- [27] Git. (2024). "Git – Distributed Version Control System." [Online]. Available: <https://git-scm.com/>. [Truy cập: 15-05-2026].
- [28] GitHub. (2024). "GitHub – Where the world builds software." [Online]. Available: <https://github.com/>. [Truy cập: 15-05-2026].
- [29] Postman. (2024). "Postman – The Collaboration Platform for API Development." [Online]. Available: <https://www.postman.com/>. [Truy cập: 15-052026].
- [30] Pressman, R. S. & Maxim, B. R. (2020). "Software Engineering: A Practitioner's Approach." 9th Edition, McGraw-Hill Education, ISBN 978-1-259-87297-6.
- [31] Sommerville, I. (2016). "Software Engineering." 10th Edition, Pearson Education, ISBN 978-0-13-394303-0.
- [32] Martin, R. C. (2017). "Clean Architecture: A Craftsman's Guide to Software Structure and Design." Prentice Hall, ISBN 978-0-13-449416-6.
- [33] Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994). "Design Patterns:

Elements of Reusable Object-Oriented Software." Addison-Wesley, ISBN 978-0201-63361-0.

- [34] OWASP Foundation. (2024). "OWASP Top Ten Web Application Security Risks." [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Truy cập: 15/05-2026].